



Use of Machine Learning techniques for flood forecasting

Valentin Mercier

► To cite this version:

Valentin Mercier. Use of Machine Learning techniques for flood forecasting. Physics [physics]. Université de Toulouse, 2025. English. ⟨NNT : 2025TLSEP016⟩. ⟨tel-05111069⟩

HAL Id: tel-05111069

<https://theses.hal.science/tel-05111069v1>

Submitted on 13 Jun 2025

HAL is a multi-disciplinary open access archive for the deposit and dissemination of scientific research documents, whether they are published or not. The documents may come from teaching and research institutions in France or abroad, or from public or private research centers.

L'archive ouverte pluridisciplinaire **HAL**, est destinée au dépôt et à la diffusion de documents scientifiques de niveau recherche, publiés ou non, émanant des établissements d'enseignement et de recherche français ou étrangers, des laboratoires publics ou privés.



HAL Authorization

Doctorat de l'Université de Toulouse

préparé à Toulouse INP

Utilisation de techniques de Machine Learning pour la
prédiction de crue

Thèse présentée et soutenue, le 4 avril 2025 par

Valentin MERCIER

École doctorale

EDMITT - Ecole Doctorale Mathématiques, Informatique et Télécommunications de Toulouse

Spécialité

Informatique et Télécommunications

Unité de recherche

IRIT : Institut de Recherche en Informatique de Toulouse

Thèse dirigée par

Serge GRATTON

Composition du jury

Mme Hélène ROUX, Présidente, Toulouse INP

M. Patrick GALLINARI, Rapporteur, Sorbonne Université

M. Ronan FABLET, Rapporteur, IMT Atlantique

M. Nicolas THOME, Examineur, Sorbonne Université

M. Corentin LAPEYRE, Examineur, Nvidia

M. Serge GRATTON, Directeur de thèse, Toulouse INP

Membres invités

Mme Marie-Christine Germain, BRLi

M. Philippe Toint, Université de Namur

Abstract

Climate change is intensifying extreme precipitation events, underscoring the critical need for accurate and rapid flood forecasting to protect lives, infrastructure, and the environment. In France alone, annual flood-related damages average 520 million euros, emphasizing the urgency of developing more efficient predictive tools. Traditional flood forecasting methods rely on computationally intensive physical models based on shallow water equations, which are often too slow for real-time decision-making, especially in urban settings where rapid responses are essential. This research explores and advances two complementary deep learning paradigms for flood forecasting, reflecting an adaptive and innovative research trajectory. Initially, we focused on Physically-Informed Neural Networks (PINNs) due to their inherent physical constraints and data-efficient nature. Although PINNs showed significant promise and led to meaningful advancements (such as multi-level hierarchical optimization strategies that enhanced convergence and accuracy) scaling them to the shallow water equations proved challenging despite substantial methodological improvements. Innovations like multi-level hierarchical optimization strategies, frequency-informed mechanisms, and domain decomposition algorithms addressed key limitations and advanced the field, positioning PINNs as a valuable tool for solving canonical PDEs like the Poisson and heat equations. In parallel, we pursued an alternative approach using Graph Neural Networks (GNNs) to create surrogate models for flood simulation. This pivot leveraged real flood data, enabling us to produce operationally relevant solutions. Applied to the Têt River basin in southern France, these GNN models demonstrated a great computational efficiency (orders of magnitude faster than traditional numerical methods) while maintaining a correct predictive accuracy. By effectively capturing both gradual water level changes and sudden flood events, they proved suitable for real-time applications in emergency response scenarios. This work establishes a foundation for hybrid physics-ML approaches to hydrological modelling and advances the field in several key aspects. The developed models provide operationally relevant solutions for flood forecasting, balancing the physical accuracy of traditional methods with the efficiency and responsiveness required for emergency response. Future research directions include real-time model adaptation capabilities, robust uncertainty quantification, and seamless integration with existing warning systems. Together, these efforts represent a significant step towards more reliable and responsive flood forecasting systems, which are essential for adapting to the challenges of climate change.

Key words : Flood forecasting, Deep learning, Computational physics, PINN, GNN

Résumé

Le changement climatique favorise les précipitations extrêmes, soulignant le besoin critique de prévisions précises et rapides des inondations afin de protéger les vies humaines, les infrastructures et l'environnement. En France seulement, les dommages annuels liés aux inondations s'élèvent en moyenne à 520 millions d'euros, ce qui souligne l'urgence de développer des outils de prévision plus efficaces. Les méthodes traditionnelles de prévision des inondations reposent sur des modèles physiques à forte intensité de calcul basés sur les équations de Barré de Saint-Venant qui sont souvent trop lents pour permettre une prise de décision en temps réel, en particulier dans les zones urbaines où des réponses rapides sont essentielles. Cette thèse explore et présente deux paradigmes complémentaires d'apprentissage profond pour la prévision des inondations, reflétant une trajectoire de recherche adaptative et innovante. Au départ, nous nous sommes concentrés sur les réseaux de neurones informés par la physique (PINN) en raison de leur respect de la physique inhérent et de leur nature peu dépendante des données. Bien que les PINN soient très prometteurs et qu'ils aient permis des avancées significatives (comme les stratégies d'optimisation hiérarchique à plusieurs niveaux qui ont amélioré la convergence et la précision), leur transposition aux équations de Barré de Saint-Venant s'est avérée difficile malgré des améliorations méthodologiques substantielles. Les innovations comme les stratégies d'optimisation hiérarchique à plusieurs niveaux, les mécanismes informés par la fréquence et les algorithmes de décomposition de domaine ont permis de remédier à des limitations importantes et de faire progresser le domaine, en positionnant les PINN comme un outil précieux pour résoudre les EDP canoniques comme les équations de Poisson et de la chaleur. Parallèlement, nous avons poursuivi une autre approche en utilisant des réseaux de neurones en graphes (GNN) pour créer des modèles de substitution pour la simulation des inondations. Ce volet s'appuie sur des données d'inondation réelles, ce qui nous permet de produire des solutions pertinentes d'un point de vue opérationnel. Appliqués au bassin de la Têt dans le sud de la France, ces modèles GNN ont démontré une grande efficacité de calcul (plusieurs ordres de grandeur plus rapides que les méthodes numériques traditionnelles) tout en conservant une précision raisonnable. En capturant efficacement les variations de niveau d'eau et les événements d'inondation soudains, ils se sont révélés adaptés aux applications en temps réel dans les scénarios d'intervention d'urgence. Ce travail établit une base pour les approches hybrides physique-ML dans la modélisation hydraulique tout en faisant progresser le domaine dans plusieurs aspects clés. Les modèles développés fournissent des solutions opérationnelles pour la prévision des inondations, proposant un juste milieu entre la précision physique des méthodes traditionnelles avec l'efficacité et la réactivité requises pour les interventions d'urgence. Les orientations futures de la recherche comprennent des capacités d'adaptation du modèle en temps réel, une quantification robuste de l'incertitude et une intégration transparente avec les systèmes d'alerte existants. L'ensemble de ces efforts marque une étape importante vers des systèmes de prévision des inondations plus fiables et plus réactifs, essentiels pour s'adapter aux défis posés par le changement climatique.

Mots clés : Prévision de crue, Apprentissage profond, Physique numérique, PINN, GNN

Remerciements

Avant de se lancer dans ce manuscrit, je me dois de remercier tous les gens qui ont rendu ça possible.

Je tiens tout d’abord à remercier mes deux encadrants, Serge Gratton et Corentin Lapeyre, sans qui cette thèse n’aurait jamais vu le jour. Serge, qui m’a transmis le goût de la recherche dès les bancs de l’ENSEEIH, et qui m’a ensuite offert la chance d’y apporter ma pierre à travers cette thèse. Merci pour ton accompagnement constant, ta bienveillance, ainsi que pour tes remarques toujours justes et constructives. Corentin, dont l’expérience pratique, le dynamisme et l’enthousiasme ont été des moteurs essentiels dans l’avancée de mes travaux. Tu as su rendre ces années de recherche à la fois stimulantes et joyeuses, et m’as permis d’atteindre des résultats que je n’aurais jamais imaginés.

Merci à Patrick Gallinari et Ronan Fablet d’avoir accepté de consacrer du temps pour lire mon manuscrit et évaluer mon travail. Je tiens aussi à remercier Hélène Roux et Nicolas Thome pour ces échanges enrichissant lors de ma soutenance.

Je souhaite également remercier Philippe Toint et Elisa Riccietti, pour leur collaboration précieuse depuis Namur et Lyon, et pour tout ce qu’ils m’ont permis d’apprendre et de développer au fil de nos échanges.

Un grand merci à toute l’équipe HSL de BRLi : Marie-Christine Germain, Antonin Mazoyer, Emmanuel d’Ervau, Fabrice Cebron et Gwenaël Chevalier, pour m’avoir accueilli chaleureusement, accompagné avec patience dans la découverte des métiers de l’hydraulique, et intégré avec naturel à la vie de l’équipe.

Je ne saurais oublier l’autre vie d’équipe, celle de l’équipe APO : Antoine Bernigaud, Antoine Jégo, Bastien, Boris, Olivier, Rémy, Jérémy, Sadok, Sophie, Théo bientôt remplacés par la jeune génération : Jordan, Karmijn, Mathieu... et certainement d’autres que j’oublie. Merci à vous tous pour la bonne humeur quotidienne et l’ambiance si agréable qui a rythmé ces années. Merci à Sadok et Théo, mes compagnons de bureau, pour nos débats sans fin, qu’il s’agisse de football ou de poulet de Bresse. Je regrette simplement de ne pas avoir réussi à vous convaincre de la supériorité du PSG sur les Reds ou les Gunners. Une mention spéciale à Antoine Bernigaud pour les repas partagés

devant South Park, l'initiation à la salle de sport et les discussions sans fin. Merci aussi à Maxime Tavian pour sa compagnie fidèle durant ces années à l'ENSEEIH et sa sérénité communicative.

Je remercie également tous mes amis de prépa : Lucas, Fabien, Mathis, Alexy, Nacho, Daria, Manon, Laura, Léa, Guilhem, Loïc, Amélie C., Amélie P, pour leur indéfectible soutien et pour avoir toujours cru en moi. Merci Fabien de m'avoir écouté parler d'intelligence artificielle pendant des heures sans jamais souffler. Merci Lulu pour les cafés du matin qui donnaient envie de se lever avant d'aller travailler. En parlant de cafés du matin : merci Thibaut, pour ceux partagés mais surtout pour tout ce temps passé ensemble depuis que tu as débarqué d'Avignon en 2002.

Merci à mes tantines et à mes bougs de la ban'zil : Marion, Layla, Louis, Julien, Nicolas, Patrick, pour avoir illuminé mon quotidien par leur bonne humeur, leur énergie et tous les fous rires partagés.

Je remercie tous les amis qui m'entourent au quotidien ou de loin : Daniel, Simon, Victor, Jacques, Enzo, Baptiste, Adrien, Thibault, Timothé, Léo, Quentin, Quentin, Hugo, Clara, Maéva, Paul, Romane, Léa, Mathilde, Manon, Agathe, Émilie, Cédric. Je ne saurais en distinguer un plus qu'un autre, tant chacune de ces relations m'est précieuse et unique. Merci à vous tous de me rappeler, chaque jour, la place centrale de l'amitié dans une vie.

Je remercie évidemment mes parents pour la vie douce, pleine d'amour, de bienveillance et d'épanouissement qu'ils m'ont offerte depuis ma naissance. Je suis profondément conscient du privilège d'avoir grandi dans un foyer aussi aimant et sécurisant. Merci de m'avoir transmis la curiosité et la confiance en moi qui m'ont guidé jusqu'à cette thèse.

Merci à mon frère, mon plus ancien compagnon de route, pour son soutien indéfectible. Depuis l'enfance, tu as été à mes côtés dans tous les moments importants, ton soutien m'a porté bien plus souvent que tu ne l'imagines.

Je tiens aussi à remercier mes grands-parents, Marcelle, Suzanne, Rémy et Moïse, pour l'amour qu'ils me donnent chaque jour et qu'ils m'ont toujours donné tout au long de ma vie. Merci de m'avoir appris la pêche, la pétanque, la cuisine, à cultiver un potager, à faire du pain, et transmis le sens du travail. Merci de continuer à me nourrir régulièrement le midi, et de m'offrir, à travers votre présence, une image apaisante de la vieillesse. J'espère qu'un jour, lorsque j'en aurai l'âge, je saurai être un grand-parent comme vous.

Une autre famille, plus récente celle-là, m'a aussi accompagné au fil des années : Olivier, Françoise, Alexandre. Avec le temps, vous êtes aussi devenus ma famille. Votre présence, votre bienveillance et votre soutien m'ont énormément apporté tout au long de ces huit

dernières années. Merci, sincèrement, d'avoir été là.

Enfin, je tiens à remercier la personne la plus importante de ma vie : Delphine. Merci de m'avoir supporté pendant ces trois dernières années (même si, soyons honnêtes, les cinq précédentes n'ont sans doute pas été de tout repos non plus). Si tu n'étais pas entrée dans ma vie avec Marin et Hiboude, je ne serais sûrement pas l'homme que je suis aujourd'hui. Merci pour ton soutien indéfectible, ta douceur, ta patience, et toutes ces qualités sans lesquelles je n'aurais jamais pu mener cette thèse à son terme. Je n'aurai jamais assez de place dans un seul manuscrit pour dire tout l'amour que j'ai pour toi. Alors je vais m'appliquer à te le montrer chaque jour, tout au long de notre vie. Cette thèse t'est dédiée.

Contents

Abstract	i
Résumé	ii
Remerciements	iii
Contents	1
List of Figures	2
List of Tables	3
1 Introduction	5
1.1 The challenges of flood forecasting	5
1.2 Deep Learning	6
1.3 Organization of the thesis	7
I State-of-the-Art	10
2 Deep Learning	11
2.1 Deep Learning in a nutshell	11
2.1.1 Supervised Learning	11
2.1.2 Loss Function	12
2.1.3 Deep Neural Networks	12
2.1.4 Training Neural Networks	12
2.1.5 GPUs	13
2.1.6 Some Interesting Architectures	13
2.2 Deep Learning for Solving Partial Differential Equations	19
2.2.1 Classification	20
2.2.2 Machine learning-assisted numerical simulations	20
2.2.3 Data-driven surrogates	24
2.2.4 Physics informed machine learning	29

3	Numerical simulations for Free Surface Flows	35
3.1	A Sketch of Numerical Methods	35
3.2	Free Surface Flows theoretical modeling	37
3.2.1	Notations and geometry	37
3.2.2	Saint-Venant equations	37
3.2.3	Boundary conditions	39
3.3	Telemac2D	40
3.3.1	An overview of the method	40
3.3.2	Advection Step	41
3.3.3	Propagation, diffusion, source terms	42
3.3.4	Finite elements	43
3.3.5	Linear system resolution	44
3.3.6	Treatment of uncovered bed	44
3.4	Practical Application	45
3.4.1	Pre-processing	46
3.4.2	Numerical modeling	47
3.4.3	Post-processing	48
3.4.4	Notes on flood alert system	48
4	Deep Learning surrogates for Geosciences	50
4.1	Surrogate Models for Weather Forecasting	50
4.1.1	FourCastNet	51
4.1.2	GraphCast	52
4.1.3	Pengu-Weather	53
4.1.4	Recent developments	54
4.2	Modern application to Flood-prediction	56
4.2.1	Shallow Water Equation–Graph Neural Network	57
4.2.2	FloodSformer	58
4.2.3	GeoPINS	59
II	Physics informed neural networks contributions	61
5	A Block-Coordinate Approach of Multi-level Optimization with an Application to Physics-Informed Neural Networks	64
5.1	Introduction	64
5.2	A block-coordinate perspective on Galerkin multilevel optimization	66
5.3	A generic BCD algorithm and its convergence	70
5.4	Application to Physics-Informed Neural Networks	76
5.4.1	Alternating training in a hierarchical context	76
5.5	A “frequency-aware” distributed approach	81
5.5.1	The F-Principle	81
5.5.2	A frequency-aware network architecture	82
5.6	Numerical results	85

5.6.1	Poisson problems	85
5.6.2	The heat equation	90
5.7	Conclusion and perspectives	91
6	A coarse space acceleration of deep-DDM	93
6.1	Introduction	93
6.2	The Deep Domain Decomposition Method (Deep DDM)	94
6.2.1	Basics of domain decomposition	94
6.2.2	Deep DDM	95
6.3	Extension via a Coarse Network	98
6.4	Numerical Results	99
6.5	Conclusion	101
	Conclusion	103
III	Flood-prediction	104
7	Graph neural networks for flood alert system	105
7.1	Introduction	105
7.2	Dataset Creation	106
7.2.1	Initial and boundary conditions	107
7.2.2	Generating Realistic Floods	108
7.3	Graph neural networks for flood alert system	108
7.3.1	Model basis	108
7.3.2	Multimesh creation	112
7.3.3	Boundary conditions	112
7.3.4	Loss Function	113
7.3.5	General training Setup	113
7.3.6	Metrics	114
7.4	Numerical results	115
7.4.1	Multimesh MeshGraphNet	115
7.4.2	Overflow detection	121
7.4.3	Frequential analysis	126
7.5	Conclusions and Perspectives	129
7.5.1	Future Directions	129
8	Conclusion and perspectives	131
8.1	Conclusion on the contributions of this thesis	131
8.2	Perspectives for future research	132

IV	Annexes	135
	Appendix of Paper 1	136
8.3	Proofs of convergence theorems	136
8.3.1	Proof of Theorem 5.3.1	136
8.3.2	Proof of Theorem 5.3.2	137
8.4	A Mscale FAML variant and its performance	138
8.4.1	Another frequency-aware architecture	138
8.4.2	Numerical results	139
	Appendix of Chapter 4	141
8.5	Telemac .cas file	141
8.6	Speed measurements	143
8.6.1	Multimesh MeshGraphNet	143
8.6.2	Overflow detection	143
	Bibliography	145

List of Figures

1.1	Visualisation of the Têt study area during the BRLi mission.	6
2.1	GPU 2022 benchmark from lambdalabs comparing Training throughput accross several models ranging from resnet50 to BERT from lambdalabs.com	14
2.2	A sample from the Modified National Institute of Standards and Technology (MNIST) database used by LeCun et al. [1998]	15
2.3	Illustration of the functioning of convolutional layers	16
2.4	Error rate on the test set from LeCun et al. [1998], for further details on each implementation, please refer to the original reference.	17
2.6	Classification of deep learning methods used in this document, linked to the corresponding section.	21
2.7	Illustration of the hybrid solver method presented in Bar-Sinai et al. [2019]	22
2.8	Evolution of the number of physics research articles mentioning the terms 'AI model' and 'surrogate model' over the years in the PORTICO and JSTOR databases (generated using constellate.org).	24
2.9	An example of pressure groundtruth and prediction of the cylinder flow test case in Pfaff et al. [2020]	27
2.10	Illustration of the DeepONet architecture	28
2.11	An example of Fourier Layer from Li et al. [2020c] with $\nu(x)$ the input, $\mathcal{F}$ the fourier transform, W and R the linear transform and σ the activation function	29
3.1	Calculation domain	38
4.1	FourCastNet architecture detailed from Pathak et al. [2022]	51
4.2	Illustration of the multi-mesh from Lam et al. [2022].	52
4.3	Illustration of the encode/process/decode of Graphcast from Lam et al. [2022].	53
4.4	Pangu architecture detailed from Bi et al. [2023]	54
5.1	Evolution of the MSE as a function of the computational cost for our different models	80

5.2	Evolution of the loss as a function of the computational cost for our different models	80
5.3	Results of our method on the Poisson problem (6.2.2) for different frequencies. The method indicated as FAML (Frequency-Aware-Multilevel) is the method proposed in the forthcoming Section 5.5.	80
5.4	(a) The input function and its Fourier coefficients as inset, (b) Relative error between the DNN and the true Fourier coefficients per epoch for the three main frequencies. From Xu et al. [2019]	82
5.5	An example of P-WWP architecture	84
5.6	Test problem 1 (cf. (6.2.2), (5.20)). Left: the domain Ω and its subdomains. Center: the solution u . Right: the source term r	85
5.7	Test problem 2 (cf. (6.2.2), (5.21)). Left: the domain Ω and its subdomains. Center: the solution u . Right: the source term r	86
5.8	Test problems 3 (cf. (6.2.2), (5.22)) and 4 (cf. (6.2.2), (5.23)). Left: the domain Ω and its subdomains. Center: the solution u for Test Problem 3. Right: the solution u for Test Problem 4.	87
5.9	Comparison of average MSE values (from Table 5.2)	89
5.10	Evolution of MSE and loss values for a typical run on Test problem 3 . . .	90
5.11	Comparison of average MSE values (from Table 5.3)	91
6.1	The diagram used by Schwarz in 1870.	94
6.2	Sampling points for the two-level Deep-DDM method	96
6.3	Sampling points for the two-level Deep-DDM method	97
6.4	Strong scalability test for test problem with $\omega_1 = 1, \omega_2 = 3$: Test with 2500 epoch	100
6.5	An example of sampling with a 6×6 decomposition	101
6.6	Strong scalability test for test problem with $\omega_1 = 1, \omega_2 = 6$ with 2500 epoch	102
6.7	Testor test problem with $\omega_1 = 1, \omega_2 = 6$ on 6×6 decomposition with variation in the number of epochs	102
7.1	Density map used to create the mesh used for simulations on the river Têt.	107
7.2	The four groups of hydrographs obtained by clustering with the shape coefficient. In color the normalized historical hydrographs and in black the mean hydrogram.	109
7.3	Transforming Telemac mesh data into data that can be interpreted by our MeshGraphNet. The colours on the nodes indicate different types of node (various boundary conditions or interior nodes).	110
7.4	Summary of the different stages of the MeshGraphNet method. Steps (a) encoding data to a latent space on nodes and edges using two separate MLPs. Steps (b) and (c) message passing, aggregation and updating of values on the nodes, these steps are repeated as many times as there are message passing layers. Step (d) decoding values at nodes to physical space and updating.	111

7.5	Visualization of the various meshes generated for multi-mesh creation, focused on a zoomed-in section of the domain.	113
7.6	Speedup of the training code with respect to the number of nodes used on the entire training dataset (47 flood events).	114
7.7	Evolution of L1 error on h (blue) and CSI (green) on test floods as a function of the number of unrolling steps	116
7.8	An example of a three-hour prediction from our model where the overflow is missed. Here we show a flood map with the areas with a water height greater than 5cm in blue. From left to right we have the starting point of the prediction, the prediction of our model and our groundtruth generated with Telemac 2D.	116
7.9	Initial wetted zone polygon used as a mask.	117
7.10	From bottom to top: CSI_{over} calculated at a prediction horizon of six hours at all possible time steps of the test flood with a peak flow of $1600\text{m}^3/\text{s}$. Change in wetted area of at least five centimetres for each of the time steps at which the CSI_{over} is calculated. Illustrations of the area flooded by at least 5 cm of water at different times during the flood.	118
7.11	From bottom to top: CSI_{over} calculated at a prediction horizon of six hours at all possible time steps of the test flood with a peak flow of $2200\text{m}^3/\text{s}$. Change in wetted area of at least five centimetres for each of the time steps at which the CSI_{over} is calculated. Illustrations of the area flooded by at least 5 cm of water at different times during the flood.	119
7.12	From bottom to top: CSI_{over} calculated at a prediction horizon of six hours at all possible time steps of the test flood with a peak flow of $3200\text{m}^3/\text{s}$. Change in wetted area of at least five centimetres for each of the time steps at which the CSI_{over} is calculated. Illustrations of the area flooded by at least 5 cm of water at different times during the flood.	120
7.13	Metrics for each model on a regular grid	122
7.14	Comparison of floodmaps produced by each model and the ground-truth reference for test flood with a peak flow of $3200\text{m}^3/\text{s}$. All predictions are unrolled starting from the first timestep of the reduce dataset.	122
7.15	From bottom to top: CSI_{over} calculated for all the trained models at a prediction horizon of six hours at all possible time steps of the test flood with a peak flow of $1600\text{m}^3/\text{s}$. Change in wetted area of at least five centimetres for each of the time steps at which the CSI_{over} is calculated. Illustrations of the area flooded by at least 5 cm of water at different times during the flood.	123
7.16	From bottom to top: CSI_{over} calculated for all the trained models at a prediction horizon of six hours at all possible time steps of the test flood with a peak flow of $2200\text{m}^3/\text{s}$. Change in wetted area of at least five centimetres for each of the time steps at which the CSI_{over} is calculated. Illustrations of the area flooded by at least 5 cm of water at different times during the flood.	124

7.17	From bottom to top: CSI_{over} calculated for all the trained models at a prediction horizon of six hours at all possible time steps of the test flood with a peak flow of $3200\text{ m}^3/\text{s}$. Change in wetted area of at least five centimetres for each of the time steps at which the CSI_{over} is calculated. Illustrations of the area flooded by at least 5 cm of water at different times during the flood.	125
7.18	Mean frequency spectra of the predicted water heights, averaged over all timesteps in the test set.	127
7.19	Semi-log FTT for different time slices of the $3200\text{ m}^3/\text{s}$ peak flow test flood.	128
8.1	The s2Relu activation function (left) and its Fourier transform for several scalings (right). The black stars highlight the amplitude peaks.	139
8.2	Evolution of L1 error on u (blue), v (red) and CSI (green) on test floods as a function of the number of unrolling steps under the same conditions as Fig. 7.7	143
8.3	L_1 error for u across each starting time step in each test floods, evaluated for different prediction horizons and models under the same conditions as Fig. 7.13.	143
8.4	L_1 error for v across each starting time step in each test floods, evaluated for different prediction horizons and models under the same conditions as Fig. 7.13.	144

List of Tables

2.1	Overview of Data-driven Surrogates methods, the methods in blue are discussed in subsection 2.2.4 and the methods in green in Chapter 4	25
5.1	Network architecture for the experiments with alternating training in the hierarchical context	78
5.2	Best value of the median loss for the given budget and associated MSE for standard single-level and FAML training (10 independent runs)	88
5.3	Best median of the loss for the given budget and associated MSE for standard single-level and FAML training (10 independent runs)	91
6.1	Median of the wall time for the experiments of the figure 6.7	100
7.1	Minimum, average and maximum distance in metres between two connected points on the graph with the number of mesh nodes. The multi-meshes of a chosen mesh contain the edges created by all the coarser meshes according to the detailed method.	113
7.2	Inference speed (in seconds) for a one-step (30 minutes) prediction, for each model resolution.	121
8.1	Details of the parallel Mscale architecture	139
8.2	Best value of the median loss and associated MSE for standard single-level and Mscale-FAML training (10 independent runs)	140

Chapter 1

Introduction

1.1 The challenges of flood forecasting

The study of floods, characterised by a significant increase in the flow and water level of rivers, is a subject of vital importance in environmental and hydrological research. These phenomena, in which water flows beyond the limits of the minor bed to extend beyond the banks, thus defining flood zones, play a crucial role in natural ecosystems. Flooding is essential to biodiversity. The historical example of the ancient Egyptian calendar divided into three seasons: Akhet (Flood), Peret (Emergence of the land), and Chémou (Heat) illustrates their importance perfectly.

However, in today's context of intensive urbanisation of coastal areas, these natural events often turn into human, economic and environmental disasters. Loss of life, material damage, pollution and soil contamination are just some of the challenges facing the authorities responsible for river management.

Against a dramatically changing climate, with periods of drought alternating with episodes of intense rainfall, flash floods are becoming increasingly frequent. Forecasting and managing these phenomena are therefore becoming major issues of our time.

Flood modelling is a multidisciplinary challenge requiring the integration of a variety of skills from different professional fields. The process begins with the collection of hydrological data upstream of the river section targeted for modelling. In this context, the main objective of hydrology is to transform complex data, such as soil conditions, rainfall and snowmelt, into a hydrograph. The hydrograph provides a temporal representation of water flow, a key element in characterising the flood under study.

Next, it is necessary to model the topology of the river section concerned. This involves taking into account various factors such as the height of the ground, its characteristics (for example, the coefficient of friction) and the surrounding urban infrastructure such as dykes or medians. This stage also requires the creation of a mesh adapted to the chosen modelling.

Finally, the use of calculation codes based on the Barré de Saint-Venant equations, which describe the flow of water in a river, is essential. These codes, such as [Hervouet \[2007\]](#) and [Brunner \[1997\]](#), are based on advanced numerical methods, such as finite



Figure 1.1: Visualisation of the Têt study area during the BRLi mission.

elements and finite volumes, enabling precise and detailed modelling of flood phenomena.

Here is an example to illustrate this methodology. BRL Ingénierie developed a Telemac-2D model [Hervouet, 2007] for the lower catchment of the River La Têt (see Fig. 1.1), near Perpignan, to simulate various flood scenarios. By integrating detailed topographic data, such as LiDAR and cross-sectional surveys, alongside small-scale features, they produced high-resolution inundation maps. These maps illustrate how rising water levels at a critical point of interest, the Joffre Bridge, propagate flooding across the downstream plain. Given the computational intensity of running such fine-scale simulations in real time, the calculations were pre-processed to generate a library of pre-computed inundation maps.

In fact, even though these flood models are highly accurate, they present significant challenges, particularly in terms of calculation costs. Their accuracy is often achieved at the cost of integrating small structures into the mesh (see for example Moulinec et al. [2011], Orozco et al. [2023], Gharbi et al. [2016], Pathan et al. [2022]), which increases the number of points required for calculations. In addition, the time step for solving the equations is directly linked to the size of the smallest mesh in the mesh, leading to a further increase in calculation costs.

Despite the use of supercomputers and the application of parallelism techniques to solve the equations, these calculations can be extremely costly in terms of resources and time [Moulinec et al. 2011]. This constraint represents a major challenge, particularly in the context of unexpected flash floods, where responsiveness and rapid exploitation of data are crucial. It is in this quest for responsiveness to floods that the subject of my thesis originates.

1.2 Deep Learning

Over the last decade, deep learning has emerged as a cornerstone of artificial intelligence and machine learning, driving transformative advancements across diverse scientific disciplines. By training models on large datasets and leveraging significant computational resources, deep learning systems can achieve impressive performance. Once trained, these models perform predictions efficiently and quickly, making them ideal for real-time applications and large-scale deployment. Deep learning has demonstrated remarkable success in fields such as computer vision, natural language processing, and weather forecasting, where its ability to learn representations, recognize patterns, and make accurate predictions has exceeded traditional methods. In computer vision, deep learning has addressed the limitations of conventional image processing techniques. The advent of deep convolutional neural networks marked a turning point, with [Krizhevsky \[2014\]](#) achieving groundbreaking results in the ImageNet Large Scale Visual Recognition Challenge. This milestone spurred further innovations, including Vision Transformers [[Dosovitskiy, 2020](#)], which use self-attention mechanisms to enhance flexibility and performance in image analysis. Similarly, in natural language processing, deep learning has revolutionized the field by shifting from traditional feature engineering to end-to-end learning. Recurrent neural networks enabled the modeling of sequential data with long-range dependencies, while transformer architectures like BERT and GPT-3 [[Kenton and Toutanova, 2019](#), [Brown, 2020](#)] significantly advanced natural language processing capabilities. These models leverage unsupervised pre-training on vast text corpora to learn powerful language representations, achieving state-of-the-art results in tasks such as machine translation, question answering, and text generation. The scaling of parameters, as exemplified by GPT-3 with its 175 billion parameters, has further driven performance, enabling few-shot learning and efficient inference for real-time applications like conversational AI. Deep learning has also shown substantial potential in weather forecasting by utilizing high-resolution atmospheric datasets like ERA5 from the European Centre for Medium-Range Weather Forecasts. Models such as FourCastNet, GraphCast, and Pangu-Weather [[Pathak et al., 2022](#), [Lam et al., 2022](#), [Bi et al., 2023](#)] employ innovative architectures, including Adaptive Fourier Neural Operators, graph neural networks, and three-dimensional transformers, to capture spatial and temporal dependencies in atmospheric data. These models can generate global forecasts for several days ahead in seconds or minutes, vastly outperforming traditional numerical methods in speed. Building on these advancements, the application of machine learning to flood prediction presents a promising opportunity. Flooding, as a complex and dynamic phenomenon, demands robust, real-time predictive systems to mitigate risks effectively. Machine learning offers a data-centric paradigm, where models are trained on extensive datasets during an initially computationally intensive phase but operate with significantly lower computational cost during inference. Early studies, such as those cited in [Bentivoglio et al. \[2022\]](#), have laid the foundation for this research area, which continues to evolve with advances in data availability and model sophistication.

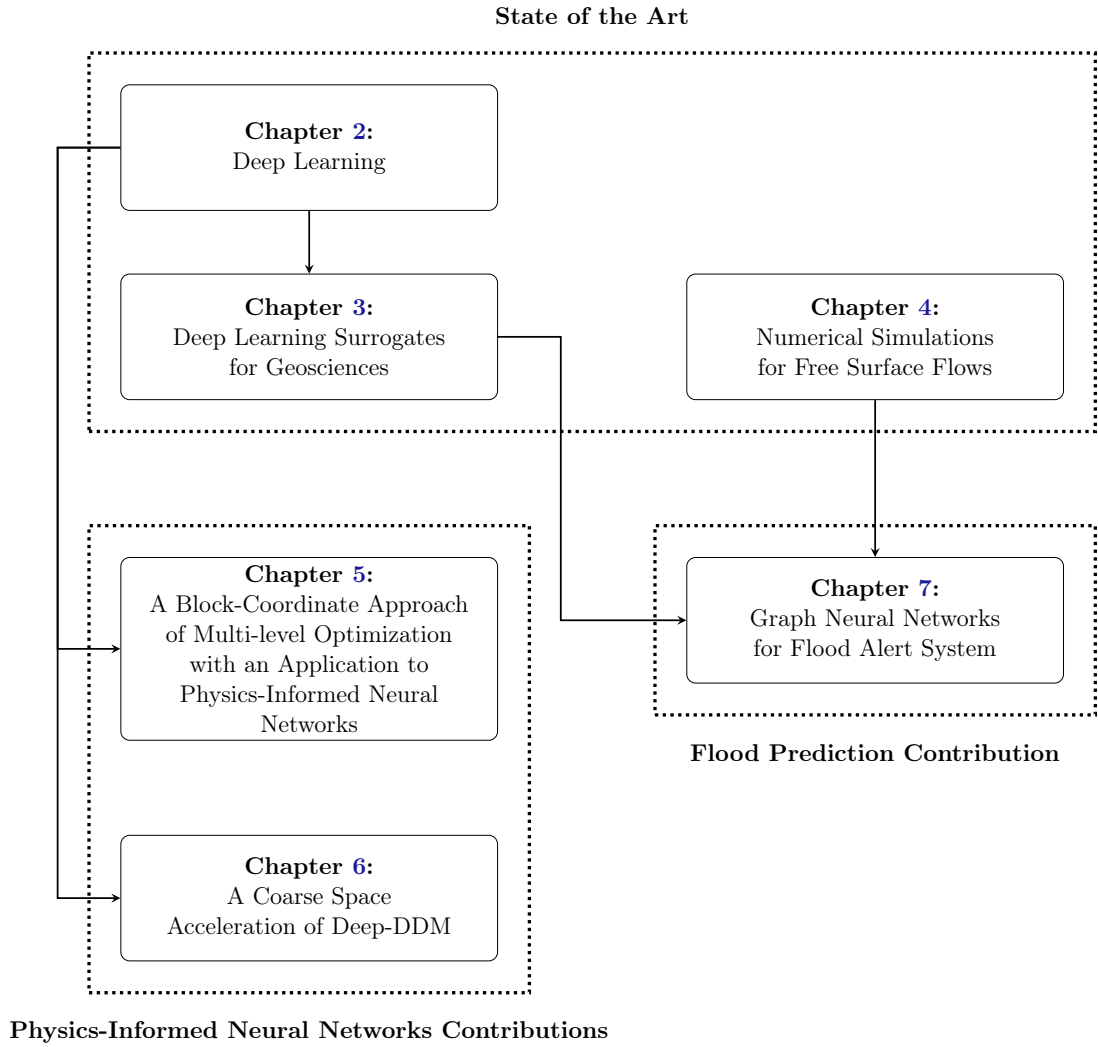
1.3 Organization of the thesis

In this PhD thesis, we aim to address the following question: "How can deep learning be applied to flood prediction?" The document is organized into three parts.

Part 1, comprising Chapters 2, 3, and 4, lays the foundational concepts. Chapter 2 introduces deep learning and its various applications for solving partial differential equations. Chapter 3 provides an overview of numerical methods, introduces the shallow water equations used to model floods, and details the numerical techniques employed by BRLi for flow simulation. Additionally, it presents the workflow used by the BRL engineering team for river modeling. Chapter 4 reviews recent advancements in deep learning for geosciences, with a particular focus on weather prediction and flood forecasting.

Part 2, consisting of Chapters 5 and 6, focuses on the development of physics-informed neural networks (PINNs) to improve their training and application to solving partial differential equations. The research on PINNs presented in Chapters 5 and 6 introduced several methodological innovations, including multi-level hierarchical optimization strategies, frequency-informed mechanisms, and domain decomposition algorithms. Importantly, the work on multi-level hierarchical optimization strategies and frequency-informed mechanisms resulted in the publication of a peer-reviewed paper, while the domain decomposition algorithms led to the submission of a proceeding article.

Part 3, presented in Chapter 3, explores the application of Graph Neural Networks as surrogate models to address an industrial flood prediction problem on the Têt River in southern France. The GNN models developed in this chapter leveraged real flood data to provide operationally relevant solutions. This work led to the development of practical code, which is now actively used by BRL for flood prediction and management.



Part I

State-of-the-Art

Chapter 2

Deep Learning

2.1	Deep Learning in a nutshell	11
2.1.1	Supervised Learning	11
2.1.2	Loss Function	12
2.1.3	Deep Neural Networks	12
2.1.4	Training Neural Networks	12
2.1.5	GPUs	13
2.1.6	Some Interesting Architectures	13
2.2	Deep Learning for Solving Partial Differential Equations	19
2.2.1	Classification	20
2.2.2	Machine learning-assisted numerical simulations	20
2.2.3	Data-driven surrogates	24
2.2.4	Physics informed machine learning	29

This chapter provides an essential foundation in deep learning, a subset of machine learning, to support the topics discussed throughout this thesis. We begin by reviewing the mathematical foundations of deep learning. We then examine several prominent models often cited in the literature, outlining their structures, strengths and limitations. While the concepts introduced here are intended to provide the necessary background for understanding this thesis, readers interested in a deeper exploration of these concepts are encouraged to consult the following references: [Goodfellow \[2016\]](#), [Bishop \[2006\]](#), [Hastie et al. \[2009\]](#). The chapter concludes with an overview of deep learning applications specifically tailored to solving Partial Differential Equations (PDE), and discusses how these methods interface with traditional numerical simulation techniques. This introduction aims to provide the reader with the necessary conceptual tools to engage with the analyses and applications in the following chapters.

2.1 Deep Learning in a nutshell

The term Machine Learning refers to numerical methods that work differently from traditional methods. Whereas a classical method takes data as input to obtain an output,

Machine Learning takes data as input and provides a method that associates an output with given data. Thus, unlike traditional methods where the algorithm is explicitly defined by the user, in machine learning the algorithm 'learns' from the data provided. This ability to learn autonomously is a paradigm shift. The term Machine Learning includes Deep Learning methods, which use deep neural networks for their learning processes, as we will define below.

2.1.1 Supervised Learning

In this manuscript we will only encounter supervised learning. Supervised learning refers to learning that is performed on a labelled dataset. These labels (or outputs) represent the expected output of our algorithm. In this context, the algorithm learns to associate given inputs with their correct outputs. We can formalise this as follows: we refer to the training set as the set of input-output pairs $x \in \mathcal{X}$ and $y \in \mathcal{Y}$, denoted by $\mathcal{D} = \{(x^i, y^i)\}_i$ for i ranging from 1 to N , the number of elements in the data set. We assume that there is an unknown function ϕ that maps all inputs x to their outputs y . The goal of supervised learning is to learn a function $f : \mathcal{X} \rightarrow \mathcal{Y}$ such that $f \approx \phi$.

2.1.2 Loss Function

To evaluate the error between our model's prediction and the actual value, it is essential to define a function called the loss function. In this work, we focus on a regression context where the set of possible outputs, $\mathcal{Y}$, is a subset of $\mathbb{R}^p$. We will denote this loss function by $L : \mathcal{Y} \times \mathcal{Y} \rightarrow \mathbb{R}$. The higher the value of this function, the further away the model's prediction is from the actual value. The choice of this loss function is therefore crucial in formulating our optimisation problem, as it directly influences the performance and efficiency of the supervised learning model.

2.1.3 Deep Neural Networks

We are going to explore this by choosing the form of our function f to be a specific type of deep neural network: a feedforward neural network without skip connections. This type of network is defined by a sequence of linear and non-linear transformations. Each pair of linear (denoted by T) and non-linear (denoted by σ) transformations is called a layer, and we will denote the layer number by l . We will refer to the non-linear transformations as activation functions. Each of these transformations is parameterized by a vector w^l associated with it, the set of these vectors $w = \{w^l\}_{l=1}^L$ is called the parameter set of the neural network. A feedforward neural network has at least three layers: an input layer, an output layer, and at least one hidden layer. So we write:

$$f_w(x) = \sigma^L \circ T^L \circ \dots \circ \sigma^1 \circ T^1(x)$$

where $l \in [1, L]$ is the layer number. We denote the intermediate result at layer l by x^l and the dimension of this result by N^l .

While this form represents a basic feedforward neural network, it is important to note that deep neural networks encompass a wide variety of architectures. Many networks incorporate more complex features such as skip connections, recurrent layers, and sophisticated connectivity patterns. These variations are designed to address specific challenges and improve performance in various applications. In this introduction, we focus on the feedforward architecture as a foundational example to understand how neural networks work. More advanced architectures will be discussed in subsequent sections as appropriate.

2.1.4 Training Neural Networks

So far we have discussed training or learning, which in practice means applying optimisation methods. The combination of the chosen neural network, the data set and the loss function is an optimisation problem that we want to minimise. This can be formalised as follows

$$\min_w L(f_w(x), y)$$

over the entire data set. The defined problem is highly non-convex, which means that in practice we aim to find a local minimum. To achieve this, we use gradient descent methods, expressed by the formula $w_{t+\delta t} = w_t - \alpha \nabla L(f_w(x), y)$. Since the data set is often too large to compute the gradient in a single operation, we employ stochastic variants of gradient descent. These variants are often enhanced by techniques such as momentum and other refinements such as the Adam algorithm, as described in [Kingma \[2014b\]](#).

2.1.5 GPUs

As detailed in the following section, the first approximation proof of neural networks was made in 1989, and a concrete daily application was designed by [LeCun et al. \[1998\]](#) in 1998. So why has it taken so long for the scientific community to delve deeper? A key issue in deep learning is computational power. Neural networks require extensive matrix multiplication, which is highly parallelizable. Graphics Processing Units (GPUs), originally designed to accelerate computer graphics, are ideal for these tasks because of their design. Compared to CPUs, GPUs dedicate more transistors to arithmetic logic units and fewer to caches and flow control. With thousands of smaller, more efficient cores, GPUs can perform multiple operations simultaneously, greatly speeding up the training process. In contrast, CPUs have fewer cores optimised for sequential processing, resulting in slower performance for the repetitive calculations in neural network training. As a result, GPUs are the preferred choice for deep learning applications. As GPUs became more popular, frameworks were developed to efficiently train neural networks on them, facilitating the spread and popularity of deep learning. Today, specialised GPUs and tensor processing units (TPUs) are designed to work with deep learning frameworks. GPUs are still evolving rapidly; the following graph compares, among others, the GPUs used in the first part of this thesis (NVIDIA GeForce GTX 1080 Ti) and those used in

2.1. Deep Learning in a nutshell

the last part (Nvidia A100) Fig. 2.1. The chosen metric from this graph is the training throughput, which measures the number of samples (e.g. tokens, images, etc...) processed per second by the GPUs. The whole process is detailed on lambdalabs.com, the producers of the chart.

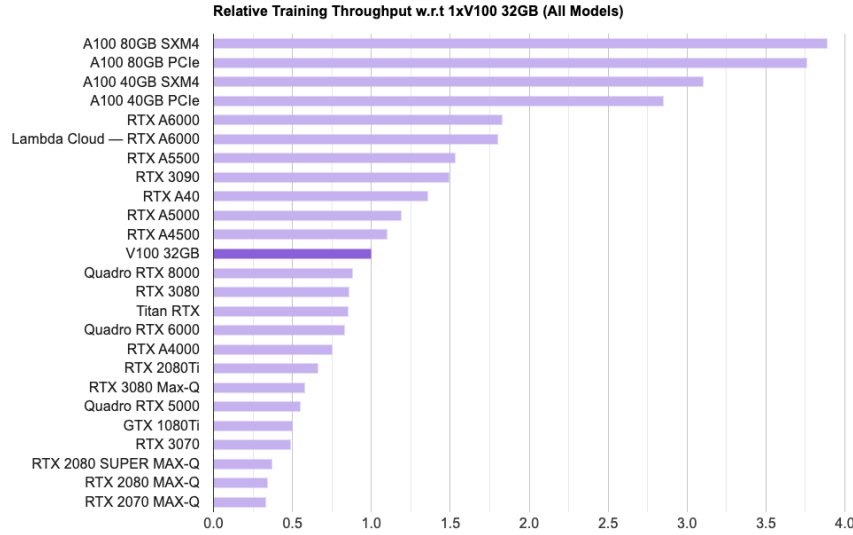


Figure 2.1: GPU 2022 benchmark from lambdalabs comparing Training throughput across several models ranging from resnet50 to BERT from lambdalabs.com

2.1.6 Some Interesting Architectures

In section 2.1.3, we defined a general structure of neural networks. We will delve deeper by detailing some architectures that will be useful for a better understanding of this manuscript.

Multi-Layer Perceptrons

The Multilayer perceptron (MLP) also called Fully Connected Neural Networks (FCNN) is the first architecture capable of solving complex non-linear problems. It was on these networks that the first mathematical proofs of the ability of neural networks to approximate function classes were made Cybenko [1989]. The linear transformation used is a matrix-vector product to which a vector called bias is added.

Definition 2.1.1. *An MLP layer is defined with the linear transformation*

$$T^l(x^{l-1}) = W^l x^{l-1} + b^l$$

where $W^l \in \mathbb{R}^{N_{l-1,l}}$ and $b^l \in \mathbb{R}^{N_l}$ followed by an activation function. This type of layer will be referred to as fully-connected layers.

For this type of network we have $w^l = \{W^l, b^l\}$. The choice of activation functions depends on the problem being solved and can vary for each layer. For example, [LeCun et al. \[1998\]](#) used a two-layer MLP to achieve an accuracy of 95.3% on the MNIST database problem. The MNIST database is a collection of 70,000 black and white images, each 28x28 pixels, representing handwritten digits. The goal is to train the network to recognise the digit in the image. The image, represented as a 28x28 vector, is fed into the MLP, which predicts the corresponding digit. However, there are two problems with this method. Not only is structural information about the image lost when it is represented as a vector (a pixel is far from most of its neighbours), but the number of parameters also grows with the input dimension and thus the image size. Using an MLP quickly becomes impractical in terms of computational cost. In the same article, [LeCun et al. \[1998\]](#) proposed a different architecture that overcomes these problems.

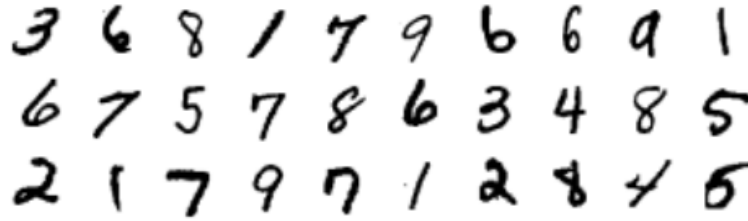


Figure 2.2: A sample from the MNIST database used by [LeCun et al. \[1998\]](#)

Convolutional Neural Networks

This architecture is a constrained variant of classic MLP that exploits the grid structure of images. These networks are called Convolutional Neural Network (CNN). To understand how CNN work, it is necessary to define two new neural network layers. Convolutional layers are the main building blocks of a CNN. Each convolutional layer uses a set of learnable filters with a small receptive field, but extending over the entire depth of the input volume. By sliding over the input image (or the output of the previous layer), these filters create feature maps. We can formally define them as follows.

Definition 2.1.2. Let x^{l-1} be our input of dimension $W \times H \times D$, where W is the width, H the height, and D the depth of a 3D image (e.g., an RGB image). Let W be our filter of dimension $f_w \times f_h \times D$ and b be the bias associated with filter number p . Denoting x and y as the coordinates over the width and height of x^{l-1} , we can define the result of the convolution of our filter on each pixel as:

$$C^p(x, y) = b^p + \sum_{d=1}^D \sum_{i=1}^{f_w} \sum_{j=1}^{f_h} W^p(i, j, d) x^{l-1}(x+i-1, y+j-1, d)$$

A convolutional layer consists of P filters, the number of which will determine the depth of the layer's output.

2.1. Deep Learning in a nutshell

For this type of layer, we have $w^l = \{W^p, b^p\}_{p=1}^P$. Their functioning can be visualized in figure 2.3.

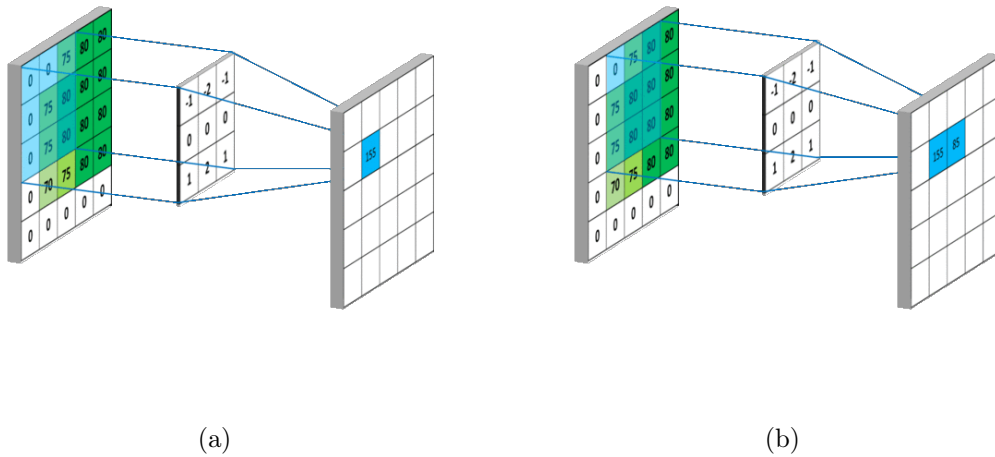


Figure 2.3: Illustration of the functioning of convolutional layers

The other layer to define is the pooling (or subsampling) layer. This is another essential component of a CNN that follows one or more convolutional layers. It performs a subsampling operation along the spatial dimensions (width and height) of the input volume, thereby reducing its size while preserving the most important features. Its main objectives are to reduce the computational cost of the network, to minimise overfitting by providing an abstract form of representation, and to improve the invariance of the network to small translations of the input.

A typical CNN layer T^l consists of a convolutional layer followed or not by a pooling layer. Fully connected layers can also be found at the output to obtain the desired format (especially in classification). The effectiveness of this architecture in computer vision was demonstrated in [LeCun et al. \[1998\]](#). Fig. 2.4, adapted from [LeCun et al. \[1998\]](#), visually compares the performance of Multilayer Perceptrons and Convolutional Neural Networks on the previously presented MNIST dataset. In this figure, MLP models are highlighted within a blue square, while CNN models, such as LeNet, are marked within a red square. The figure clearly shows that CNNs outperform MLPs across various preprocessing techniques and architectural variations. Not only do CNNs achieve significantly lower error rates, but they also do so more efficiently, making them a more cost-effective solution for image recognition tasks.

This superiority was further reinforced by [Krizhevsky \[2014\]](#), triggering the widespread adoption of deep neural networks for a variety of applications well beyond computer vision.

While this architecture is highly effective, it is limited to structured grids, whereas many types of data, such as meshes in Computational fluid dynamics (CFD), are in the

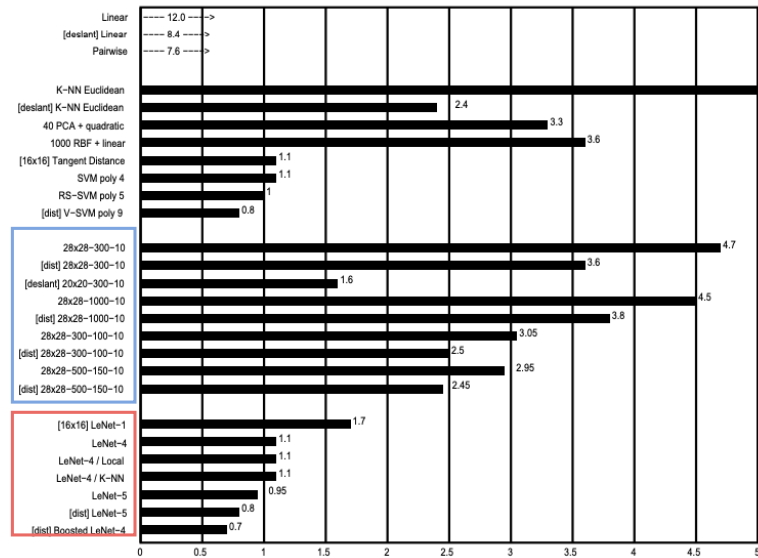


Figure 2.4: Error rate on the test set from [LeCun et al. \[1998\]](#), for further details on each implementation, please refer to the original reference.

form of graphs. We will therefore discuss a new architecture designed to handle this unstructured data: Graph Neural Network (GNN).

Graph Neural Networks

[Battaglia et al. \[2018\]](#) developed a framework where learning is applied to the representation of entities and their relationships. When discussing CNNs, the pixels of an image are the entities, and their relationships are determined by locality and spatial invariance. A filter acts only on a pixel's neighbourhood and remains the same regardless of the pixel's position. For more complex and arbitrary entity relationships, graphs are used. Here, nodes represent entities and edges symbolise their relationships. For example, in unstructured grids used to solve PDE, graph nodes represent physical values at geometric points in a domain Ω , and edges can carry multiple relative information, such as the distance between these points. The resulting learning model, designed to learn from this entity/relationship representation in a graph, is called a GNN.

There are several forms of GNNs, depending on the goal of the architecture (node-level, edge-level, or whole graph-level prediction) or the method used. For example, [Kipf and Welling \[2016\]](#) proposes an architecture similar to that used for images: Graph Convolutional Networks. However, this approach is rigid and uses a local convolution operation based on immediate neighbours, mainly using node values. In the rest of this manuscript, we will only discuss Message passing neural network (MPNN) [Gilmer et al. \[2017\]](#) and focus on node-level predictions in graphs. MPNNs represent relationships between entities as messages carried by edges. These messages are exchanged between

nodes and transformed by the MPNN layers, known as message-passing layers.

We can define an undirected graph as $G = (u, V, E)$. Here, u is a vector of constants valid for the entire graph, $V = \{v_i\}_{i=1}^{N_v}$ is the set of nodes, and $E = \{e_{v_i, v_j}\}_{i=1}^{N_e}$ is the set of edges. Denoting $C(v_i)$ as the set of nodes connected to v_i , we can define our message passing layer.

Definition 2.1.3. *For each node v_i in the graph, a general message passing layer is defined as:*

$$T^l(u_i) = g^l(T^l(u_i), \square_{v_j \in C(v_i)} f^l(u_i, u_j, e_{v_i, v_j}))$$

Here, f^l is generally a function composed of MLPs that calculates the node update based on the value of each of its neighbors and the associated edges, $\square$ is a permutation-invariant aggregation function, and g^l is an update function that defines how to combine the update values and the node's previous value.

This definition remains general, and we will detail practical examples in the following sections.

Long Short Term Memory networks and Attention neural networks

To capture time dependencies, the machine learning community first created recurrent neural networks (RNNs), which are networks that keep a memory called the hidden state. When processing data, like the vector representation of a word in a sentence, the RNN keeps track of previous information, making it good for tasks involving sequences of data. However, basic RNNs tend to forget older information when processing new data, which makes them poor at detecting long-term dependencies. This is a big problem in natural language processing (NLP), where context needed for understanding a paragraph may come from much earlier in the text, as discussed by [Bengio et al. \[1994\]](#). To solve this, Long Short Term Memory networks (LSTMs) were created to handle these tasks better. However, LSTMs still have problems like complexity, long training times, and difficulty in capturing long-term dependencies. Ultimately, LSTMs have been replaced in NLP and other sequential tasks by a new mechanism called attention.

Originally proposed by [Bahdanau \[2014\]](#), the concept of attention popularized by [Vaswani \[2017\]](#), allows the model to focus on important parts of a sequence during processing rather than treating each element with equal importance. For instance, in the sentence "he is eating a red apple", the word "apple" has a strong correlation with "red" and "eating" while "red" is mainly correlated with "apple". To implement this system, three vectors are defined:

- **Query (Q):** Represents the position or item of interest, such as a word to be translated.
- **Key (K):** Represents all positions or items to compare against; each word in the sentence has its own key.
- **Value (V):** Contains the information to be aggregated or extracted, corresponding to each key.

Once these vectors are defined, the attention score, indicating the similarity between the query and each key, is calculated. The most common method is to use the dot product and divide by the square root of the key vector's dimension:

$$\text{score}(Q, K) = \frac{Q \cdot K^T}{\sqrt{d_k}} \quad (2.1)$$

We then transform this score into a probability using a softmax function (Goodfellow [2016]), thus obtaining a set of normalized $\alpha_{i,j} = \text{softmax}(\frac{Q_i \cdot K_j^T}{\sqrt{d_k}})$ weights representing the attention of the i -th query for the j -th key. Finally, we use these weights to calculate a weighted sum of the value vector, which gives us our output.

$$\text{Attention}(Q, K, V) = \sum_j \alpha_{i,j} V_j \quad (2.2)$$

Self-attention is a particular type of attention where the query, keys and value vectors can come from the same sequence. It's this specific mechanism that allows us to detect dependencies in our sentence. In our case we represent our sentence as a matrix where each column is a vector representing each word. We define three linear transformation each with learned matrix W_Q, W_K, W_V giving us our needed vectors : $Q = W_Q x, K = W_K x, V = W_V x$ we then apply our attention scheme and we obtain our attention-layer :

$$T(x) = \text{Attention}(W_Q x, W_K x, W_V x)$$

In practice, self-attention is often implemented using multi-head attention. Essentially, this is a linear combination of the outputs of multiple attention layers, each of which receives a different learned linear transformation of the input. This approach allows the model to capture various dependencies of multiple projections.

This attention mechanism addresses the previous limitations, enabling efficient parallel training and capturing long-range dependencies. By employing multi-head self-attention, Vaswani [2017] introduced the Transformer architecture, which serves as the foundation for state-of-the-art results in NLP. For instance, the popular generative AI models GPT-1 to GPT-3 use the classic Transformer as their building block Kulkarni et al. [2023]. Starting with GPT-4, the architecture has become multimodal, accepting inputs beyond text. In image processing, GPT-4 once again utilizes an attention-based architecture, Vision Transformer (ViT) Dosovitskiy [2020], demonstrating the versatility of such mechanisms.

In this section, we have reviewed the basics of deep learning necessary for understanding the rest of the manuscript. We will now explore how to apply these methods to solve partial differential equations.

2.2 Deep Learning for Solving Partial Differential Equations

In a few years, Deep learning has already demonstrated its capabilities in diverse fields, including image recognition, natural language processing, and biology, where it has reshaped methodologies and greatly improved predictive accuracy.

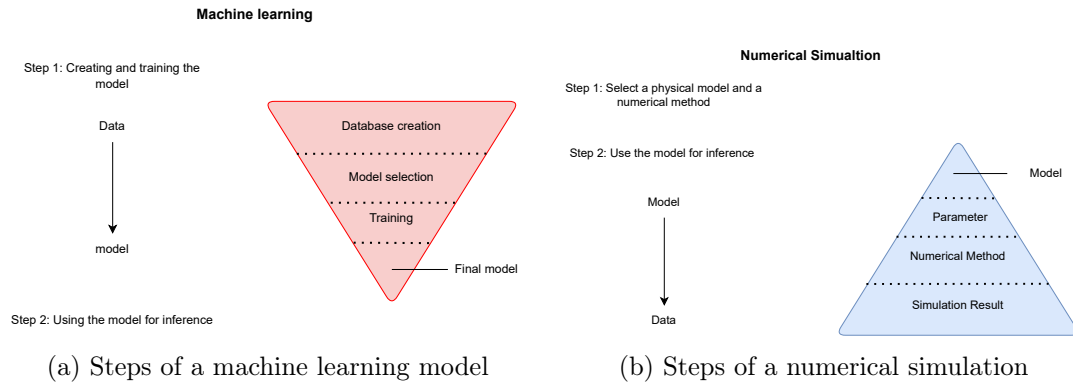
In physics, deep learning techniques offer a promising approach to modelling complex systems, enabling more efficient and accurate simulations. These methods excel at uncovering patterns in large datasets, enabling them to capture intricate physical phenomena that traditional modelling techniques may find challenging. The speed and efficiency of neural networks, especially when implemented on GPUs, further enhance their potential for real-time prediction and analysis.

While the integration of deep learning into physics modelling presents exciting opportunities, it also raises critical questions about effective implementation strategies. How can we best use these advanced techniques to improve our understanding and prediction of physical processes? This section aims to explore the intersections between deep learning and physics modelling, and to identify relevant approaches for their successful integration.

In this section, we outline key advancements from recent years. For more in-depth discussion, the reader is referred to the following reviews: [Herrmann and Kollmannsberger \[2024\]](#), [Wang et al. \[2024\]](#).

2.2.1 Classification

In their work, [von Rueden et al. \[2020\]](#) present numerical simulation and machine learning as two distinct but complementary approaches. These are conceptualised as two branches of a triangular model (see Fig. 2.5a, Fig. 2.5b), where machine learning generates models by processing input data, selecting a model, and training it on a dataset to yield a ready-to-use predictive model. Simulation, on the other hand, is based on predefined, parameterised models. These are used in combination with numerical methods to generate data.



Within this framework, various interactions between machine learning and simulation can be envisaged at each stage of the modelling process. We focus here in three main categories of potential combinations (see Fig. 2.6). The first, machine learning-assisted numerical simulations, encompasses the use of ML techniques throughout the simulation pipeline to replace or enhance specific components. This category can be further divided into three applications: pre-processing, post-processing and hybrid numerical methods.

The second category, data-driven surrogates, includes models that seek to entirely replace traditional numerical simulations using ML-based models trained on simulation results or empirical measurements. These surrogates, which can be either dependent on discretization (using structured grids or irregular meshes) or independent, offer a flexible alternative to conventional methods. In this thesis we will not discuss Lagrangian methods which, in contrast to Eulerian methods which predict the evolution of a fluid in a fixed environment, predict the movement of individual particles of this fluid through the domain. To go further we suggest to the reader these references [Wessels et al. \[2020\]](#) [Li and Farimani \[2022\]](#) [Torres et al. \[2023\]](#). Finally, we have the physics-informed surrogate, which relies on a physics prior during training. These three categories are not mutually exclusive and a surrogate can include a physical prior and use data, for example. Deep learning methods for data assimilation will not be developed in this work. For interested readers, please refer to [Peyron \[2024\]](#).

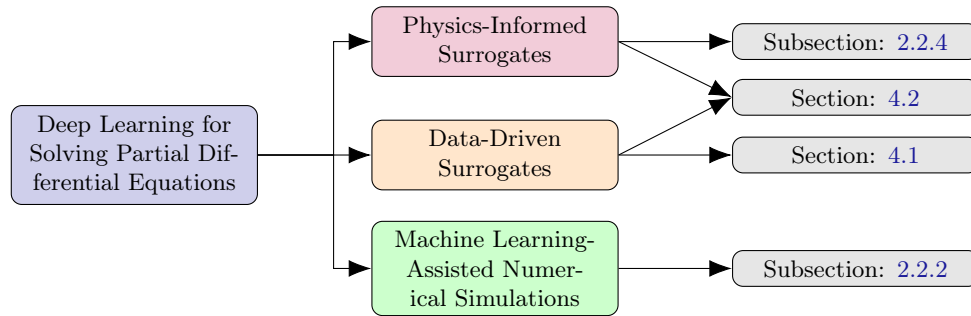


Figure 2.6: Classification of deep learning methods used in this document, linked to the corresponding section.

2.2.2 Machine learning-assisted numerical simulations

In this category, we bring together work aimed at speeding up existing physical solvers and, even more broadly, speeding up their use. The classic process for setting up a simulation is as follows:

- pre-processing
- numerical methods
- post processing

We will combine pre and post processing in a single category to obtain our classification.

Pre- and Post-Processing with Neural Networks

Neural networks can be used to pre-process data during parameter treatment or to post-process the results of numerical simulation. A tedious step in setting up a numerical simulation is mesh generation. Recall that a mesh provides a discrete representation of

the physical domain to be modeled. It can, for example, consist of nodes and edges for Finite element method (FEM). The quality of the mesh strongly influences the accuracy of the results obtained. Critical points of the domain, such as bridge piers for a river or elements of an airplane wing, require fine discretization to obtain coherent results. While one might be tempted to create a highly refined mesh everywhere, this would significantly increase the computational cost of our CFD simulation. Meshing is therefore a task that requires the expertise of specialists. Using their knowledge and experience, they optimise the mesh to best describe the physics being simulated, while keeping the computational cost reasonable. This step is time consuming. To automate this process and save valuable time, deep learning methods have been developed that make use of existing mesh databases. For example, [Nobari et al. \[2023\]](#) propose a method based on GNNs to automate the generation of meshes for aircraft models. This method outperforms state-of-the-art surface classification models and avoids under-refinement in CFD meshes, while being five times faster than adaptive remeshing methods. Similarly, [Lock et al. \[2023\]](#) uses MLPs to exploit the large number of existing CFD meshes to create near-optimal new meshes. This approach significantly reduces the time required to create meshes, while ensuring the quality of the results meets industry standards. Deep learning techniques can also be applied in post-processing to reduce the numerical errors in computed solutions. The most popular approach involves using convolutional neural networks to map coarse solutions to finer ones. By employing a dataset of corresponding coarse and fine solutions, a model can be trained in a supervised manner to learn the mapping for a specific problem [Fukami et al. \[2019, 2021\]](#).

Hybrid Numerical Methods

Machine learning methods can be integrated at different stages of the numerical method. [Bar-Sinai et al. \[2019\]](#) proposes a method for solving PDEs that is very similar to well-known techniques. The starting observation is that many classical methods (Finite difference method (FDM), FEM, etc.) are based on discretising space and approximating the spatial derivatives at the points obtained. Thus, to approximate the spatial derivative of a physical field $v_i(t)$, each method uses a formula of the form

$$\frac{\partial^n v}{\partial x^n} \approx \sum_i \alpha_i^{(n)} v_i$$

where the $\alpha_i^{(n)}$ are precalculated coefficients of varying complexity depending on the technique used. The quality of the approximation therefore depends strongly on the fineness of the discretisation. The idea presented in this paper is to generate the $\alpha_i^{(n)}$ coefficients using a neural network. This approach moves from generic coefficients to coefficients specific to the equation, allowing a reduction in the fineness of the discretisation. The detailed training process considers multiple loss functions to incorporate a notion of numerical stability into the time integration. Building on this idea, [Kochkov et al. \[2021\]](#) propose integrating a convolutional neural network within a CFD framework to replace the traditional interpolation methods used in finite volume methods. The network is

2.2. Deep Learning for Solving Partial Differential Equations

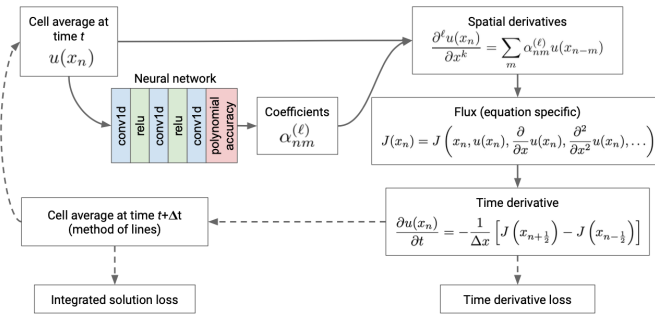


Figure S1: **Neural network architecture.** During training, the model is optimized to predict cell average time-derivatives or time evolved solution values from cell average values, based on a precomputed dataset of snapshots from high resolution simulations. During inference, the optimized model is repeatedly applied to predict time evolution using the method of lines.

Figure 2.7: Illustration of the hybrid solver method presented in Bar-Sinai et al. [2019]

trained on high-resolution simulations, with fine mesh results projected onto the solver's coarse grid. This approach enables substantial computational speedups (up to 80-fold) while maintaining high accuracy. In a broader collaborative perspective, Um et al. [2020] address the problem of correcting numerical errors introduced by the discretisation of PDEs. They propose an approach in which a convolutional neural network corrects the solutions provided by a low-fidelity solver, alternating between the solver and the neural network at each time step. This method, called "solver-in-the-loop", allows gradients to propagate back through the PDE solver, thus taking into account the influence of the correction on the trajectory of the solution. The results show that this technique significantly improves the accuracy of the solutions compared to classical supervised approaches, allowing stable recurrent iterations over several hundred time steps. Building on the concept of integrating numerical solvers with deep learning, Lapeyre et al. [2019] propose a novel hybrid approach to combustion modelling using convolutional neural networks. In this method, a CNN is used to replace one component of the numerical simulation, specifically the modelling of subgrid scale reaction rates. The problem is framed as predicting a 'flame surface density' from a progress variable. The data for this task is generated using a numerical solver on a fine homogeneous Cartesian grid, which is then sampled with a Gaussian filter on a grid eight times coarser.

The deep learning problem can thus be formulated as:

$$\bar{\Sigma}(X) = f(\bar{c}(X)), X \in \mathbb{R}^{n^3}$$

$$f : \mathbb{R}^{n^3} \rightarrow \mathbb{R}^{n^3}$$

Where $X \in \mathbb{R}^{n^3}$ is a cube of $n \times n \times n$ adjacent nodes, $\bar{\Sigma}$ is the filtered 'flame surface density', and $\bar{c}(X)$ is the filtered progress variable. The use of a CNN is justified by several key points:

- Convolutions allow for approximating derivatives of any order of a scalar field.

- Flames are complex structures, so it makes more sense to treat them as such rather than point-by-point.
- Convolutions enable learning with shared parameters, allowing for handling problems with a large parameter n .

A network inspired by U-Net [Ronneberger et al. \[2015\]](#) is thus adapted to the task. The results show that this hybrid approach, where the CNN enriches the traditional numerical solver, is highly effective. After training, the network is both accurate (sometimes better than certain algebraic models) and robust, generalising well to other test cases. This integration of CNNs into numerical simulations provides a data-driven, more accurate representation of the complex interactions in turbulent combustion and demonstrates the potential of hybrid solvers to improve the accuracy and reliability of computational models.

Another approach, proposed by [Sapli et al. \[2019\]](#), uses neural networks as a preconditioner to improve the convergence rate of iterative solvers in large, sparse linear systems solved in numerical methods. Specifically, they employ a convolutional neural network to construct preconditioning matrices tailored to the specific problem, enhancing the efficiency of conjugate gradient solvers in computational fluid dynamics and other applications. This data-driven approach adapts the preconditioner dynamically, leading to faster convergence than traditional methods. A widely used preconditioner is the multigrid preconditioner. [Greenfield et al. \[2019\]](#) propose building multigrid preconditioners by replacing the traditional prolongation operator (extension operator) with a convolutional neural network. This CNN-based operator is trained in an unsupervised manner to enhance convergence on regular grids, specifically for diffusion problems. [Luz et al. \[2020\]](#) extend this idea to irregular grids by employing a graph neural network to construct the prolongation operator. The GNN-based approach adapts to varying graph structures, optimizing algebraic properties for a broader range of grid topologie.

2.2.3 Data-driven surrogates

The use of machine learning in pre-processing, post-processing and hybrid solvers has significantly improved accuracy, reduced computational costs and streamlined steps in the numerical simulation pipeline. However, these advances still fall short of the acceleration needed for real-time processing of physical problems. With these methods, real-world scenarios can still take hours to solve. In our specific case of flood forecasting, near-instantaneous predictions are essential to provide early warnings to vulnerable populations. This need has led the ML community to develop surrogate models. In our context, we define a surrogate model as a machine learning model that takes input parameters of a dynamical system, such as the initial state/condition, boundary conditions and other physical parameters, and predicts the evolution of the system over time. In addition, a surrogate can also be used to predict solutions to non-time-dependent PDEs, providing flexibility in modelling both dynamic and static systems. In the last five to six years, surrogate models have gained considerable interest in the physics and ML communities [Fig. 2.8](#). In this section we present an overview of these techniques. Our aim

is not to provide an exhaustive review, but to place these works in the context of the ongoing developments in physics and machine learning. Surrogates can be categorized in

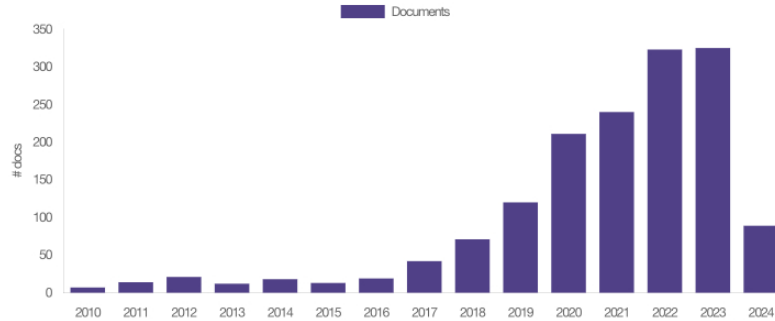


Figure 2.8: Evolution of the number of physics research articles mentioning the terms 'AI model' and 'surrogate model' over the years in the PORTICO and JSTOR databases (generated using constellate.org).

various ways, with one of the most intuitive distinctions being between those that rely on discretization and those that function independently of it. In this section, we outline the foundational principles of these categorization methods, setting the stage for a deeper examination of their applications in geosciences in Chapter 4. A comprehensive summary of all discussed methods is presented in Table 2.1 for ease of reference. In particular, as mentioned above, this classification does not completely exclude the integration of physics-informed surrogates, which will be further explored in section 2.2.4.

Discretisation-dependent surrogates

One of the first popular surrogate breakthroughs was proposed by Wiewel et al. [2019], where the authors introduced a method in which the surrogate model operates in a latent space created by an encoder-decoder architecture. Consider x as a function representing a physical field such that $x : \mathbb{R}^3 \times \mathbb{R}^+ \rightarrow \mathbb{R}^d$, where d is the dimension of the target field. The goal is to predict the future o states of the function from the previous n states using a neural network $f_{\omega_t}^t(X)$. This can be expressed as:

$$f_{\omega_t}^t([x(t-nh), \dots, x(t)]) = [x(t+h), \dots, x(t+oh)]$$

To reduce computational cost associated with the high dimensionality of x , the solution provided by $f_{\omega_t}^t$ is computed in a latent space of dimension ms , created by an encoder. Let $f_{\omega_e}^e(x) : \mathbb{R}^d \rightarrow \mathbb{R}^{ms}$ be the encoder and $f_{\omega_d}^d(c) : \mathbb{R}^{ms} \rightarrow \mathbb{R}^d$ be the decoder. This ensures that:

$$f_{\omega_d}^d(f_{\omega_e}^e(x)) \approx x$$

The function $f_{\omega_t}^t$ is reformulated as $\hat{f}_{\omega_t}^t$ to operate in the latent space, and the problem is rewritten as:

$$f_{\omega_d}^d(\hat{f}_{\omega_t}^t(f_{\omega_e}^e([x(t-nh), \dots, x(t)]))) = [x(t+h), \dots, x(t+oh)]$$

2.2. Deep Learning for Solving Partial Differential Equations

Table 2.1: Overview of Data-driven Surrogates methods, the methods in blue are discussed in subsection 2.2.4 and the methods in green in Chapter 4

References	Method	Backbone	Training Case
Discretization Dependent Models			
<i>Regular Grids</i>			
Wiewel et al. [2019]	Latent Space Physics	LSTM/CNN	Navier-Stokes (smoke and liquid)
Pathak et al. [2022]	FourCastNet	Transformers/Fourier Layers	Global Weather Forecasting
Bi et al. [2023]	Pangu-Weather	3D Earth-Specific Transformer	Global Weather Forecasting
Chu et al. [2020]	ANN-based emulation	MLP	Flood Prediction
Kao et al. [2021]	SAE-RNN	MLP/LSTM	Flood Prediction
Pianforini et al. [2024]	FloodSformer	CNN/VPTransformer	Flood Prediction
Chen et al. [2023], Sun et al. [2024]	FuXi/FuXI-DA	MLP/Swinransformer	Global Weather Forecasting
Vaughan et al. [2024]	Aardvark	VPTransformer	Global Weather Forecasting
<i>Irregular Meshes</i>			
Pfaff et al. [2020]	MeshGraphNet	GNN	Cylinder FLOW, Airfoil
Fortunato et al. [2022]	Multiscale MeshGraphNet	GNN	Cylinder FLOW
Lam et al. [2022]	GraphCast	GNN	Global Weather Forecasting
Bentivoglio et al. [2023a]	SWE-GNN	GNN	Flood Prediction
Bentivoglio et al. [2024]	mSWE-GNN	GNN	Flood Prediction
Huang et al. [2024]	DiffDA	GNN	Global Weather Forecasting
Lang et al. [2024]	AIFS	GNN/Transformer	Global Weather Forecasting
Grid Independent Models			
Lu et al. [2021]	DeepONet	MLP	Reaction-Diffusion, multiple ODEs
Lanthaler et al. [2022]	Shift-DeepONet	MLP	Advection, Burgers, Euler
Venturi and Casey [2023]	SVD-DeepONet	MLP	Multiple ODEs
Seidman et al. [2022]	NOMAD	MLP	Advection, Shallow Water, Anti-Derivative
Lee et al. [2023]	HyperDeepONet	MLP	Advection, Burgers, Shallow Water
Li et al. [2020c]	FNO	Fourier Layers	Burgers, Darcy, Navier-Stokes
Helwig et al. [2023]	G-FNO	Fourier Layers	Navier-Stokes, Shallow Water
Li et al. [2024a]	GINO	Fourier Layers/GNN	Reynold Averaged Navier-Stokes
Xu et al. [2024]	GeoPINS	MLP/Fourier Layers	Flood Prediction
Guan et al. [2024]	LUCIE	Spherical FNO	Global Weather Forecasting
Wang et al. [2021a]	PiDeepONet	MLP	Anti-Derivative, Reaction-Diffusion, Burgers
Goswami et al. [2022]	V-DeepONet	MLP	Brittle fracture
Li et al. [2024b]	PINO	Fourier Layers	Burgers, Darcy, Navier-Stokes, Kolmogorov
Konuk and Shragge [2021]	Physics-guided FNO	Fourier Layers	Acoustic wave equation

In this paper, the encoding-decoding process is handled by a convolutional neural network, and the prediction in the latent space is performed by a recurrent neural network using LSTM (long short-term memory) cells, structured as a temporal encoder/decoder. The temporal encoder creates a temporal context from the n data representations in the latent space. This context is then provided o times to the decoder, which uses it along with its internal state to generate the o predictions in the latent space. This architecture is efficient for computing the evolution of physical fields, offering speed-ups up to 150 times faster than traditional solvers in their tests, although improvements in accuracy are still needed. However, its reliance on convolutional layers limits it to regular grids.

No such limitation exists Pfaff et al. [2020], which proposes an improved encoder-processor-decoder architecture that extends the approach to graph contexts, enabling its application to a wider variety of numerical simulation mesh types. In MeshGraphNet, data is represented as nodes and edges.

In the main test case, node values, denoted u_i , represent the concatenated velocity and a one-hot encoded vector indicating the type of node (e.g., normal, inflow, wall). Edge values, denoted e_{v_i, v_j} , represent the relative position between points on the physical mesh, associated with the norm of this vector.

The nodes and mesh features are encoded into a latent space by two dedicated encoders, each composed of fully connected neural networks. These latent values are then

updated using several message-passing layers and subsequently decoded to predict the acceleration.

For the encoding part, two FCNNs are defined, noted as $f_{w_\epsilon}^\epsilon$ and $f_{w_\mu}^\mu$. In the latent space, message passing is performed using a mechanism described in 2.1.3. During each message passing, the message is a learned transformation of the current message and the connecting nodes, achieved with an FCNN. These messages are aggregated for each node by summing all the connected layers: $\square = \sum_{e_{v_i}, e_{v_j}}$. The concatenation of the resulting value and the current node value is then passed through another FCNN (acting as the update function g) denoted as $f_{w_u}^u$ to create the updated value. The results on the

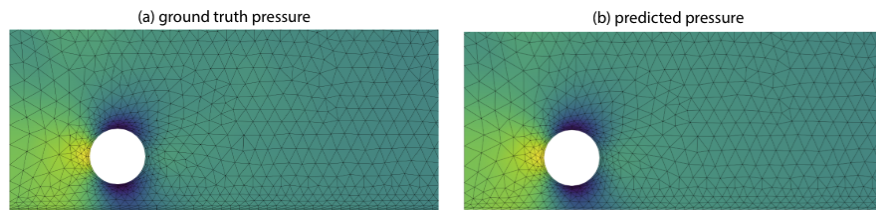


Figure 2.9: An example of pressure groundtruth and prediction of the cylinder flow test case in Pfaff et al. [2020]

'cylinder flow', a flow of water around a cylinder obstacle Fig. 2.9, test case demonstrate significant improvements in speed compared to traditional methods keeping a good precision. These stable and accurate predictions were generated at a much faster rate, running 1-2 orders of magnitude quicker than conventional solvers. The model is also able to generalize, handling different inflow conditions and boundary scenarios with ease. However, the test case remains simple and far from real-life applications. Expanding on the progress made by MeshGraphNet, the MultiScale MeshGraphNet (MS-MGN) defined in Fortunato et al. [2022] addresses specific challenges in high-resolution simulation tasks. In the standard MeshGraphNet, increasing the mesh resolution leads to larger distances between nodes in the graph structure. This spacing introduces inefficiencies in message-passing operations and can result in over-smoothing effects, which limit the model's performance. MS-MGN overcomes these limitations by employing a hierarchical, multi-resolution approach that works simultaneously with both fine and coarse mesh levels. This dual-scale method enables the model to capture both localized and broader patterns within the data more accurately. By conducting message passing on the coarser mesh, MS-MGN accelerates information flow throughout the network, enhancing both computational efficiency and overall prediction accuracy.

Discretisation-independent surrogates

We have previously discussed the universal approximation theorem, which states that single-layer neural networks can universally approximate continuous functions. However, in 1995, Chen and Chen [1995] demonstrated that a specific neural network architecture can also serve as a universal operator approximator. This implies that neural networks

are capable of accurately learning mappings from one function space to another. Let's take the following differential equation :

$$\mathbf{L}u = f \quad (2.3)$$

define on a domain Ω where $\mathbf{L}$ is an operator which map the solution of our equation $u : \Omega \rightarrow \mathbb{R}$ to a function $f : \Omega \rightarrow \mathbb{R}$. The idea is to approximate the $\mathbf{L}^{-1}$ operator which map our known f function to our solution. To this end, [Lu et al. \[2021\]](#) propose the DeepONet, a neural network architecture based on [Chen and Chen \[1995\]](#) results. The architecture is composed of two separate networks 2.10. A Branch network encodes the

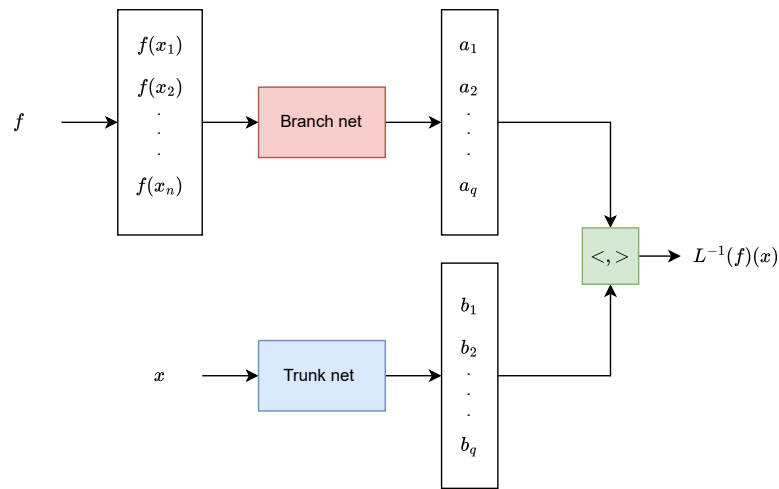


Figure 2.10: Illustration of the DeepONet architecture

values of the function f in a q -dimensional space based on a fixed irregular "sensor map" (an ensemble of sampled points) of Ω . Meanwhile, a Trunk network takes the coordinates of Ω as input to generate another q -dimensional vector. The output of the DeepONet is obtained by computing the dot product between these two vectors. The network is trained to match the output u corresponding to the function f at the coordinates provided to the Trunk network. Even if the sensor map is fixed, the output can be taken freely in Ω which is notable.

Several extensions to this work have been proposed. [Lanthaler et al. \[2022\]](#) introduced Shift DeepONet, which incorporates two auxiliary neural networks, a scale and a shift network, that dynamically adjust the weights and biases of the initial trunk network layers based on sensor data to address discontinuities. In a similar approach, [Venturi and Casey \[2023\]](#) proposed adding a "prenet" to compute the initial layer's weights and biases, modifying the branch network to add a bias term to the final output calculation. [Seidman et al. \[2022\]](#) presented a method that uses the branch network output as an embedding, concatenating it with the trunk network's input vector to enhance feature representation. Finally, [Lee et al. \[2023\]](#) proposed HyperDeepONet, leveraging

the branch network as a hypernetwork to produce all weight matrices and biases of the trunk network. These enhancements, grounded in theoretical advancements, significantly strengthen the DeepONet framework but remain insufficient for tackling large-scale industrial applications.

This philosophy of operator learning have been developed in Kovachki et al. [2023] where a framework is proposed for so called Neural Operator. A prominent member of this family is the Fourier Neural Operator (FNO, Li et al. [2020c]). The problem is framed as using a neural network to map initial and/or boundary conditions to the solution in an image-to-image manner. The idea is that, like real-world images, PDE solutions have edges and shapes at different scales. While CNNs are effective for capturing edges in images, operator learning typically deals with PDEs whose inputs (e.g., boundary or initial conditions) and outputs (the PDE solutions) are continuous fields, making Fourier representations more appropriate. According to this observation, they proposed Fourier Layer using Fast Fourier Transform to go to Fourier space, a linear transform on the lowest fourier mode and an Inverse Fast Fourier Transform. This result is added with a linear transformation of the original input and then passed through a non-linearity Fig. 2.11. The use of this layer allows for effective capturing of different frequencies in the problem

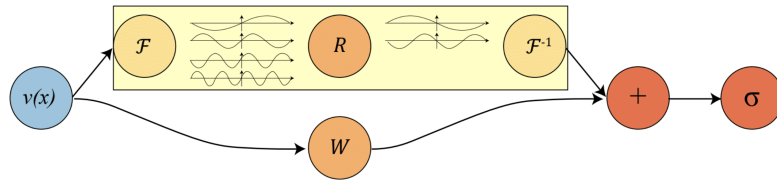


Figure 2.11: An example of Fourier Layer from Li et al. [2020c] with $\nu(x)$ the input, $\mathcal{F}$ the fourier transform, W and R the linear transform and σ the activation function

and ensures that the learning operators are discretization-invariant. However, it restricts the network’s application to inputs on a regular grid.

The Fourier Neural Operator has seen substantial advancements aimed at enhancing its utility and effectiveness in handling complex PDE tasks. The G-FNO model (Helwig et al. [2023]) introduces a group-equivariant architecture that expands FNO’s capabilities by embedding symmetry properties, such as translation, rotation, and reflection, directly into the Fourier layers. This architecture enables G-FNO to process symmetric physical phenomena more effectively, as demonstrated in Navier-Stokes simulations where rotational symmetry is critical to fluid flow dynamics. Meanwhile, the Geometry-Informed Neural Operator (GINO from Li et al. [2024a]) incorporates geometric information to support large-scale 3D simulations involving irregular grids, commonly encountered in computational fluid dynamics. GINO combines Graph layers for localized geometric processing with FNO layers for efficient global integration. This design is particularly well-suited to applications such as predicting pressure fields on car surfaces, achieving a performance improvement up to 26,000 times faster than traditional CFD solvers.

In this section, we briefly reviewed the various ways to combine deep learning and physical solvers to accelerate numerical simulations. We then presented several types of

substitution models. This presentation is completed in chapter 4 by a presentation of surrogate models applied to the geosciences.

2.2.4 Physics informed machine learning

Although highly effective, purely data-driven machine learning methods can be inconsistent with physical laws and lose robustness. This lack of robustness and these difficulties in generalization depend in part on the quality of the data available, which is sometimes noisy or derived from numerical methods that have not completely converged. The massive raw data used for our models is not the only representation of the knowledge available to us. Informative priors, from sources as heterogeneous as empirical observations to PDEs governing physical laws, are information that we can share with our models. The field of study that attempts to integrate this information into machine learning is known as Physics-Informed Machine Learning (PIML). PIML approaches allow different forms of physics to be embedded in machine learning models. These can include physical laws in the form of differential equations, conservation principles, physical symmetries, and empirical models derived from the observation of natural phenomena. The integration of physics into machine learning can take different forms: the incorporation of inductive biases into model architectures or the use of adapted loss functions to impose physical constraints, for an example. These approaches make it possible to develop models that are not only accurate but also interpretable, providing interpretations that respect known physical laws. We have already seen in this manuscript an architecture that implicitly incorporates an inductive bias: CNNs. CNNs are intrinsically translation invariant, which is very useful for detecting objects at different positions. Some architectures have been proposed to integrate physical biases into their structure, such as the SWE-GNN presented by [Bentivoglio et al. \[2023b\]](#) in chapter 4. We will focus in this section on approaches that incorporate physical constraints in a flexible way by penalising learning with an adapted loss function. The best known examples are Deep-Ritz neural networks and physics-informed neural networks (PINNs). While the former use a weak formulation of the PDEs to define their loss function, the latter use a strong formulation. We will look at this method in more depth in this section.

A Physics-informed neural networks

The physics-informed neural networks have raised significant interest [Raissi et al. \[2019\]](#) allowing to combine supervised data-learning with PIML. Given a domain $\Omega \subset \mathbb{R}^d$, we consider the following differential system:

$$\begin{cases} \mathcal{L}(u(x)) = r(x) & \text{in } \Omega \\ \mathcal{B}(u(x)) = g(x) & \text{on } \Gamma \end{cases} \quad (2.4)$$

where Γ is the boundary of Ω , $\mathcal{L}$ and $\mathcal{B}$ are two (possibly nonlinear) differential operators and r and g are two given functions. We can have several $\mathcal{B}$ representing various type of boundary conditions from Dirichlet to Robin or initial conditions, we limit this example to one for the sake of readability. PINNs approximate the solution of the problem by

a sufficiently smooth neural network $f(w) : \mathbb{R}^d \rightarrow \mathbb{R}$. The neural network is trained by minimization of a loss that takes into account the physical information contained in the PDE. Specifically, denoting $X_\Omega = \{x_\Omega^j\}_{j=1}^{N_\Omega}$, $X_\Gamma = \{x_\Gamma^j\}_{j=1}^{N_\Gamma}$ a set of training points sampled in Ω and Γ respectively, the loss function is defined as

$$f(x) = \underbrace{\frac{\lambda_\Omega}{N_\Omega} \sum_{j=1}^{N_\Omega} [\mathcal{L}(f(w)(x_\Omega^j)) - r(x_\Omega^j)]^2}_{L_r} + \underbrace{\frac{\lambda_\Gamma}{N_\Gamma} \sum_{j=1}^{N_\Gamma} [\mathcal{B}(f(w)(x_\Gamma^j)) - g(x_\Gamma^j)]^2}_{L_i} \quad (2.5)$$

where λ_Ω , λ_Γ are positive weights which balance the contribution of the residual of the PDE and the residual of the boundary conditions. The differentiability properties of the neural networks are exploited to explicitly compute the differential operators $\mathcal{L}(f(w))$ and $\mathcal{B}(f(w))$, which are then evaluated on the set of training points.

Learning theory

The first works that establish a theoretical framework for the learning of PINNs were conducted by [Shin et al. \[2020\]](#). The error committed by a PINN is decomposed into three distinct types of errors: approximation error, optimization error, and estimation error.

- **Approximation Error:** When using a PINN, one starts by selecting a neural network architecture. Denoting w as the vector of parameters of this architecture and n as the size of this vector (i.e., the number of parameters), we can define the class of functions in which we seek a solution as $\mathcal{H}_n = \{f_w : n\}$. This architecture is directly related to the approximation error, which represents the distance between the desired solution f^* and the best possible approximation within the class of functions. This is the most studied error.
- **Optimization Error:** Optimization error arises during the training of the neural network. The objective is to minimize a loss function that is highly non-convex, making optimization difficult. Gradient-based optimization methods, such as Adam and L-BFGS, are commonly used in training PINNs. However, there is no guarantee that these methods will find a global minimum for general machine learning problems, including PINNs.
- **Estimation Error:** Estimation error results from using a finite number of data points. This error, along with approximation error, constitutes generalization error, which measures the accuracy of predictions on unseen data. In PDE problems, generalization error is the distance between a global minimizer of the loss function and the solution to the PDE, with this distance needing to be defined to reflect its regularity.

In the work by [Shin et al. \[2020\]](#), convergence estimates are provided for linear second-order elliptic and parabolic type PDEs. As the number of data increases, PINNs generate

a sequence of minimizers corresponding to a sequence of neural networks. In this context, convergence means that, under certain assumptions, the sequence of minimizers strongly converges to the PDE solution in L^2 . Later, in [Shin et al. \[2023\]](#), these convergence results are extended to encompass all linear problems, including hyperbolic equations, thereby broadening the theoretical framework’s applicability to a wider class of linear PDEs. The work by [De Ryck et al. \[2021\]](#) provides an explicit bound for the approximation error of a function in Sobolev space using a two-layer neural network with the hyperbolic tangent activation function. This bound depends on the number of training points and the problem’s dimensionality, illustrating that such networks can effectively approximate Sobolev-regular and analytic functions.

Later [Mishra and Molinaro \[2023\]](#) worked on the generalization error of PINN’s providing an upper bound using an abstract formalism and properties of the underlying PDE. This bound depends on the approximation error and the number of training points. [De Ryck et al. \[2024\]](#) targets specifically for the Navier-Stokes equation and gives a bound on the generalization error depending on a small training error. However this bound, like the one in [De Ryck et al. \[2021\]](#), suffers from the curse of dimensionality : to reduce the error by a factor, the number of training points needed and the size of the network scale up exponentially.

The PINN’s landscape

Since the introduction of physics-informed neural networks by [Raissi et al. \[2019\]](#), a considerable number of studies have been published to improve the efficiency of this method. A global comparison has been proposed in the paper by [Hao et al. \[2023\]](#), which provides an exhaustive benchmark of existing improvements for various partial differential equations (PDEs) as well as a detailed classification of these improvements. Loss reweighting/resampling methods are one of the main categories of improvements. These techniques aim to adjust the lambda weights of the loss functions to optimise convergence at different training epochs, or to resample the points used to compute these losses. Another category of improvements concerns specialised optimisers. These optimisers are designed specifically for PINNs to improve their efficiency and robustness. The new loss functions are another type of improvement. They modify or add terms to the classical loss function to better capture the dynamics of PDE solutions. Finally, new neural network architectures propose modifications to the existing architecture of PINNs. These include changes in layers, activation functions, or other more complex adjustments to improve the networks’ ability to solve PDEs. Using this classification, we’ll present some of the main improvements proposed in the literature.

Loss reweighting/resampling In their work, [Wang et al. \[2020\]](#) observe that the gradient of the loss associated with boundary conditions is significantly lower than the gradient related to the PDE, thereby biasing the learning process. Upon further analysis, it appears that the eigenvalues of the Hessian matrix of the loss terms are responsible for these disparities in learning. A learning rate annealing technique is proposed 2.6 to update the weights of the loss function, which significantly improves the convergence of

networks on the treated examples. The formula used to update the weight associated to the boundary terms λ_i values (initialized to 1) is given by:

$$\lambda_i = (1 - \alpha) \lambda_i + \alpha \frac{\max_w |\nabla_w L_r(w)|}{|\nabla_w L_i(w)|} \quad (2.6)$$

Where α is an hyperparameter. [Wu et al. \[2023\]](#) provide a comprehensive study on the impact of sampling methods and their formats. In addition to comparing different sampling methods and their efficiency, they also examine dynamic sampling methods. The construction of the PINN method allows for changing the sampled points over epochs, and numerous methods have been proposed to sample more points in regions where the error is greatest. They compare these methods with a proposed method and demonstrate its effectiveness, particularly when the number of sampled points is small. The work provides a good overview of the effectiveness of different sampling variations.

Specialised optimisers In [Yao et al. \[2023\]](#) the problem of the loss term imbalance is brought back to light and a special version of Adam called MultiAdam is proposed. The main idea is to separate the L_r term from the boundary term and calculate the Adam momentum for each group. The second momentum of Adam effectively reflects the difference in scale between the loss terms and allows the terms to be normalised and put on a comparable scale. The updates calculated are then averaged and used to update the parameters of the network. This method is compared to [Wang et al. \[2020\]](#) and provides better results and stability on the tested PDE.

New loss functions Using a Petrov-Galerkin framework and employing PINNs as the test function, [Kharazmi et al. \[2019\]](#) proposes the Variational PINN (V-PINN) method, which results in a modification of the loss function. This modification depends on the choice of the linear test function and the use of the variational formulation of the PDE equation. The use of this formulation reduces the order of differentiation required, resulting in lower costs during the optimisation process. The performance of V-PINN is strongly dependent on the choice of test function, and no clear evidence of its superiority is given in the [Hao et al. \[2023\]](#) benchmark. Gradient enhanced PINNs (gPINNs) are proposed in [Yu et al. \[2022\]](#). The main idea is to add a term to the original loss function that penalises the derivative of the residual loss with respect to the inputs to be zero. In fact, if the residual must be zero everywhere, then its derivatives should also be zero. This method outperforms vanilla PINNs on the problems tested, but depends heavily on hyperparameters that weight the new term in the loss function. Thus, adding an order of magnitude in the derivative makes the cost of gPINN two to three times higher than PINNs, making it not worth it for every problem.

Novel activation or architectures Novel activation functions have been proposed in [Jagtap et al. \[2020b\]](#) with Globally Adaptive Activation Functions (GAFF) and [Jagtap et al. \[2020a\]](#) with Locally Adaptive Activation Functions (LAAF) to improve the efficiency and accuracy of PINNs. Both methods incorporate scalable hyperparameters into

the activation functions, allowing for dynamic optimisation during training. This adaptability improves the convergence rate and solution accuracy by modifying the topology of the loss function. GAAF introduces a single scalable parameter for the whole network, which is dynamically optimised to improve convergence, especially in the early stages of training. Theoretical analysis confirms that gradient descent algorithms using GAAF avoid suboptimal critical points or local minima, ensuring robust and efficient training. LAAF methods, which include layer-wise (L-LAAF) and neuron-wise (N-LAAF) approaches, introduce scalable parameters for each layer or neuron, allowing finer control over the slopes of the activation function. This local adaptation accelerates convergence by implicitly conditioning the gradient dynamics, optimising the learning process without the need to explicitly compute conditioning matrices. In addition, a slope recovery term is added to the loss function to further accelerate training. Theoretical proofs guarantee that LAAF methods avoid suboptimal critical points and local minima, leading to faster convergence and improved accuracy. Empirical results show that LAAF methods significantly outperform both standard and GAAF methods in terms of training speed and accuracy on several deep learning and PDE benchmark problems. The paper [Ren et al. \[2022\]](#) presents two novel neural network architectures, PhyCRNet and PhyCRNet-s. These architectures employ an encoder/processor/decoder framework using convolutional long short-term memory (ConvLSTM) networks to extract low-dimensional spatial features and learn their temporal evolution. To ensure accuracy, the models incorporate hard-encoded initial and boundary conditions, and use autoregressive and residual links for efficient temporal resolution. The output, represented on a regular grid, requires a filter-based differentiation approach inspired by finite difference methods to construct the PDE elements. This technique is also used to hard-encode the boundary and initial conditions. The PhyCRNet methods exhibit improved scalability and accuracy, reducing the dependence on extensive hyperparameter tuning and improving solution quality, especially at the boundaries. Empirical results show that PhyCRNet significantly outperforms traditional physics-informed neural networks in terms of training speed, accuracy and extrapolation capabilities. However, the use of convolutional layers removes this method from the mesh-free framework provided by vanilla PINNs and introduces scaling challenges similar to those of regular mesh solvers.

The majority of the presented methods efficiency have been compared in [Hao et al. \[2023\]](#) over a dataset of 22 unique case of PDE problems. These problems, ranging from Poisson’s equation to chaotic PDEs, exhibit a wide range of properties such as complex geometry, multi-scale phenomena, non-linear behaviour and high dimensionality. The results show that there is no single PINN to rule them all, but a complementarity of methods depending on the problem to be tackled. Hyperparameters also play a major role in the efficiency of the methods, making the comparison more complex. The methods presented here are only a fraction of the PINN research field, but give a good idea of the range of possibilities.

Physics-informed neural operators

The approaches presented in the Neural Operators section are purely data driven and require large and exhaustive datasets. As mentioned above, such datasets are rarely available for most engineering applications. However, there is no obstacle to using the PINN framework with these models. DeepONets are differentiable with respect to their input coordinates, allowing the use of automatic differentiation to obtain the required values and to regularise the data-driven loss with physical knowledge. It should be noted that the cost of automatic differentiation can be high, especially for higher order derivatives. Depending on the sampling, Sobel filters [Zhu et al. \[2019\]](#) or finite difference schemes can be used. [Wang et al. \[2021a\]](#) proposes the introduction of the classical PINN loss in DeepONet training, while [Goswami et al. \[2022\]](#) uses a variational formulation of their PDE problem to better handle the discontinuity characteristics of their problems. Following the example of physics-informed DeepONets, physics-informed FNO (PINO) has been proposed in [Li et al. \[2024b\]](#) and [Konuk and Shragge \[2021\]](#). Unlike DeepONets, FNO outputs a grid, and the process uses fast Fourier transform, which prevents the use of automatic differentiation. However, other approaches are available:

- Using conventional numerical schemes such as finite differences and Fourier gradients. Finite differences require an output grid that is fine enough to give good precision, while Fourier gradients require smoothness and a regular grid.
- Define gradients in Fourier space and then use the chain rule to calculate the required gradients.

[Li et al. \[2024b\]](#) chose to implement gradients in Fourier space, which gives a good physically informed model. However, the proposed model is still not a universal solution, as depending on the runtime/relative error trade-off, the curve intersects with the GPU-implemented solver for the problems tested.

Chapter 3

Numerical simulations for Free Surface Flows

3.1	A Sketch of Numerical Methods	35
3.2	Free Surface Flows theoretical modeling	37
3.2.1	Notations and geometry	37
3.2.2	Saint-Venant equations	37
3.2.3	Boundary conditions	39
3.3	Telemac2D	40
3.3.1	An overview of the method	40
3.3.2	Advection Step	41
3.3.3	Propagation, diffusion, source terms	42
3.3.4	Finite elements	43
3.3.5	Linear system resolution	44
3.3.6	Treatment of uncovered bed	44
3.4	Practical Application	45
3.4.1	Pre-processing	46
3.4.2	Numerical modeling	47
3.4.3	Post-processing	48
3.4.4	Notes on flood alert system	48

This chapter presents a detailed overview of the methodologies and tools used for flood modeling, combining theoretical concepts with practical applications to provide a comprehensive understanding. It is structured to guide the reader through the complete workflow of flood prediction, as applied in this thesis and by BRLi, covering theoretical foundations, numerical modeling, and practical implementation.

The chapter begins with a concise introduction to numerical methods, providing context for the various techniques used to simulate free-surface flows. Building on this background, it explores the theoretical principles behind free-surface flow modeling, including the governing equations and key physical concepts. Following this, the chapter

introduces Telemac2D [Hervouet, 2007], an advanced computational tool widely employed in engineering for simulating flood scenarios and addressing free-surface flow problems.

The final section focuses on the practical application of these methodologies, outlining the complete modeling process. This includes pre-processing steps to prepare the data, conducting numerical simulations, and applying post-processing techniques to analyze and interpret the results. The chapter also highlights the importance of these methods in the development of flood warning systems.

3.1 A Sketch of Numerical Methods

Partial differential equations (PDEs) are essential mathematical tools for modelling a wide range of physical phenomena. They are equations involving unknown functions of several variables and their partial derivatives. PDEs help us describe how physical quantities such as temperature, pressure and speed change in space and time. For example, they can model how heat spreads through a material or how a free surface flows. To find meaningful solutions to these equations, we need to specify initial conditions and boundary conditions. Initial conditions specify the state of the system at the start time, while boundary conditions define what happens at the edges or surfaces of the region we're studying. These conditions are crucial because they ensure that the solution we obtain is unique and accurately reflects the real world situation we are trying to understand.

Except in special cases, partial differential equations usually don't have analytical solutions. So, we need to use numerical methods to find approximate solutions. The basic idea of all numerical methods for PDEs is to calculate approximate values at specific points to represent the true solution. This means we are making an approximation because we are moving from a continuous model to a set of discrete points; this process is called discretisation.

We break down our model into discrete parts in space and, if necessary, in time. Discretising time gives us a time step, which divides the time period into small intervals. Discretising space results in a mesh or grid made up of points and the connections between them, showing how these points interact. This mesh can be regular, like a grid of squares or cubes, or irregular, which helps fit complex shapes better.

Discretisation allows us to turn differential equations into algebraic equations that computers can solve. By using smaller time steps or finer meshes, we can make our approximation more accurate, but this requires more computing power. Balancing accuracy and computational effort is important when using numerical methods for PDEs.

On regular grids, we can use the finite difference method. The basic idea is to replace the derivatives in the equations with approximations from Taylor series, where we ignore the small leftover terms. For example, the Laplacian in dimension one of the function u at time t_n and position x_j can be approximated by :

$$-\frac{\partial^2 u(t_n, x_j)}{\partial x^2} \approx \frac{-u_{j-1}^n + 2u_j^n - u_{j+1}^n}{(\Delta x)^2}$$

The finer the mesh is discretised (Δx), the more accurate our approximation becomes.

We also need to decide how to discretise the time derivatives. We can choose a centered scheme, a backward difference (looking at earlier points), or a forward difference (looking at later points).

By applying these finite difference approximations across the entire grid, we transform the partial differential equation into a system of algebraic equations. This system can be represented in matrix form, where each equation corresponds to a point on the grid and relates the unknown values at that point to its neighbors. We can then use computational algorithms to solve this matrix system on a computer, finding approximate values of the solution at each grid point.

Depending on the equations and schemes we select, there's a condition to prevent numerical oscillations called the Courant-Friedrichs-Lewy (CFL) condition. This condition is an inequality between the space and time steps, telling us the necessary ratio between them to achieve a stable result. We won't delve further into finite differences, as they aren't suitable for irregular meshes needed to create reasonably sized grids for simulating multi-scale phenomena like river floods. For more details see [Allaire \[2007\]](#).

To address the limitations of the finite difference method in handling irregular meshes and complex geometries, we turn to the finite volume method. Unlike the finite difference method, which approximates derivatives, the finite volume method focuses on approximating integrals to solve partial differential equations. It divides the study area into a mesh made up of small, separate volumes in 3D (surfaces in 2D, segments in 1D) that together cover the whole domain. By integrating the equations over each of these finite volumes and applying principles like the divergence theorem, the method turns the differential equations into algebraic equations that represent the flow of quantities across the boundaries of the volumes. Next, it approximates these flows and any source terms within the volumes. For problems that change over time, it uses methods to step forward in time. Then, it assembles these equations into a system and uses numerical methods to solve them, finding an approximate solution to the original problem. Finally, the results are processed to analyse and visualise the solution. This makes the finite volume method especially useful for problems in areas like fluid dynamics and heat transfer, where accurately tracking how quantities move through space is important. Software like HEC-RAS [\[Brunner, 1997\]](#) employs finite volume method for flood modeling due to these advantages. However, in our study, we utilize the finite element method, which offers its own set of benefits for simulating free-surface flows. Therefore, we will not delve further into finite volume methods in this chapter. Interested readers can consult [Darwish \[2016\]](#) for a more comprehensive understanding of finite volume method.

The method presented in Section 3.3 uses finite element methods. The idea is to use a variational approach, employing the weak formulation of the equations involved. This involves replacing the space in which the variational approach is formulated with a finite-dimensional subspace. The approximate problem on this subspace then reduces to solving a linear system. The key aspect of finite elements lies in constructing this subspace so that it is "close" to the exact solution. The finite element method yields good results on a wide variety of problems using different types of unstructured meshes. For a detailed approach of the finite elements the reader is referred to [Bathe \[2006\]](#).

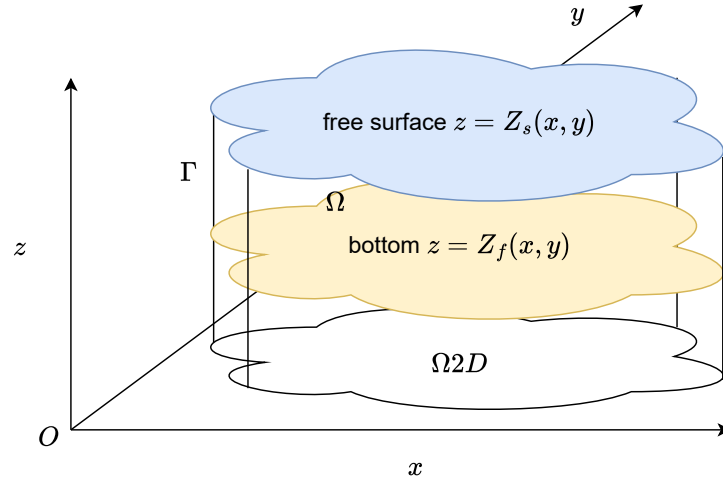


Figure 3.1: Calculation domain

3.2 Free Surface Flows theoretical modeling

3.2.1 Notations and geometry

Our study domain Ω is defined in the Cartesian referential $R = (O, x, y, z)$ where the axis Oz is vertical. Its lower boundary, in practice the bottom of our river, is defined by $z = Z_f(x, y)$, and the free surface is defined by $z = Z_s(x, y)$. It is bounded on its sides by vertical lines which are its boundaries denoted by Γ . We will call the projection of our domain onto the plane (O, x, y) as Ω_{2D} , and our domain is thus defined by:

$$\Omega = \{(x, y, z) \in \mathbb{R}^3 \mid (x, y) \in \Omega_{2D}, Z_f(x, y) \leq z \leq Z_s(x, y)\}$$

Gravity is oriented in the downward direction of z . The water height will be denoted by h and is equal to $Z_s - Z_f$. Time will be denoted by t .

3.2.2 Saint-Venant equations

Flood modeling, and more broadly the modeling of free-surface flows, has been a topic of great importance for a long time. In 1871, Adhémar Barré de Saint-Venant introduced the equations that now bear his name, also known as the shallow-water equations. These equations are derived from the Navier-Stokes equations under the specific condition of a thin layer of water in the vertical direction (z) compared to its width and length (x and y). From this condition, several assumptions are made:

- **Hydrostatic water pressure:** The velocity in the z -direction is negligible, meaning that the pressure at any given point depends solely on the height of the water column above it. Thus, the pressure at the water's surface is equal to atmospheric pressure.

- **Negligible vertical velocity:** This assumption aligns with the previous one and is justified by the aim to model primarily horizontal flows, such as rivers.
- **Impermeable bed and surface:** This assumption implies that porosity does not result in water loss.

Under these assumptions, the Barré de Saint-Venant equations are obtained by vertically integrating the Navier-Stokes equations. The values of u and v , which represent the velocities in the x and y directions for simplicity, are in fact vertically integrated (or averaged) velocities in these directions. Applying this process yields the following equations.

$$\frac{\partial h}{\partial t} + \vec{u} \cdot \overrightarrow{\text{grad}}(h) + h \text{div} \vec{u} = \text{Sce} \quad (3.1)$$

$$\begin{aligned} \frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} + v \frac{\partial u}{\partial y} &= -g \frac{\partial Z_s}{\partial x} + F_x + \frac{1}{h} \text{div}(h \nu_e \overrightarrow{\text{grad}}(u)) + (u_{sce} - u) \frac{\text{Sce}}{h} \\ \frac{\partial v}{\partial t} + u \frac{\partial v}{\partial x} + v \frac{\partial v}{\partial y} &= -g \frac{\partial Z_s}{\partial y} + F_y + \frac{1}{h} \text{div}(h \nu_e \overrightarrow{\text{grad}}(v)) + (v_{sce} - v) \frac{\text{Sce}}{h} \end{aligned} \quad (3.2)$$

This formulation, commonly referred to as the depth-velocity formulation, is one of several possible representations. We present this particular expression because it is employed by the modeling software TELEMAC2D, which will be utilized in this chapter. The first equation, (3.1), is known as the continuity equation, while the other two equations, (3.2), are referred to as the momentum equations. When water enters or exits the system through the free surface, within the domain, or at the bottom, such as through rainfall or groundwater infiltration, the continuity equation incorporates a term denoted as Sce (representing a "Source," which may also function as a sink). This term, Sce , is measured in meters per second and is positive when water is introduced into the domain, as with precipitation. Additionally, u_{sce} and v_{sce} represent the momentum components associated with the source term in the x and y directions, respectively. The terms $(u_{sce} - u) \frac{\text{Sce}}{h}$ and $(v_{sce} - v) \frac{\text{Sce}}{h}$ are zero if the velocity at the source is the same as the current, they will be included in the expression F_x and F_y and will therefore not appear hereafter. The terms F_x and F_y in the momentum equations account for external forces, such as the Coriolis effect or other external influences. The gradient of Z_s corresponds to the pressure gradient, which, under shallow-water assumptions, represents the hydrostatic pressure distribution. Frictional effects, including those described by the Strickler coefficient, are incorporated into F_x and F_y and should be appropriately defined.

3.2.3 Boundary conditions

A system of partial differential equations has a solution only if the appropriate boundary conditions are defined. In our case, we can distinguish two types of boundary conditions: rigid and liquid boundaries.

Rigid boundaries

This type of boundary models walls or associated physical objects, which impose the condition that no flow should cross the boundary. Consequently, the boundary is impermeable, and only friction occurs along it. The friction can either be provided by the user or computed by the turbulence model. Mathematically, this condition is expressed by imposing the velocity on the component tangent to the wall, depending on a friction coefficient.

Liquid boundaries

Liquid boundaries represent sources that influence the domain but do not belong to it. An example of this would be an inflow from the upstream of a river. By analyzing the advection term in the 1D Saint-Venant equations using the method of characteristics [Hervouet \[2007\]](#), we can precisely categorize these boundaries as input/output and supercritical/subcritical, and specify the conditions for which the system is well-defined. However, in practice, this categorisation does not directly translate to two dimensions, and the nature of the flow (input or output) can vary over time.

In practice, the following conditions are used:

- Imposed velocity and imposed water depth
- Imposed velocity and free water depth
- Free velocity and water depth
- Free velocity and imposed water depth

These are basic conditions; however, the available data is not always in the form of depth or velocity. This is the case for flow conditions where the known input of the river is a hydrograph. To address this issue, one can use a velocity profile and retrieve the velocity values based on proportionality with the known discharge, which requires knowledge or assumptions about the velocity profile.

3.3 Telemac2D

In the previous section, we introduced the Saint-Venant equations, specifically in their depth-velocity form. However, it is important to note that this is just one of several formulations; others include the conservative form and the celerity-velocity form. Additionally, a wide range of numerical methods are available to solve these equations, such as finite differences, finite volumes, finite elements, among others. The combination and coupling of these methods has led to the development of numerous numerical schemes, each with its own strengths and weaknesses.

This section focuses on one specific numerical scheme: Telemac2D. This solver, widely used by organizations like BRLi, is the primary tool that will be employed throughout this study. Telemac2D is part of the TELEMAC-MASCARET modeling system, a robust and

comprehensive suite designed for simulating free-surface flows. Originally developed by the Laboratoire National d’Hydraulique within the Research and Development division of Electricité de France (EDF), it is now available as open-source software and is maintained by the Open Telemac-Mascaret Consortium.

The aim of this section is to provide a concise overview of Telemac2D, with a focus on its key mechanisms. Understanding these core components, as well as the parameters that can be adjusted within the solver, is essential for leveraging its full capabilities in hydraulic modeling.

The information we share here comes from Jean-Michel Hervouet’s book [Hervouet \[2007\]](#). For more detailed explanations, we recommend referring to his work directly.

3.3.1 An overview of the method

In the Telemac2D solution process, the governing equations are solved in two stages using a fractional steps method. The first stage addresses the advection terms, which are solved using the method of characteristics. In the second stage, the diffusion and source terms are handled using the finite element method. Additionally, if one chose to not use the method of characteristics, the advection terms can also be treated within this step. For clarity, we will assume that the method of characteristics is applied to the advection step.

Let us consider a temporal discretisation starting from an initial time t^0 with a time step Δt , such that

$$t^n = t^0 + n\Delta t.$$

Let f be a generic function representing the variables h , u , and v (water depth and velocity components, respectively). The solution procedure using the fractional steps method can be described as follows:

Step 1: Advection Step The first step solves the advection part of the equations. This is expressed as:

$$\frac{\tilde{f} - f^n}{\Delta t} + \text{advection terms} = 0, \quad (3.3)$$

where f^n represents the value of f at time t^n , and $\tilde{f}$ is an intermediate value computed after the advection step.

Step 2: Diffusion and Source Terms Step In the second step, the diffusion and source terms are treated, leading to the following equation:

$$\frac{f^{n+1} - \tilde{f}}{\Delta t} + \text{other terms} = 0, \quad (3.4)$$

where f^{n+1} is the value of f at the next time step $t^{n+1} = t^n + \Delta t$.

3.3.2 Advection Step

The equations we want to solve are:

$$\frac{\tilde{h} - h^n}{\Delta t} + \mathbf{u} \cdot \nabla h = 0, \quad (3.5)$$

$$\frac{\tilde{u} - u^n}{\Delta t} + \mathbf{u} \cdot \nabla u = 0, \quad (3.6)$$

$$\frac{\tilde{v} - v^n}{\Delta t} + \mathbf{u} \cdot \nabla v = 0. \quad (3.7)$$

The basic principle of the method of characteristics is to adopt a Lagrangian point of view and state that the value of a function at time t^{n+1} on a node M of the mesh is equal to the value of the same function at time t^n at a point Q . The main challenge is to identify the correct point Q , called the foot of the characteristic. The characteristic equation is:

$$\frac{d\mathbf{x}}{dt} = \mathbf{U}.$$

However, calculating the trajectory poses a problem of time order because:

$$\mathbf{U}(t) = \mathbf{U}^n + \frac{t - t^n}{\Delta t}(\mathbf{U}^{n+1} - \mathbf{U}^n),$$

but the advection field is not known at t^{n+1} . In order to save computational resources, a fixed-field hypothesis is made ($\mathbf{U}^{n+1} = \mathbf{U}^n$), as approximating $\mathbf{U}^{n+1}$ is costly.

Here is the trajectory calculation explained briefly. Each time step is divided into sub-time steps $\frac{\Delta t}{n_{\text{sub}}}$ during which the velocity remains constant (n_{sub} is calculated locally depending on how long we need to pass through each element). Let's take a point $A = (X_{\text{start}}, Y_{\text{start}})$ on a trajectory; the next point is found by the following formula:

$$X_{\text{arrival}} = X_{\text{start}} - u_{\text{start}} \frac{\Delta t}{n_{\text{sub}}}, \quad (3.8)$$

$$Y_{\text{arrival}} = Y_{\text{start}} - v_{\text{start}} \frac{\Delta t}{n_{\text{sub}}}, \quad (3.9)$$

where u_{start} and v_{start} are the velocity components at point A . This process is repeated n_{sub} times, reaching the foot of the characteristic. There, the value of the variable sought (h , u , or v) is interpolated, and we obtain $\tilde{h}$, $\tilde{u}$, or $\tilde{v}$. The key to the efficiency of this method is the navigation and localization in the mesh, which is facilitated by a well-designed data structure. An option is available to use a weak form of the characteristic method, which is mass-conservative but costs more, so it should be used only in specific cases (see [Hervouet \[2013\]](#)).

3.3.3 Propagation, diffusion, source terms

At the stage we must solve :

$$\frac{h^{n+1} - \tilde{h}}{\Delta t} + h \operatorname{div} \vec{u} = \text{Sce} \quad (3.10)$$

$$\frac{u^{n+1} - \tilde{u}}{\Delta t} = -g \frac{\partial Z_s}{\partial x} + F_x + \frac{1}{h} \operatorname{div}(h \nu_e \overrightarrow{\operatorname{grad}(u)}) \quad (3.11)$$

$$\frac{v^{n+1} - \tilde{v}}{\Delta t} = -g \frac{\partial Z_s}{\partial y} + F_y + \frac{1}{h} \operatorname{div}(h \nu_e \overrightarrow{\operatorname{grad}(v)}) \quad (3.12)$$

where $\tilde{h}$, $\tilde{u}$ and $\tilde{v}$ are the result of the advection of h, u and v .

At the stage, it's not specified if the term h, u and v are variable at time t^n or t^{n+1} . We can write these variable as :

$$f = \theta f^{n+1} + (1 - \theta) f^n \quad (3.13)$$

with $\theta \in [0, 1]$. When $\theta < 0.5$, the time discretisation scheme becomes predominantly explicit, leading to potential stability issues, similar to a fully explicit method ($\theta = 0$). Explicit schemes are highly sensitive to time step size and can become unstable unless small time steps are used, particularly in the presence of nonlinearities or boundary conditions. While $\theta = 0.5$ is theoretically ideal and second-order accurate, it may still be marginally stable in practical scenarios, especially with nonlinear problems. To ensure both stability and accuracy, it is recommended to choose θ slightly greater than 0.5, providing a balance between stability and the desired second-order accuracy (see [Hervouet \[2007\]](#)). Writing these variable under this form will be called semi-implicitation.

Sub-iterations The semi-implicit method does not fully resolve the issue of nonlinearity in the term $h \operatorname{div}(\vec{u})$, which involves a product of the unknowns h^{n+1} and $\vec{u}^{n+1}$. If left untreated, this would lead to a nonlinear system, which is computationally expensive to solve. To avoid this, the term is linearized as follows:

$$h \operatorname{div}(\vec{u}) = h_{prop} \operatorname{div}(\theta_u \vec{u}^{n+1} + (1 - \theta_u) \vec{u}^n),$$

where θ_u is a relaxation coefficient controlling the weighting between future velocity $\vec{u}^{n+1}$ and current velocity $\vec{u}^n$, and h_{prop} is the "height of propagation." Initially, h_{prop} is set to h^n to keep the system linear in the first solve.

To improve accuracy and bring h_{prop} closer to $\frac{h^n + h^{n+1}}{2}$, an iterative process is introduced within each time step. After solving the system once, h_{prop} is updated using the formula:

$$h_{prop} = \theta_h h'^{n+1} + (1 - \theta_h) h^n,$$

where θ_h is a relaxation coefficient for water depth, and h'^{n+1} is the current estimate of h^{n+1} from the latest sub-iteration. Initially, $h'^{n+1} = h^n$, but after each sub-iteration, h'^{n+1} is updated to better approximate h^{n+1} .

This process involves solving the system obtained from the equations multiple times within the same time step. Each sub-iteration refines the estimate of h^{n+1} and updates h_{prop} , while maintaining a linear system at each step. This iterative approach allows the method to handle the nonlinearity efficiently without solving a fully nonlinear system.

3.3.4 Finite elements

After applying all the approximations, we proceed with a finite element method formulation. This process involves integrating the governing equations over the computational domain using a set of chosen test functions. By multiplying each equation by its corresponding test function and integrating, we convert the partial differential equations into their weak form. The unknowns, such as h^{n+1} , u^{n+1} , and v^{n+1} , are then approximated by projecting them onto a finite-dimensional basis defined by the chosen finite element space.

This projection transforms the problem into a system of algebraic equations, typically expressed as $AX = B$, where A is the system matrix, X is the vector of unknowns, and B contains known terms from the equations.

In constructing the matrix A , a mass matrix appears, representing the interactions between the basis functions. The mass matrix is generally non-diagonal, but for computational efficiency, it can be lumped. Mass lumping involves replacing the mass matrix with a diagonal matrix D , where each diagonal entry D_i is the sum of all entries in the corresponding row of the original mass matrix.

The main benefit of mass lumping is that it increases the diagonal dominance of the system matrix, improving the conditioning of the linear system and making it easier to solve, especially with iterative solvers like the conjugate gradient method. While this introduces a small reduction in accuracy due to the approximation, the trade-off often results in faster computations and improved stability, which is valuable in applications where performance is a priority over minimal precision loss.

3.3.5 Linear system resolution

We now need to solve a linear system, where the matrix A generated by the finite element method is a sparse matrix, meaning most of its entries are zero. This sparsity is beneficial because it allows for specialized storage techniques that only retain the non-zero elements, reducing memory requirements. Additionally, the matrix can be divided into smaller submatrices, enabling more efficient parallel computation.

As the number of variables increases, direct solution methods become less feasible. These methods require operations on the entire matrix, which is both time-consuming and memory-intensive. Moreover, assembling the full matrix A can be a resource-demanding process, making direct methods less practical for large-scale problems.

In contrast, iterative methods provide a more efficient approach, particularly for sparse systems. These methods mainly rely on matrix-vector multiplications, which align well with sparse matrix formats. However, their efficiency is sensitive to the conditioning

of the matrix A . Poor conditioning can result in slow convergence or even failure to converge.

Several iterative methods can be employed, such as the conjugate gradient method for symmetric matrices, the conjugate gradient on normal equations method, and Generalized Minimal Residual Method. Each method has specific advantages depending on the problem and the properties of the matrix.

To enhance the conditioning of the system and accelerate the convergence of iterative solvers, a preconditioner is often introduced. A preconditioner transforms the system into one that is better conditioned, making it easier to solve iteratively. Once the preconditioned system is solved, the solution is mapped back to the original variables.

3.3.6 Treatment of uncovered bed

In hydrodynamic simulations, managing dry zones or uncovered beds (areas where the water depth is zero or near zero) poses a significant challenge. Applying standard equations to these regions can yield non-physical outcomes, necessitating specialized techniques for accurate treatment.

A frequently employed approach involves setting a minimum threshold for water depth. When the calculated depth drops below this threshold, it is artificially raised to the minimum value. While this method helps avoid computational issues such as division by zero or singularities in the equations, it introduces notable drawbacks.

Firstly, arbitrarily adjusting the water depth violates the principle of mass conservation. By artificially increasing the water depth, the total volume of water in the system is altered, leading to inaccuracies in the simulation outcomes. This, in turn, impacts flow dynamics and water level predictions in other parts of the domain.

Secondly, applying momentum conservation equations to an extremely thin layer of water—where the minimum depth is artificially maintained—can result in non-physical velocities. These parasitic velocities can propagate into fully submerged zones, distorting the flow field and compromising the accuracy of predictions regarding sediment transport and overall simulation performance.

To address these challenges, two alternative approaches have been developed:

1. **Correction of Invalid Terms in the Equations**

This method involves continuing to solve the governing equations across the entire computational domain, including both wet and dry regions. However, terms that become invalid in the absence of water are identified and corrected. For instance, in dry zones, the free surface gradient, which represents the slope of the water surface that drives flow, is not meaningful because there is no water. In such cases, the free surface gradient effectively becomes the bottom gradient, leading to erroneous driving forces in the simulation. By adjusting these problematic terms, such as setting the free surface gradient to zero in dry areas, this method prevents the generation of parasitic driving forces while preserving the integrity of the equations in wet regions.

2. Exclusion of Dry Elements from Calculations

The second approach dynamically adjusts the computational domain by excluding or deactivating elements that are not fully submerged. By removing these dry elements from the calculations, the governing equations are applied only in regions where water is present. This strategy eliminates issues associated with zero or near-zero depths, such as numerical instabilities and parasitic velocities. It also requires continuous updates to the computational mesh as water levels change, ensuring that the active computational domain accurately represents the wetted area at each time step.

3.4 Practical Application

The aim of this section is to present the workflow I observed during my PhD, which was employed by engineers at BRLi for flood modeling. The workflow can be divided into three main stages:

- Pre-processing
- Numerical modeling
- Post-processing

The process described here is based on the use of TELEMAC2D for numerical modeling, along with QGIS and BLUEKENUE for pre-processing tasks. However, various other methods and software tools may be utilized, depending on the specific requirements of the study and the data inputs available.

3.4.1 Pre-processing

The purpose of the pre-processing step is to transform the physical problem into data that can be interpreted by the numerical modeling software, in this case, TELEMAC2D.

The whole goal of this stage is to generate a computational mesh that accurately represents the entire domain to be analyzed. Creating an effective mesh requires significant practical expertise and domain knowledge to correctly identify key areas of interest. The goal is to determine which elements in the environment will significantly affect flooding and to model them appropriately. For example, decisions must be made regarding how detailed the representation of boundaries should be, whether bridge piers need to be included, and how the mesh resolution should be adjusted based on the characteristics of the flood event. A well-designed mesh is essential, as it directly influences the accuracy and efficiency of the simulation. High-resolution meshes may be necessary in areas with complex hydraulic features, while coarser meshes can be used in less critical regions, balancing computational cost and precision. Achieving this balance is crucial, as in practice, the computational resources available to a company are often limited. Local machines are frequently used to their full capacity, and access to external resources, such as cloud computing or high-performance computing centers, can be costly. Consequently, both

computational time and resources are valuable assets that must be carefully managed. Engineers need to consider these constraints when making decisions, ensuring that the simulation's resolution and complexity are optimized without exceeding resource limitations.

The process of mesh preparation begins with the use of QGIS, an open-source Geographic Information System that allows for the creation, visualization, and manipulation of spatial data. In this workflow, QGIS plays a crucial role in preparing data for mesh generation in software such as BLUE KENUE, specifically through the definition of density zones and breaklines. Density zones in QGIS represent areas on the map where the mesh resolution must be adjusted according to the complexity of the terrain or the significance of specific features. These zones are manually delineated by the user, highlighting regions that require finer mesh elements due to intricate details such as urban infrastructure or natural formations. Conversely, regions with less critical features or relatively uniform terrain can be assigned a coarser mesh to optimize computational resources. Breaklines, on the other hand, are linear features representing key terrain specificities such as riverbanks, ridges, roads, embankments, or shorelines. Users digitize these lines in QGIS to ensure that mesh elements are aligned along these significant features. The incorporation of breaklines is essential for maintaining the physical integrity of terrain features within the computational model, ensuring that topographical details are accurately represented.

Once the density zones and breaklines are defined in QGIS, the next step is to integrate this data into Blue Kenue. The density zones guide the mesh generator in applying finer or coarser mesh elements where necessary, based on the complexity and importance of the terrain features identified in QGIS. Similarly, the breaklines ensure that mesh elements conform to significant terrain features, preserving their geometry and ensuring that the mesh aligns with real-world topography. After generating the mesh, elevation data, such as from digital elevation models, is interpolated onto the mesh in Blue Kenue to provide it with a three-dimensional aspect that accurately reflects the actual topography or bathymetry. The Strickler index, which represents the roughness coefficient of the riverbed, is also interpolated onto the mesh. This index influences flow dynamics by affecting velocity and turbulence, and spatial data containing Strickler values can be imported to simulate how variations in surface roughness impact water movement.

A critical step in preparing the mesh for hydrodynamic modeling is the definition of boundary conditions. These conditions specify how water enters and exits the model domain and may include parameters such as inflow rates, water levels, or velocity profiles. In Blue Kenue, boundary conditions are applied by assigning appropriate parameters to specific edges or nodes of the mesh, ensuring that the model behaves realistically during the simulation. Finally, Blue Kenue provides tools for visualizing and refining the mesh, allowing users to review the structure, inspect elevation and Strickler values, and verify boundary conditions, ensuring the accuracy and reliability of the model before running simulations.

3.4.2 Numerical modeling

At this point in the workflow, we have generated two key files that are fundamental for hydrodynamic modeling. The first is the `.slf` file, commonly known as the Selafin file, which contains the mesh along with the interpolated elevation data and the Strickler roughness coefficients assigned to each mesh point. This file essentially encapsulates the physical and hydraulic characteristics of the river system within the computational domain. The second file is the `.cli` file, which holds the boundary condition information. Specifically, it includes the indices of the boundary points in the mesh and categorizes them as inflow, outflow, wall, or other types of boundary conditions.

At this stage, one important task remains: assigning actual values to the boundary conditions. While the `.cli` file specifies the location and type of each boundary point it does not provide the quantitative data necessary to simulate flows. To ensure the hydrodynamic model operates successfully, we must define values such as inflow discharge rates, water surface elevations, or velocity profiles for each boundary point.

To accomplish this, we generate a `.liq` file. This file contains the time-series data required for the simulation, including hydrographs for inflow boundaries and prescribed water levels for outflow boundaries. The data is organized to correspond with the boundary indices defined in the `.cli` file and formatted so that the model can correctly interpret these values. This step completes the boundary condition setup.

We are now ready to begin our simulations. However, running a numerical model like Telemac2D requires configuring numerous parameters to ensure accurate results. For example, the value of θ used in implicit schemes, the activation of mass lumping, the precision settings, and the choice of solver must be carefully selected (3.3.3,3.3.4,3.3.5). All these choices are made based on the engineer's experience and depend on the specific characteristics of the model being run.

3.4.3 Post-processing

Engineers leverage the outputs of Telemac 2D simulations to predict and assess flood events by integrating specialized software with custom Python scripts tailored for data analysis. After running a simulation, they extract key data such as water depths, flow velocities, and directions at various time intervals. Before turning these results into information through maps or reports, they rigorously analyze the data to verify that the model accurately represents critical flood characteristics, including maximum water levels reached, the extent of inundated areas, and the event's kinetics (correct propagation). To ensure the accuracy of the Telemac model, engineers undertake an essential calibration process using historical flood data and real-world observations. This involves modifying the model parameters such as the Strickler roughness coefficients, structural elements, and hydraulic works to closely match observed flood events. By re-simulating historical events and comparing the results with flood mark surveys, they validate the model's ability to accurately depict maximum water levels reached, the extent of flood-prone areas. The objective is to validate the model so that it represents reality as closely as possible, which is crucial for reliable flood risk assessments.

3.4. Practical Application

Tools like Fudaa and QGIS are then used to visually represent the data, generating flood extent maps, water depth contours, and velocity vectors, which aid in the interpretation of the simulation results.

Custom Python scripts play a crucial role in this workflow by handling tasks such as statistical analysis, data manipulation, and plotting. These scripts can also interface with GIS platforms to overlay the simulation results onto real-world maps, enabling detailed spatial analyses.

For flood risk assessments, engineers develop flood inundation maps that predict which areas could be affected under different flood scenarios. By analyzing the simulation data (especially water depths and flow velocities) they evaluate potential risks to infrastructure, property, and human safety.

Effective reporting and communication are central to this process. The analyzed data and visualizations are compiled into reports designed to convey findings clearly to stakeholders, policymakers, and the public. The combination of detailed visualizations and succinct analysis ensures that complex information is easily understood.

3.4.4 Notes on flood alert system

Flood alert systems play a vital role in protecting communities, infrastructure, and economies from the destructive effects of floods. Early warnings allow both authorities and residents to take preventive measures, reducing potential damage and enhancing safety. To meet this need, public authorities are investing in better flood monitoring and forecasting systems, such as the Vigicrues network. Vigicrues operates around the clock, with flood forecasters collecting and analyzing real-time data on rainfall, water levels, and river flows. By combining this data with hydraulic models, the system can create detailed flood maps that predict where and how deep floods will occur, providing critical information for decision-making.

However, effective flood warning systems must also overcome challenges related to flood modeling and simulation. These systems depend on both hydrological data (which predicts how much water will enter a basin) and hydraulic calculations (which predict how that water will move and accumulate).

Engineers typically use one of two key strategies to ensure timely and accurate flood predictions. The first strategy simplifies the computational model to make it less resource-intensive and faster to run. This might involve using a coarser grid or fewer elements in the model, or simplifying some of the physical processes it represents. Although this allows for quicker forecasts using real-time data from hydrological forecasts, it can compromise the level of detail and accuracy in predictions. One-dimensional (1-D) models are commonly used because of their low computational cost, but they have limitations in accurately representing complex water flow in floodplains.

The second approach involves running numerous simulations in advance, covering a wide range of potential flood scenarios. These pre-calculated scenarios include different variables such as rainfall intensity, river discharge levels, and potential breach locations. During an actual flood event, the system matches current conditions to the pre-simulated scenarios, allowing for immediate predictions without needing to run new

3.4. Practical Application

simulations. While this method delivers fast results, its accuracy depends on whether the pre-simulated scenarios cover all possible real-world conditions.

To create a more effective flood alert system, engineers may need to combine or balance these approaches, optimizing between fast processing and precise predictions. The choice of strategy depends on the system's objectives, available computational resources, and the need for quick warnings in areas at risk.

Chapter 4

Deep Learning surrogates for Geosciences

4.1	Surrogate Models for Weather Forecasting	50
4.1.1	FourCastNet	51
4.1.2	GraphCast	52
4.1.3	Pengu-Weather	53
4.1.4	Recent developments	54
4.2	Modern application to Flood-prediction	56
4.2.1	Shallow Water Equation–Graph Neural Network	57
4.2.2	FloodSformer	58
4.2.3	GeoPINS	59

In this chapter, we narrow our focus to the application of deep learning surrogates for PDEs specifically within the geosciences. Building on the foundational discussion in Chapter 2, which addressed deep learning approaches for PDEs broadly, this chapter concentrates on geoscientific contexts where these surrogates play an impactful role.

The first section examines the rise of surrogate models in weather forecasting. The availability of large volumes of high-quality data, especially from initiatives like the European Centre for Medium-Range Weather Forecasts (ECMWF), has significantly driven advancements in this field. This data abundance has sparked considerable interest across both industry and academia, leading to transformative developments in deep learning models capable of approximating and predicting complex atmospheric patterns with impressive speed and precision.

In the second section, the focus shifts to the main objective of this PhD research: flood prediction. Here, we trace the evolution of surrogate models for flood forecasting, with a particular emphasis on recent advancements that harness deep learning to improve the speed and accuracy of flood modeling. These innovations are paving the way toward practical, real-time flood alert systems that hold the potential to save lives and reduce property damage by delivering prompt and accurate forecasts.

4.1 Surrogate Models for Weather Forecasting

All the papers mentioned in the Chapter 2 have in common that they mainly relied on manually created, simple databases. This fact has limited the impact of the proposed method in the numerical physics world. In 2019, the European Centre for Medium-Range Weather Forecasts released the ERA5 dataset. It provides hourly estimates of atmospheric, land and oceanic climate variables. These data are available on a 31 km grid covering the Earth with 123 levels in the atmosphere. These data are derived from a combination of historical observations, modelling systems and data assimilation. This huge and high quality database has attracted the attention of the biggest players in the deep learning scene, who are looking for real-world problems to demonstrate their skills using the increasing GPU power available.

4.1.1 FourCastNet

The first breakthrough was achieved with an Nvidia-supported paper [Pathak et al. \[2022\]](#) presenting FourCastNet. The authors treated the ERA5 data as a latitude-longitude grid of 720×1440 pixels, with weather variables at each level. FourCastNet begins by dividing this grid into smaller, fixed-size patches. Each patch is then flattened and linearly transformed into a high-dimensional vector. To retain the spatial information of the grid, positional encodings are added to each vector. These vectors are then fed into a network based on the Adaptive Fourier Neural Operator (AFNO) [Guibas et al. \[2021\]](#). These layers combine elements of a vision transformer and Fourier Neural Operator (FNO). The core idea involves applying a Fourier transform to the input and using a multi-layer perceptron with block-diagonal matrices, where each block functions similarly to self-attention. Thresholding and shrinkage operations are applied, followed by an inverse Fourier transform to revert from the Fourier space, incorporating a residual connection. Finally, a linear decoder is used to reconstruct the information back into the original grid format. The model is trained to predict 6h hours far away on a subset of the datas of the whole dataset, the model is then fine tuned to predict two step ahead with precision. The end to end training takes about 16 hours wall-clock time on a cluster of 64 Nvidia A100 GPUs. FourCastNet achieves forecasting accuracy comparable to the ECMWF Integrated Forecasting System (IFS), a leading Numerical Weather Prediction model, for large-scale variables at short lead times, and surpasses IFS in predicting small-scale variables, such as precipitation. Additionally, FourCastNet can generate a week-long forecast in under 2 seconds, making it significantly faster than IFS by several orders of magnitude. This is milestone in the surrogate model for physics : a deep-learning model solve a complex real-life case in real-time.

4.1.2 GraphCast

Another method was propose by Google DeepMind with a model called GraphCast [Lam et al. \[2022\]](#). This massive GNN-based model of 36.7 million parameters produced spectacularly accurate results for 10-day weather forecasts, running them in under a minute.

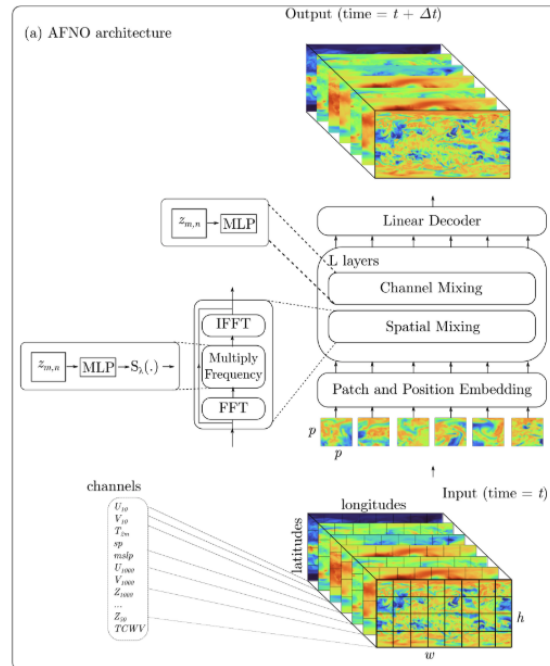


Figure 4.1: FourCastNet architecture detailed from Pathak et al. [2022]

By comparison, the High Resolution Forecast weather simulation system takes hours on a supercomputer, so how does it work? GraphCast is an encoder/processor/decoder architecture that uses two representations: the grid given by ERA5 and a multi-mesh. The multi-mesh is created by dividing a regular icosahedron iteratively to create finer

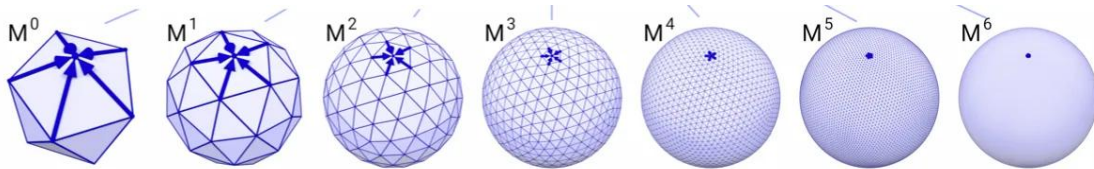


Figure 4.2: Illustration of the multi-mesh from Lam et al. [2022].

and fine mesh. This process allow information to diffuse in a local and long-distance fashion without too many message passing layer. The original grid data are encode to this 40,962 nodes 328k edges multi-mesh where 16 message passing layers process and the output is decode back to the original grid.

The loss used is a mean squared error averaged over the forecast time, forecast range, locations and all variables. These variables are weighted according to several criteria:

- Inverse residual variance to compensate for the effect of the unit of measurement
- Variable level weighting, because some levels empirically influence the weather more

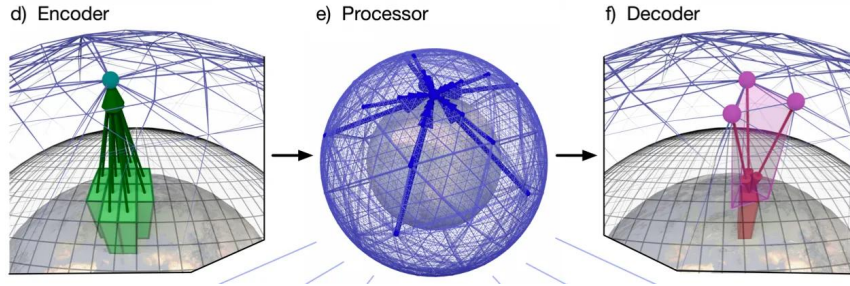


Figure 4.3: Illustration of the encode/process/decode of Graphcast from [Lam et al. \[2022\]](#).

- Coordinate weight to balance the density of points available in different areas (more points at the pole than at the equator)

Most of the time the model is trained to predict one step ahead (6 hour forecast). This training phase takes 3 weeks on 32 TPUs in parallel. The model is then fine-tuned for one week to predict 3 days ahead.

4.1.3 Pengu-Weather

In the same year, Huawei Cloud launched its advanced AI-based weather forecasting system, Pangu-Weather [Bi et al. \[2023\]](#). This architecture uses a variant of the transformer originally developed for image processing, treating the ERA5 variables as 1440×721 resolution images. The upper air variables, which have 5×13 levels, and the surface variables, which have 4 levels, are processed separately. These images are then put through a patch embedding process, where each image is divided into smaller fixed size patches, transformed into embeddings and merged. As a result, the input data becomes a volume of size $8 \times 360 \times 181$, with each point containing an embedding of size $C = 192$. This transformation into a 3D volume allows the model to better understand the interactions between different atmospheric levels. These volumes are processed through an encoder-decoder structure consisting of transformers inspired by the Swin Transformer [Liu et al. \[2021\]](#). Finally, the data are separated back into upper air and surface variables and returned to their expected dimensions using the inverse transformation of patch embedding, known as patch recovery. The whole process is summarized in Fig. 4.4. The method relies on four

4.1. Surrogate Models for Weather Forecasting

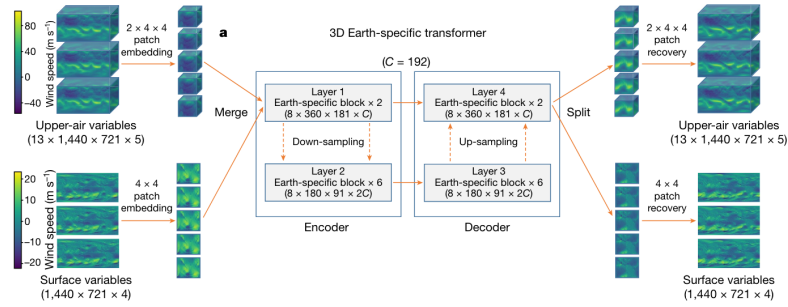


Figure 4.4: Pangu architecture detailed from [Bi et al. \[2023\]](#)

trained models which predict up to 1, 3, 6, and 24 hours ahead. The training process employs hierarchical temporal aggregation to predict the desired time. For example, to predict up to 56 hours ahead, they use the 24-hour model twice, the 6-hour model once, and the 1-hour model twice. These models were trained on a cluster of 192 NVIDIA Tesla-V100 GPUs over 16 days. The comparison with FourCastNet shows that, although Pangu-Weather is way more precise on a 5-day lead time and data (with Pangu-Weather having a restricted set of data), it is slower than FourCastNet.

In a span of two years, several methods have been proposed, allowing the early surrogate models to tackle industrial applications. This significant progress, enabling real-time predictions for a major physics problem, has been made possible by the release of a high-quality dataset and the availability of extensive computational resources for training. While such resources are not available for all physics problems, the potential of deep learning for addressing these challenges has been clearly demonstrated.

4.1.4 Recent developments

The FuXi [\[Chen et al., 2023\]](#) model present an advanced machine-learning framework for long-term weather forecasting, incorporating a specialized architecture designed to reduce forecast errors and extend predictive lead times. Central to FuXi's design is a cascading model architecture that sequences three distinct, pre-trained models tailored to specific forecast intervals: FuXi-Short (0–5 days), FuXi-Medium (5–10 days), and FuXi-Long (10–15 days). This cascade structure addresses the issue of error accumulation, a frequent limitation in long-range forecasting. Each model is fine-tuned to minimize errors within its designated forecast window, allowing FuXi to sustain high predictive accuracy across both short and extended temporal ranges. The core architecture of FuXi utilizes a U-Transformer model, which combines Swin transformers with a U-Net-inspired structure to achieve efficient downsampling and upsampling of high-resolution weather data. By limiting self-attention to local windows in each forecasting step, the U-Transformer minimizes the computational costs associated with traditional transformer models. This architecture effectively captures multi-scale spatial dependencies, which are essential for accurately modeling the atmospheric dynamics necessary for reliable weather forecasting. An enhanced version, FuXi-DA [\[Sun et al., 2024\]](#), further optimizes FuXi by integrat-

ing a robust data assimilation component, enabling the model to incorporate a variety of real-world data sources to refine initial conditions. While the base FuXi model focuses on sequential forecasting, FuXi-DA incorporates multi-modal satellite data, including information from microwave sounders and GNSS radio occultation, providing high-resolution, all-sky observations. Through a bottleneck fusion mechanism (Nagrani et al. [2021]), FuXi-DA merges these diverse data sources without conventional cloud-detection pre-processing, enhancing computational efficiency. This integration allows FuXi Weather to generate more stable and accurate long-term forecasts by mitigating error accumulation, ultimately transforming FuXi into a comprehensive, self-sustaining forecasting system.

Vaughan et al. [2024] propose Aardvark Weather : the first end-to-end data-driven weather forecasting system capable of generating predictions with no input from conventional numerical weather prediction models. It relies solely on raw observational data. Its architecture consists of an encoder, processor, and decoder, with a vision transformer backbone for the encoder and processor modules. The encoder integrates diverse observational inputs, producing a gridded initial atmospheric state, while the processor iteratively refines this state to generate multi-day forecasts. The decoder translates the processed data into localized predictions, covering variables like temperature, wind speed, and specific humidity at various pressure levels. By bypassing the computationally intensive NWP pipeline, Aardvark achieves competitive or superior forecast accuracy with reduced resource demands. Additionally, its fine-tuning capability allows for optimization of specific regions and variables, offering a flexible, scalable solution adaptable to local forecasting need

Huang et al. [2024] suggests DiffDA as a data assimilation approach that, similar to FuXi-DA, refines initial conditions in weather forecast models by using a denoising diffusion model to process large-scale atmospheric data. This technique extends systems such as GraphCast, leveraging their advanced forecasting capabilities while uniquely incorporating diffusion-based [Yang et al., 2023] conditioning for handling sparse observational data. Specifically, DiffDA integrates the pre-trained GraphCast neural network as its backbone, enabling robust forecasting performance through the autoregressive data assimilation cycle. By iteratively updating forecast states at high spatial resolution, DiffDA effectively incorporates sparse observations and achieves high forecast accuracy. This novel framework not only provides assimilated data suitable for real-world applications, such as reanalysis datasets, but also improves the initialisation of operational weather models, making it a valuable asset for improving forecast accuracy in meteorological applications.

European Centre for Medium-Range Weather Forecasts developed their own surrogate model [Lang et al., 2024], the Artificial Intelligence Forecasting System (AIFS), as part of a progressive shift towards data-driven weather forecasting. AIFS combines a graph neural network encoder-decoder structure with a sliding window transformer processor, specifically designed to handle high-resolution input data and reduce computational demands. The modular architecture of AIFS includes an encoder that aggregates information onto a lower-resolution processor grid, a transformer processor that applies shifted window attention to capture spatial dependencies across latitude bands, and a decoder

that maps the processed data back to the high-resolution grid. This architecture allows AIFS to produce accurate forecasts by effectively capturing multi-scale atmospheric dynamics, achieving skill levels that rival traditional numerical weather prediction models in both medium- and long-range forecasts.

The LUCIE [Guan et al., 2024] model offers a new approach as a lightweight, uncoupled climate emulator capable of running long-term, physically consistent simulations. Unlike traditional models that rely on large datasets and significant computational power, LUCIE remains stable over 100 years of simulation using only two years of ERA5 training data. This makes it an efficient option for climate and weather emulation. At the core of LUCIE's design is the Spherical Fourier Neural Operator, which uses spectral regularization to reduce instability and better capture atmospheric dynamics. By using a first-order integration method, the model minimizes autoregressive error growth, achieving reliable predictions of climatology, variability, and extreme events. LUCIE also excels in modeling multi-scale phenomena and can produce large ensembles for probabilistic studies, all while keeping computational demands low. These features make LUCIE ideal for rapid development and use in environments with limited resources, highlighting its innovative role in machine learning-driven weather and climate forecasting.

4.2 Modern application to Flood-prediction

The success of deep learning for weather prediction as created a craze and weather-inspired deep-learning methods for flood prediction have recently emerged. Obviously, researchers have not waited for these breakthroughs to apply deep learning to flood prediction. A complete review of the application of deep learning for flood mapping is proposed in Bentivoglio et al. [2022], which lists the main methods proposed before 2022. They classify the methods according to several criteria such as type of flood, flood mapping applications and spatial scale. In this work, we are interested in the flood categorized as river floods and flash floods and we want to obtain what is called in Bentivoglio et al. [2022] a flood hazard map. The flood hazard map is a measure of the water depths (and velocities if needed) across the area of interest for a given time. Some interesting papers presented in the review belong to these categories of methods.

We can briefly present the work of Chu et al. [2020] which proposes an MLP-based emulation modelling framework for flood inundation modelling. The study focuses on a segment of the Burnett River in Queensland, Australia, using data generated by a 2D hydrodynamic model with a spatial resolution of 20x20 meters. The framework comprises three stages: modelling problem formulation, emulation model development, and model application. Historical and design flood events provide the input data, with water depths predicted up to 8 hours ahead. The results show that the MLP models can effectively capture the time-series behavior of flood inundation, significantly reducing simulation times compared to traditional hydrodynamic models. Similarly, Kao et al. [2021] presents a model that combined a stacked autoencoder (SAE) and LSTM networks to predict water heights. This model operates on a grid representing water height on 40x40 meters cells, providing a relatively coarse spatial resolution. The data are first

processed through the encoder of the autoencoder to extract features, which are then used by the LSTM, along with rainfall data, to predict water heights for the upcoming hours. The database consists of data from Yilan County, Taiwan, including historical and designed rainfall events. The results show that the model achieves very low RMSE values (< 0.09 m) for forecasts ranging from 1 to 3 hours ahead, demonstrating its reliability and accuracy.

However, both models face significant challenges that limit their applicability and effectiveness. The ANN-based framework proposed by [Chu et al. \[2020\]](#) requires high-quality data with fine temporal and spatial resolution to accurately capture complex hydrodynamic interactions. In addition, developing a large number of models, one for each grid cell, is time consuming, computationally intensive and does not scale well. Similarly, [Kao et al. \[2021\]](#)'s model involves high complexity in the training process and requires large amounts of data to achieve optimal performance. Its coarse spatial resolution may limit its applicability in areas requiring finer resolution, and its predictive ability is currently restricted to short-term forecasts. Furthermore, the ability of both models to generalize to other regions or unobserved events remains to be verified. Despite these limitations, these works pave the way for recent studies inspired by their success and the keen interest from the weather predictions world.

4.2.1 Shallow Water Equation–Graph Neural Network

This paper [Bentivoglio et al. \[2023a\]](#) presents the Shallow Water Equation–Graph Neural Network (SWE–GNN), an Encoder/Processor/Decoder surrogate model inspired by hydraulics, which utilizes Graph Neural Networks for spatio-temporal flood modeling. The model takes advantage of the similarity between finite-volume methods used in solving shallow water equations (SWEs) and GNNs. Each finite-volume cell in a computational mesh is treated as a node, with edges representing connections between adjacent cells. The model inputs include topographical properties of the domain (e.g., elevation, slope) and initial hydraulic conditions (e.g., water depth, unit discharge). The GNN's propagation rule is inspired by the explicit numerical discretization of the SWEs. In SWEs, fluxes between cells are calculated based on hydraulic gradients. Similarly, in the SWE–GNN, fluxes are determined by a learned local function that considers the gradient of hydraulic variables between nodes. This function is expressed as:

$$s_{ij}^{(\ell+1)} = \psi(h_{si}, h_{sj}, h_{di}^{(\ell)}, h_{dj}^{(\ell)}, \epsilon_{ij}) \odot (h_{dj}^{(\ell)} - h_{di}^{(\ell)}),$$

where $s_{ij}^{(\ell+1)}$ represents the flux between nodes i and j at layer $\ell + 1$, ψ is a multi-layer perceptron (MLP) incorporating static and dynamic features, h_{si} and h_{sj} are the static node features for nodes i and j , $h_{di}^{(\ell)}$ and $h_{dj}^{(\ell)}$ are the dynamic features at layer ℓ , and ϵ_{ij} are the edge features. The term $(h_{dj}^{(\ell)} - h_{di}^{(\ell)})$ ensures water propagation based on the hydraulic gradient, similar to flux evaluation in finite-volume methods.

The model uses multiple GNN layers to overcome time-step constraints by expanding the considered spatial domain rather than increasing temporal resolution. It features

a multi-step-ahead loss function and a curriculum learning strategy, allowing for stable predictions up to 120 hours ahead based solely on the first hour's hydraulic input. The SWE-GNN model was tested against other deep-learning models, including U-Net [Ronneberger et al. \[2015\]](#), Graph Attention Networks [Veličković et al. \[2017\]](#), and graph convolutional neural network [Defferrard et al. \[2016\]](#), and consistently outperformed them in terms of Mean Average Error (MAE) and Critical Success Index (CSI) [??](#). For unseen topographies and breach locations, it achieved an MAE of 0.04 meters for water depth and $0.004 \text{ m}^2/\text{s}$ for unit discharges. The model also demonstrated remarkable computational efficiency, with speeds ranging from 100 to 600 times faster than the numerical model when run on a GPU. This model serves as a robust surrogate for numerical solvers in flood modeling, excelling in various scenarios, including unseen breach locations and larger domains. The SWE-GNN model is a useful tool for assessing flood risks and preparing for emergencies, as it provides accurate and quick flood predictions. By integrating deep learning with traditional hydraulic principles, the model enhances both interpretability and computational efficiency. However this study uses a constant breach inflow as a boundary condition on a small regular grid (from 64×64 to 128×128). Future research should extend this to time-varying boundary conditions to better mimic real-world scenarios. Additionally, future work should validate the model with real case studies, including linear elements like secondary dikes and roads, and enhance its performance for practical applications such as flood warning systems.

These promises were kept in [Bentivoglio et al. \[2024\]](#). The authors make significant progress beyond their previous work on the SWE-GNN by addressing its main limitations and introducing new improvements to enhance the model's flexibility and generalization for real-world flood modeling. One of the key advancements is the development of the multi-scale SWE-GNN (mSWE-GNN), which is specifically designed to overcome the original model's challenges, especially its limited capacity to handle complex flood dynamics across different spatial resolutions and fast-changing flood patterns.

The multi-scale approach marks a major improvement over the earlier SWE-GNN, which used a single, fixed-resolution grid, restricting its ability to effectively model areas with rapidly changing hydraulic conditions. By integrating multiple resolutions, the mSWE-GNN can simulate fine details in high-resolution zones while also capturing broader flood propagation in coarser areas. Upsampling and downsampling methods were developed to connect these different scales. This hierarchical approach not only boosts the model's accuracy but also enhances its efficiency, allowing it to simulate larger areas without proportionally increasing computational costs.

Another notable improvement is the introduction of ghost cells, which help handle time-varying boundary conditions, a major limitation of the previous model.

Beyond the multi-scale modeling and dynamic boundaries, the authors also incorporated rotation-invariant features into the model architecture, which improves the generalization ability of the mSWE-GNN.

Despite these important advancements, the authors recognize the need for more extensive testing with real-world flood events, especially those involving linear elements like roads, levees, or other flood control structures. They also note that the current test

cases remain relatively small, with the domain consisting of "only" 22,881 cells.

4.2.2 FloodSformer

This paper [Pianforini et al. \[2024\]](#) presents a new flood prediction model called FloodSformer, which combines autoencoder and transformer architectures. FloodSformer predicts long sequences of water depth maps for dam-break scenarios. The model has three main parts: an encoder, a video prediction transformer (VPTR) [Ye and Bilodeau \[2023\]](#) framework, and a decoder.

The encoder uses 2D convolutional neural network layers to extract spatial information from input maps and reduce their size, creating latent features. The VPTR framework then processes these features using a multi-headed self-attention mechanism to predict the latent features of future frames. The decoder, also using 2D CNN layers, reconstructs the predicted water depth maps from these features. This setup allows the model to predict the evolution of flood maps by learning from past frames and generating future frames one step at a time. The input to the model is a sequence of water depth maps in regular grids, and the output is the predicted water depth map for the next time step, which can be iteratively used for long-term predictions. The data used in this study consisted of regular grids with a size of 10 meters for high resolution and 20 meters for low resolution, derived from LiDAR surveys. The real-life scenario tested was the hypothetical collapse of the Parma River flood control reservoir dam in Italy. This case study covered a flood-prone area of about 45 km², from the reservoir to the city center of Parma. Two digital terrain models (DTMs) with 10 m and 20 m resolutions from LiDAR surveys were used for topographical data. Initial water levels in the upstream reservoir varied from 98 to 105.6 meters above sea level, representing different flood scenarios. The model was tested with water depth maps sampled every 2 minutes to capture rapid flood propagation typical of a dam-break event. Boundary conditions were modeled by imposing a uniform flow at the downstream boundary and setting an initially dry downstream river region. For the hypothetical dam collapse, a far-field boundary condition was applied downstream to ensure that floodwater could flow out of the simulation domain, preventing artificial accumulation at the boundaries. Different initial water levels in the upstream reservoir were considered to simulate various flood scenarios, with a constant water surface elevation as the initial condition in the reservoir. The FloodSformer model predicted flood dynamics with an average RMSE of 10.4 cm in this test case. This accuracy is suitable for real-time flood forecasting, as the computational time for forecasting 90 maps (a 3-hour lead time) was less than 1 minute. This performance shows the model's potential for emergency response situations where quick and reliable predictions are essential. However, there are several limitations. The autoregressive nature of the model can lead to error accumulation in long-term predictions, and training a transformer-based non-autoregressive model could reduce this error. Also, using a constant boundary condition in one scenario may not fully capture the dynamics of real-world situations where boundary conditions can change over time. Further research is needed to include time-varying boundary conditions to improve model accuracy and applicability in different scenarios.

4.2.3 GeoPINS

A recent work [Xu et al. \[2024\]](#) applied physics-informed deep learning to flood modeling through a comprehensive framework called FloodCast, which consists of two main components: multi-satellite observation and hydrodynamic modeling. The observation component uses real-time change detection and rainfall analysis to make the most of multi-satellite data for large-scale flood forecasting. Our primary focus is on the hydrodynamic modeling, where a geometry-adaptive physics-informed neural solver is used. This solver provides a fast and accurate approach that adapts to complex river geometries, enabling efficient flood simulations at large scales without requiring extensive training data.

The hydrodynamic model in the FloodCast framework, specifically the geometry-adaptive solver, is designed to manage flood simulations over large areas and long time periods with high efficiency. It solves the two-dimensional shallow water equations and adjusts to the complex shapes of rivers and floodplains by transforming irregular spatial domains into a regular computational grid, simplifying the process of solving partial differential equations. The model takes inputs such as spatial coordinates, time, and initial water conditions like height or velocity, which are processed using Fourier Neural Operators. These layers efficiently map inputs to outputs (such as water heights and velocities) by performing operations in the frequency domain, ensuring the predictions remain accurate regardless of the resolution. The model uses neural networks to lift the input into a higher-dimensional space, followed by several layers that capture the spatial and temporal dynamics of flooding. It also integrates boundary conditions, like river inflow, directly into the model to ensure predictions respect the physical constraints of the system. These constraints are enforced using Fourier transforms for spatial derivatives and finite difference schemes for time gradients.

The physics is embedded into the model through a combined loss function that balances fitting the data (such as observed flood extents and depths) with the physical laws of mass and momentum conservation. This ensures the model's predictions remain consistent with these laws, penalizing any deviations. By incorporating both the data and the physical principles, the model can generate realistic and accurate flood predictions.

The GeoPINS model shows strong potential for large-scale flood simulations due to its adaptive and resolution-independent design, achieving low error rates. However, it faces certain limitations. Errors tend to accumulate during long-term simulations, especially when real-time observational data is unavailable, leading to a decline in prediction accuracy over time. Additionally, while the Fourier Neural Operators are efficient, they struggle to capture rapid changes in flood dynamics, which can smooth out sharp variations in water levels and underestimate extreme events. The hard-coded boundary conditions also limit the model's flexibility in dynamic environments, particularly when inflows or outflows change quickly. Lastly, although the model is computationally more efficient than traditional methods, it still faces scalability challenges when applied to very large or high-resolution areas. Addressing these limitations is crucial to improving the reliability of real-time flood forecasting with GeoPINS.

Part II

Physics informed neural networks contributions

The field of numerical analysis for partial differential equations (PDEs) has been active for over a century. A significant milestone was set in 1928 by Courant, Friedrichs, and Lewy [Courant et al. \[1928\]](#), who demonstrated the convergence of a finite-difference scheme using a regular mesh for representing the modeled domain in Laplace’s equation. They also introduced the Courant–Friedrichs–Lewy (CFL) condition for hyperbolic equations, which relates the time step to the mesh size. This work laid the foundation for the development of the finite difference method (FDM), which saw further advancements, particularly in establishing convergence rates. With the advent of computers, FDM progressed significantly, especially in handling time-dependent problems through numerous contributions. The idea of using a variational formulation for boundary value problems dates back to the early 1900s. Building on this, Courant proposed in [Courant et al. \[1943\]](#) the use of continuous piecewise linear approximating functions based on triangulations suited to the problem’s geometry. This led to the emergence of the finite element method (FEM) in the 1950s from the engineering world as a computational technique. Since then, numerous improvements have been made, and the theory has been extensively analyzed. The large sparse matrix systems resulting from these methods, which grow with the problem’s scale, are not well-suited for direct methods. Therefore, iterative methods were adapted to solve them. The properties of Jacobi and Gauss-Seidel methods have been studied in the context of FDM, and the conjugate gradient method, introduced in [Hestenes et al. \[1952\]](#), has been used to solve the positive-definite matrices arising from FEM. To achieve better performance in numerical systems, multi-level methods such as multigrid techniques have been developed, utilizing different levels of representation. Additionally, domain decomposition methods have been proposed to exploit the capabilities of multi-node computers. These ongoing advancements have enabled the solution of increasingly complex and large-scale physical systems with greater precision and stability. These evolutions have been well documented in [Thomée \[2001\]](#) for more information.

Inspired by the evolution of traditional numerical methods, the first part of this PhD applies the successful principles of these methods to the new and promising physics-informed neural networks (PINNs), trying to make them suitable for more complex physical problems, similar to the advancements achieved by their traditional counterparts.

This work result in two contributions. The chapter 5 led to a journal paper in Computational Optimization and Applications in 2024.

Serge Gratton, Valentin Mercier, Elisa Riccietti and Philippe L. Toint. A block-coordinate approach of multi-level optimization with an application to physics-informed neural networks. Computational Optimization and Applications 2024.

The work presented in chapter 6 produce a first preprint in 2021 : Valentin Mercier,

Serge Gratton, Pierre Boudier. A coarse space acceleration of deep-DDM. [The preprint link](#). And was rework in 2023/2024 leading to a proceedings submission :

Victorita Dolean, Serge Gratton, Alexander Heinlein, Valentin Mercier.
Toint. Two-level deep domain decomposition method. DD28.

We have chosen to arrange these chapters in this order because the concept of the F-principle, which is not utilized in the preprint of 6 but is included in the proceeding, is introduced in 5. This sequence provides a more coherent and fluid reading experience.

Chapter 5

A Block-Coordinate Approach of Multi-level Optimization with an Application to Physics-Informed Neural Networks

5.1	Introduction	64
5.2	A block-coordinate perspective on Galerkin multilevel optimization	66
5.3	A generic BCD algorithm and its convergence	70
5.4	Application to Physics-Informed Neural Networks	76
5.4.1	Alternating training in a hierarchical context	76
5.5	A “frequency-aware” distributed approach	81
5.5.1	The F-Principle	81
5.5.2	A frequency-aware network architecture	82
5.6	Numerical results	85
5.6.1	Poisson problems	85
5.6.2	The heat equation	90
5.7	Conclusion and perspectives	91

Disclaimer on Notation**Important Notice:**

Please be aware that a change in notation occurs in this chapter. The notation used in earlier chapters has been modified to better align with the advanced concepts and methodologies discussed herein. For clarity, the new notation is explicitly defined in the relevant sections. However, for simplicity, a brief summary is provided below:

- The neural networks will be denoted by y .
- The parameters of the neural networks will be denoted by x .
- The inputs to the networks will be denoted by z .
- The loss function will be denoted by f .

5.1 Introduction

Many numerical optimization problems of interest today are large dimensional, and techniques to solve them efficiently are thus an active field of research. A very powerful class of algorithms for the solution of large problems is that of multi-level methods. Originally, the concept of a method exploiting multiple levels, i.e., multiple resolutions of an underlying problem, was introduced for the solution of large scale systems arising from the discretization of partial differential equations. In this context these methods are known as multigrid (MG) methods for the linear case or full approximation schemes (FAS) for the nonlinear one [Briggs et al. \[2000\]](#), [Trottenberg et al. \[2000\]](#). These schemes were later extended to nonlinear optimization problems, in which context they are known as multi-level optimization techniques [Nash \[2000\]](#), [Gratton et al. \[2005\]](#), [Gratton \[2006\]](#), [Gratton et al. \[2008\]](#), [Calandra et al. \[2021\]](#). The central idea of all these approaches is to use the structure of the problem in order to significantly reduce the computational cost compared to standard approaches applied to the full unstructured problem.

In this work we introduce a new interpretation of multi-level methods as block coordinate descent (BCD) methods: iterations at coarse levels (i.e., low resolution) can be interpreted as the (possibly approximate) solution of a subproblem involving a set of variables smaller than that required to describe the fine level (high resolution). We propose a framework that allows us to encompass multi-level methods for several classes of problems as well as a unifying complexity analysis based on a generic block coordinate descent, which is simple yet comprehensive.

To illustrate the effectiveness of the proposed approach, we apply our framework in the context of deep learning. The idea of exploiting multiple scales in learning has been explored for different kind of networks. For instance, [Kopaničáková and Krause \[2022\]](#), [Haber et al. \[2018\]](#), [Wu et al. \[2020\]](#), [von Planta et al. \[2021\]](#) propose multilevel

methods for the training of deep residual networks (ResNets), in which the multilevel hierarchy and the transfer operators are constructed by exploiting a dynamical system's interpretation. Multi-scale methods for convolutional neural networks and recurrent networks have also been proposed in [Haber and Ruthotto \[2017\]](#), [Ke et al. \[2017\]](#) and [Chung et al. \[2016\]](#), respectively. Our focus in this work is on physics-informed neural networks. These networks have been introduced in [Raissi et al. \[2019\]](#) and have exhibited good performance in practice, soon supported by theoretical results. See [2.2.4](#) for more details on the topic.

Despite their success, the training of such networks may remain difficult, in particular for highly nonlinear or multi-frequency problems [Wang et al. \[2021b\]](#). In particular, choosing an efficient detailed network architecture and an associated training procedure is far from obvious, especially if the problem's solution involves high frequency components ([Xu et al. \[2019\]](#) has described under the name of *F-principle* why it might be so, see also [Rahaman et al. \[2019\]](#), [Ronen et al. \[2019\]](#), [Wang et al. \[2022\]](#), [Farhani et al. \[2022\]](#)). While specific "frequency aware" network architectures, such as WWP [Wang et al. \[2021b\]](#) structures and Mscale networks [Liu et al. \[2020\]](#), [Li et al. \[2020b\]](#) have been proposed to circumvent this latter difficulty, the mere size of the networks necessary to represent solutions of PDEs with sufficient accuracy still make their computationally efficient training very challenging. Our objective is to make this challenge more tractable.

Contributions. In this context, the specific contributions of this work may be summarized as follows.

1. We present a unifying framework for a large class of multi-level problems (Section [5.2](#)).
2. We then introduce a suitable block-coordinate descent algorithm, and analyze its evaluation complexity bound under standard assumptions (Section [5.3](#)).
3. We next investigate how this approach can be applied to PINNs for the solution of Laplace and heat propagation problems on complex geometries.
 - (a) In a first step, we show that a multi-level technique based on alternate training of "coarse" and "fine" networks may bring substantial computational benefits (Section [5.4](#)).
 - (b) We then exploit frequency aware network architectures in this multi-level context, allowing the efficient solution of more complex problems (Section [5.5](#)).

Compared with standard (single-level) training, both approaches are shown to yield better solutions at a much reduced computational cost.

5.2 A block-coordinate perspective on Galerkin multilevel optimization

Our purpose is to present a new (at least as far as we know) but yet simple perspective on multilevel optimization using Galerkin approximations. Given some space $\mathcal{F}$ of continuous functions from $\mathbb{R}^p$ to $\mathbb{R}^q$ and some objective function from $\mathcal{F}$ into $\mathbb{R}$, our global aim is to compute a function y , which is a (possibly approximate) solution of the variational problem

$$\min_{y \in \mathcal{F}} f(y). \quad (5.1)$$

The objective function f is often given by some norm of the residual of a problem of interest (PDE, ODE, boundary value problem, linear system or other) but other cases are possible (such as minimum surface or contact problems, for instance). We assume that $\mathcal{F}$ consists of functions constructed by linearly or nonlinearly combining elemental/basis functions using a parametrization involving the parameters $x \in \mathbb{R}^n$, so that y is denoted by $y(x)$, whereas the value of $y(x)$ at z will be denoted by $y(x)(z)$. The problem then reduces to finding the value(s) of x such that $f(y(x))$ is minimized. In this work, we focus on a class of “splitting” techniques whose objective is to reduce the computational cost associated with this minimization. More specifically, we consider the case where y is viewed as the sum of two terms y_1 and y_2 , themselves depending on their own sets of parameters $x_1 \in \mathbb{R}^{n_1}$ and $x_2 \in \mathbb{R}^{n_2}$, that is

$$y(x) = y_1(x_1) + y_2(x_2). \quad (5.2)$$

This yields the optimization problem

$$\min_{(x_1, x_2) \in \mathbb{R}^n} f(y_1(x_1) + y_2(x_2)),$$

where $n = n_1 + n_2$. We also associate with the splitting (5.2) the following “approximation sets”

$$\mathcal{A}_{12} = \{y \in \mathcal{F} \mid y(x) = y_1(x_1) + y_2(x_2) \text{ for some } (x_1, x_2) \in \mathbb{R}^n\} \quad (5.3)$$

as well as

$$\mathcal{A}_i = \{y \in \mathcal{F} \mid y(x) = y_i(x_i) \text{ for some } x_i \in \mathbb{R}^{n_i}\} \quad (i = 1, 2). \quad (5.4)$$

While our present development is based on an additive structure of the function y , other formulations could obviously be of interest. We limited the number of terms to two in order to simplify exposition, but this not restrictive (as we discuss below).

We now discuss some examples of this approach, in which we distinguish two main contexts.

The hierarchical context: The terms y_1 and y_2 are constructed such that

$$\mathcal{A}_2 \subset \mathcal{A}_1 = \mathcal{A}_{12}. \quad (5.5)$$

This may occur in variety of cases, the simplest being the classical multigrid framework in which one assumes that f is a strictly convex quadratic and the y_i are linear. The quadratic's minimization is then equivalent to the solution of a positive definite linear system. The y_i are constructed as linear combination of basis functions $\{b_j\}_{j=1}^m$ of $\mathcal{F}$ (typically from a finite-differences or finite-elements basis), that is

$$y_1(x_1) = \sum_{j=1}^m (x_1)_j b_j \quad \text{and} \quad y_2(x_2) = \sum_{j=1}^m (Px_2)_j b_j, \quad (5.6)$$

where $m = n_1$ is the dimension of $\mathcal{F}$ and where P is a $(n_1 \times n_2)$ linear “prolongation” operator from a “coarse” space of dimension $n_2 \leq n_1$ to the “fine” space of dimension n_1 . In the multigrid framework, these coarse and fine spaces often correspond to coarse and fine discretizations of an underlying continuous problem, but other interpretations such as domain decomposition, are possible. The quadratic optimization problem then becomes

$$\min_{(x_1, x_2) \in \mathbb{R}^{n_1+n_2}} \frac{1}{2} (x_1 + Px_2)^T A (x_1 + Px_2) + b^T (x_1 + Px_2) \quad (5.7)$$

for some positive definite matrix A and right-hand side b depending of the basis $\mathcal{B} = \{b_j\}_{j=1}^m$. One easily verifies that “restricting” the minimization to the x_2 variables amounts to solving the n_2 -dimensional problem

$$\min_{x_2 \in \mathbb{R}^{n_2}} \frac{1}{2} x_2^T P A P^T x_2 + (P^T b + P^T A x_1)^T x_2, \quad (5.8)$$

which is the usual Galerkin approximation of f at the coarse level. We clearly have that

$$\mathcal{A}_2 = \left\{ \sum_{j=1}^m (Px_2)_j b_j \right\} \subset \left\{ \sum_{j=1}^m (x_{1,j} + (Px_2)_j) b_j \right\} = \text{span}(\mathcal{B}) = \mathcal{A}_1 = \mathcal{A}_{12}, \quad (5.9)$$

ensuring (5.5). Classical multigrid methods then alternate approximate resolutions of the n_2 -dimensional “coarse level” problem (5.8) and the n -dimensional “fine level” one given by (5.7).

A second, more nonlinear, case is when f may no longer be a convex quadratic, but a smooth nonlinear function (which we assume is bounded below for consistency), while keeping (5.6) and its interpretation in terms of “coarse” and “fine” spaces. Reusing (5.6) and (5.9) we may now consider

$$\widehat{f}(x_1, x_2) \stackrel{\text{def}}{=} f \left(\sum_{j=1}^m x_{1,j} b_j + \sum_{j=1}^m (Px_2)_j b_j \right) = f \left(\sum_{j=1}^m (x_{1,j} + (Px_2)_j) b_j \right) \stackrel{\text{def}}{=} F(x_1 + Px_2), \quad (5.10)$$

which is a reformulation of the original objective function incorporating the dependence of the basis $\mathcal{B}$ and, as above, alternatively perform iterations on the problems

$$\min_{(x_1, x_2) \in \mathbb{R}^{n_1+n_2}} \widehat{f}(x_1, x_2) \quad \text{and} \quad \min_{\substack{x_2 \in \mathbb{R}^{n_2} \\ x_1 \text{ fixed}}} \widehat{f}(x_1, x_2)$$

and note that, because of (5.9), the first of these problems is equivalent (in the sense that they yield the same value for $y_1(x_1) + y_2(x_2)$) to the lower-dimensional

$$\min_{\substack{x_1 \in \mathbb{R}^{n_1} \\ x_2 \text{ fixed}}} \widehat{f}(x_1, x_2).$$

This second approach has been explored in the framework of nonlinear optimization by the MG-OPT Nash [2000] and RMTR Gratton et al. [2008] methods. Both these algorithms consider a subproblem where a coarse-level model h of the objective function is approximately minimized to find an increment δ to update the fine iterate. Our hierarchical approach corresponds to the choice of the Galerkin Taylor models, defined by

$$h_1(\delta x_2) := \sigma(P^T \nabla_x F(x_1 + Px_2))^T \delta x_2$$

for first order and

$$h_2(\delta x_2) := \sigma(P^T \nabla_x F(x_1 + Px_2))^T \delta x_2 + \frac{1}{2} \sigma^2 \delta x_2^T P \nabla_x^2 F(x_1 + Px_2) P^T \delta x_2,$$

for second order, where σ is a fixed positive constant and δx_2 is an increment in x_2 from the point (x_1, x_2) . Note that the form of h_2 is identical to that of (5.8).

The framework just described is also closely related to the FAS approach [Briggs et al., 2000, p. 98] for solving a set of nonlinear equations. If we consider the equation $\nabla F(x) = 0$, the standard FAS approach would consist in solving at a given fine iterate x_1 the problem $P^T \nabla F(P^T x_1 + x_2) P = \text{rhs}$, where the right-hand side rhs is such that if x_1 solves the problem (i.e., annihilates the gradient of F), x_2 is zero. This approach is different from ours in that the correction Px_2 in the coarse problem is added to $PP^T x_1$ in the coarse problem definition, and not to x_1 itself. Taking this into account, we may consider the coarse equation in x_2 , $P^T \nabla F(x_1 + Px_2) = 0$, for which we obtain a formulation that is now in line with our hierarchical context since the derivative of this coarse equation are identical to those involved in the above definition of h_2 . Note that the right-hand side of this equation is now zero since $x_2 = 0$ solves the coarse problem when $\nabla F(x_1) = 0$.

In what follows, we will be especially interested in the case where

$$y_i(x_i) = \text{NN}_i(x_i) \tag{5.11}$$

where $\text{NN}_i(x_i)$ is a neural network of input z , parameters (weights and biases) given by $x_i \in \mathbb{R}^{n_i}$, and output $\text{NN}_i(x_i)(z)$. Neural networks are clearly nonlinear and nonconvex functions of the parameters x_i and the hierarchical context occurs when y_2 is a subnetwork of y_1 .

The distributed context: We may now abandon (5.5) and consider a situation where neither $\mathcal{A}_1$ or $\mathcal{A}_2$ is identical to $\mathcal{A}_{12}$. This is for instance the case when (5.11) holds, but y_2 is not a subnetwork of y_1 and n_1 and n_2 (the number of network parameters in y_1 and y_2 , respectively) are now independent of m . Our proposal is to use a similar methodology for this more complex case and alternate approximate minimizations on the “ $\mathcal{A}_2$ subproblem” given by

$$\begin{aligned} \min_{\substack{x_2 \in \mathbb{R}^{n_2} \\ x_1 \text{ fixed}}} f(y_1(x_1) + y_2(x_2)) \end{aligned} \quad (5.12)$$

with that on the “ $\mathcal{A}_1$ subproblem” given by

$$\begin{aligned} \min_{\substack{x_1 \in \mathbb{R}^{n_1} \\ x_2 \text{ fixed}}} f(y_1(x_1) + y_2(x_2)). \end{aligned} \quad (5.13)$$

In all cases described above, the computational cost of the overall minimization is expected to decrease because we may choose n_2 (and, for the distributed case, n_1) to be significantly smaller than n . Alternating standard minimization steps on the fine $\mathcal{A}_1$ level with cheap ones at the coarse $\mathcal{A}_2$ level is therefore computationally attractive, *provided the coarse steps significantly contribute to the overall minimization*.

We conclude this section by noting that it is obviously possible to consider more general additive splittings of the form

$$y(x, z) = \sum_{i=1}^s y_i(x_i, z)$$

in our above developments. In the hierarchical context, that is in the classical multi-grid approach and in the RMTR/MG-OPT algorithms, this is achieved by considering a hierarchy of nested approximations sets

$$\mathcal{A}_{\{\ell, \dots, s\}} = \left\{ y \mid y = \sum_{j=1}^m (x_{\ell, j} + P_1(x_{\ell+1, j} + P_2(\dots P_k(x_{s-1, j} + P_s x_{s, j}) \dots))) b_j \right\}$$

for $\ell \in \{1, \dots, s\}$, so that $\mathcal{A}_{\{\ell+1, \dots, s\}} \subset \mathcal{A}_{\{\ell, \dots, s\}}$ for each $\ell \in \{1, \dots, s-1\}$.

In the distributed context, one could consider the sets

$$\mathcal{A}_{\mathcal{S}} = \left\{ y \mid y = \sum_{i \in \mathcal{S}} y_i(x_i) \right\} \quad (5.14)$$

for all nonempty subsets $\mathcal{S}$ of $\{1, \dots, s\}$. This allows for a wide variety of set architectures such as the “recursive” one using $\mathcal{A}_{\{\ell, \dots, s\}}$ for $\ell \in \{1, \dots, s\}$, the “flat” one using $\mathcal{A}_{\{\ell\}}$ for $\ell \in \{1, \dots, s\}$, or any mixture of these. In full generality, the approximation sets (5.14)

need not be disjoint and it is then useful, for a computationally effective architecture, to identify which subsets $\mathcal{S}$ generate identical $\mathcal{A}_{\mathcal{S}}$ and to ignore those for which $\sum_{i \in \mathcal{S}} n_i$ (the dimension of the associated minimization problem) is not minimal.

As a consequence of the above discussion, we see that a wide variety of multilevel optimization methods may be viewed as block-coordinate minimization problems, where the blocks of variables are given by the x_i .

5.3 A generic BCD algorithm and its convergence

We now examine why splitting minimization into alternate block minimization can be useful, and consider the associated convergence guarantees. We first note that such guarantees are already available, for linear multigrid case Briggs et al. [2000], Hackbusch [1995], Trottenberg et al. [2000] for FAS-like methods Borzi and Kunisch [2006], as well as for MG-OPT Nash [2000] and RMTR Gratton et al. [2008], Gratton [2006], Groß and Krause [2021], Calandra et al. [2021]. The objective of this section is to motivate and state a simple yet comprehensive complexity analysis, covering the two contexts described above.

The success of existing multilevel methods is based on exploiting a “complementarity” between the various minimization problems involved, which we pursue as follows. Given the overall problem (5.1), one starts by considering a particular minimization method and isolate a class of problems for which this method is efficient. In the hierarchical context, this is typically the Gauss-Seidel or Jacobi method and the class of problems where such “smoothing methods” are efficient is that of problems involving high-frequency behaviour in the sought y function of the underlying variable z (see [Briggs et al., 2000, Chapter 2]). This suggests that one might wish to split the problem (if at all possible) depending of its frequency content. To achieve this, one chooses (at least implicitly) the basis $\mathcal{B}$ (which spans $\mathcal{A}_{12}$ and $\mathcal{A}_1$) to be a Fourier basis of $\mathcal{F}$ and split the problem into finding the coefficients of the high-frequency basis functions (using a smoothing method in $\mathcal{A}_{12} = \mathcal{A}_1$) and finding those of the low frequency ones ($\mathcal{A}_2$). The key of the approach is to transform the low-frequency subproblem into a high frequency one by shifting frequencies, making the smoothing algorithm efficient also for this subproblem. This shift is usually achieved by considering a coarser discretization of the underlying continuous problem (the “coarse space”). Classical multigrid methods (and also nonlinear methods in the hierarchical context) then alternate minimizations steps for the high-frequency (“fine”) subproblem with minimizations in the low-frequency (“coarse”) one, $\mathcal{A}_2$. Note that, since the frequency shift may be obtained only by considering the underlying geometrical space, an explicit expression in the Fourier basis is unnecessary. Access to the high frequency basis elements may be unavailable as such, but is included in the contribution of the complete basis spanning $\mathcal{A}_{12}$ and $\mathcal{A}_1$.

We propose to follow the same approach for the distributed context. We then select a particular minimization method. In our focus example where $y(x)$ is a neural net, first-order training methods such as variants of gradient descent are a natural choice. Remarkably, it has been shown in Xu et al. [2019] that such methods are significantly

more efficient for the solution of problems whose solution involves *low* frequencies. This observation, called the “F-principle”, is interesting on two accounts. The first is that it stresses the fact that frequency content is also significant for neural net training, and the second is that it acts “in the opposite direction” when compared to a multigrid approach: low frequencies are favourable instead of being problematic. One is then led to consider using a Fourier basis also in the new context, split the problem into a part containing the high-frequency basis (finding a suitable network $y_1(x_1) \in \mathcal{A}_1$) and its low frequency part (finding a network $y_2(x_1) \in \mathcal{A}_2$). and then to shift the frequencies of the subproblems to make them more efficiently solvable. As we will discuss when presenting our numerical examples in Section 5.5, this can be achieved by using Mscale networks Liu et al. [2020] and WWP Wang et al. [2021b]. As above, this (fortunately) does not require the explicit problem formulation in the Fourier basis, although one needs to be somewhat specific regarding the subproblems’ frequency content.

We also note that, if the objective function f were separable in x_1 and x_2 in the selected basis $\mathcal{B}$ (the Fourier basis, in our examples), then only one of each subproblem minimization would be sufficient for solving the overall problem. This is not the case in general, but an argument based on quadratic approximation shows that a weak coupling within f between x_1 and x_2 improves the speed of convergence for the block-coordinate minimization.

At each stage of the minimization of f , we may therefore compute one or more step(s) for a subproblem defined by selecting a subset of variables or, equivalently, a set of y_i , to (approximately) minimize, in a typical block-coordinate descent (BCD) approach.

Pure cycling between the relevant subproblems is clearly an option, and is the strategy most often used in the multigrid case, where V or W cycles are defined to organise the cycling. Alternatively, we may opt for some sort of randomized cycling (see Richtárik and Takáč [2014], Nesterov [2012] for the convex case) or follow (as we choose to do below) the procedure used in the RMTR algorithm for the hierarchical case and select a subset of variables for which the expected (first-order) decrease in the objective function (as measured by the norm of the objective’s gradient with respect to the variables in the subset) is sufficiently large.

We may therefore consider a simple block-coordinate descent algorithm for minimizing f , where we use a second subscript for x to denote iterations numbers, and where we have limited the exposition to the bi-level/blocks case.

In the form stated above, the ML-BCD algorithm requires computing the full gradient at every “major” iteration (i.e., iterations where Step 2 is used), at variance with a pure (potentially randomized) cycling where only the successive subproblem’s gradients need to be computed. Thus the number of major iterations must remain small compared to the total number of iterations for this approach to be useful.

Also note that the subproblem termination criterion may take different forms: the number of subproblem iterations may be limited, a threshold may be imposed on the norm of the subproblem’s gradient and/or on the objective function decrease, below which the subproblem minimization is terminated, or any combination of these. In any

Algorithm 5.3.1: Multilevel Optimization (ML-BCD) - Deterministic version

Step 0: Initialization: An initial point $(x_{1,0}, x_{2,0}) \in \mathbb{R}^n$, a threshold $\tau \in (0, \frac{1}{\sqrt{2}})$ and a gradient accuracy threshold $\epsilon \in (0, 1]$ are given. Set $k = 0$.

Step 1: Termination test. Evaluate the gradients $\nabla_x^1 f(x_k) = (\nabla_{x_1}^1 f(x_k), \nabla_{x_2}^1 f(x_k))^T$. Terminate if $\|\nabla_x^1 f(x_k)\| \leq \epsilon$.

Step 2: Select a subproblem and a subproblem termination rule.

For instance,

select i such that $\|\nabla_{x_i}^1 f(x_k)\| > \tau \|\nabla_x^1 f(x_k)\|$ and choose to minimize $f(x_1, x_2)$ as a function of x_i .

Also select a termination rule for the chosen subproblem.

Step 3: Approximately solve the chosen subproblem. Apply a first-order minimization method to the chosen subproblem, starting from $(x_{1,k}, x_{2,k})$ and iterate for ℓ_k iterations until the selected termination rule is activated. This yields a new iterate $(x_{1,k+\ell_k}, x_{2,k+\ell_k})$.

Step 4: Loop. Increment k by ℓ_k and go to Step 1.

case, it does not make sense to continue the subproblem minimization if the subproblem's gradient becomes smaller than $\frac{\epsilon}{\sqrt{2}}$ ($\frac{\epsilon}{\sqrt{\# \text{ of blocks}}}$ in the general case).

The convergence theory for block-coordinate optimization has a long history, starting with a famous paper by Powell [Powell \[1973\]](#) showing that the method may fail on nonconvex continuously differentiable functions. While the theory was further developed for the convex case (see the excellent survey [Wright \[2015\]](#) and the references therein), it was only recently that further progress was made for nonconvex functions, overcoming Powell's reservations, and that a worst-case complexity analysis (implying convergence) was produced [Amaral et al. \[2022\]](#). The idea is quite simple and rests on the notion of "sufficient descent", which requires, when using first-order methods, that

$$f(x_k) - f(x_{k+1}) \geq \kappa \|\nabla_x^1 f(x_k)\|^2,$$

for all $k \geq 0$, where κ is a positive constant only depending on f . This sufficient descent is, in particular, guaranteed by the Lipschitz continuity of $\nabla_x^1 f$ along the path of iterates $\cup_{k \geq 0} [x_k, x_{k+1}]$ (see [\[Cartis et al., 2022, Notes for Section 2.4\]](#)), in which case κ is proportional to the inverse of the gradient's Lipschitz constant¹. For the sake of completeness, we give a simple proof in [Appendix 8.3.1](#) for a version of the ML-BCD algorithm using fixed-stepsize gradient descent in Step 3, i.e., when, for all $k \geq 0$,

$$x_{k+1} = x_k - \alpha \overline{\nabla_{x_i}^1 f(x_k)} \quad (5.15)$$

where $\overline{\nabla_{x_i}^1 f(x_k)} = (\nabla_{x_1}^1 f(x_k)^T, 0)^T$ if $i = 1$ and $(0, \nabla_{x_2}^1 f(x_k)^T)^T$ if $i = 2$. This proof rephrases a standard "single block" result (see for instance [\[Nesterov, 2013, Example 1.2.3\]](#)) for the BCD case. The key complexity result can be stated as follows.

Theorem 5.3.1. Suppose that f is continuously differentiable with Lipschitz continuous gradient on the path of iterates $\cup_{k \geq 0} [x_k, x_{k+1}]$ generated according to (5.15) by the ML-BCD algorithm with fixed stepsize, and that it is bounded below by f_{low} . Suppose also that the stepsize α is small enough to ensure $\alpha < 1/L$, where L is the gradient's Lipschitz constant and that the i -th subproblem is terminated at the latest as soon as $\|\nabla_{x_i}^1 f(x_k)\| \leq \epsilon/\sqrt{2}$. Then the ML-BCD algorithm requires at most $\kappa_* \epsilon^{-2}$ iterations to produce an iterate x_k such that $\|\nabla_x^1 f(x_k)\| \leq \epsilon$, where κ_* is a positive constant only depending of α , the initial gap $f(x_0) - f_{\text{low}}$ and the number of blocks.

Other versions of the algorithm including more elaborate globalization techniques such as line searches, trust-regions or adaptive regularization are also possible and yield

¹This assumption is standard in complexity analysis and prevents arbitrary change in the gradient over short steps. In particular, it prevents the occurrence of infinite gradients at iterates, which would cause the algorithm to fail. It is more general than assuming bounded gradients, as can be seen by considering convex quadratics. See also the discussion at the end of the present Section.

5.3. A generic BCD algorithm and its convergence

the same $\kappa_* \epsilon^{-2}$ complexity bound for different values of κ_* (we then say that their complexity is $\mathcal{O}(\epsilon^{-2})$).

We now consider a complexity result for a stochastic variant of ML-BCD, in which a fixed-stepsize first-order stochastic gradient (SG) method is used in Step 3, i.e., when, for all $k \geq 0$,

$$x_{k+1} = x_k - \alpha \overline{g_i(x_k, \xi_k)} \quad (5.16)$$

where $g_i(x_k, \xi_k)$ is a stochastic estimate of $\nabla_{x_i}^1 f(x_k)$ and $\overline{g_i(x_k, \xi_k)} = (g_i(x_k, \xi_k)^T, 0)^T$ if $i = 1$ and $(0, g_i(x_k, \xi_k)^T)^T$ if $i = 2$. The proof is given in Appendix 8.3.2. The result can easily be extended to the case of diminishing step-sizes.

Theorem 5.3.2. Suppose that f is continuously differentiable with L -Lipschitz continuous gradient on the path of iterates $\cup_{k \geq 0} [x_k, x_{k+1}]$ generated according to (5.16) by the ML-BCD algorithm, and that it is bounded below by f_{low} . Assume that, for some $\mu > 0$, $\sigma_1 \geq 0$, $\sigma_2 > 0$ and all $k \in \mathbb{N}$, the stochastic gradient estimate $g_i(x_k, \xi_k)$ satisfies

$$\nabla_{x_i}^1 f(x_k)^T \mathcal{E}_{\xi_k}(g_i(x_k, \xi_k)) \geq \mu \|\nabla_{x_i}^1 f(x_k)\|^2 \quad (5.17a)$$

$$\mathcal{E}_{\xi_k}(\|g_i(x_k, \xi_k)\|^2) \leq \sigma_1^2 + \sigma_2^2 \|\nabla_{x_i}^1 f(x_k)\|^2, \quad (5.17b)$$

and that the fixed stepsize is such that $\alpha \leq \mu/L\sigma_2^2$. Suppose also that the i -th subproblem is chosen according to the rule $\|\nabla_{x_i}^1 f(x_k)\| > \tau \|\nabla_x^1 f(x_k)\|$ for some $\tau \in (0, 1/\sqrt{2})$ and terminated as soon as $\|g_i(x_k, \xi_k)\| \leq c_\epsilon$, with $c_\epsilon > \sqrt{2}\sigma_1$ or after a maximum number of iterations $\bar{\ell}$. Then, for all $K \in \mathbb{N}$,

$$\mathcal{E} \left(\sum_{k=1, k \in \mathcal{M}}^K \frac{\|\nabla_x^1 f(x_k)\|^2}{K} \right) \leq \frac{2(f(x_1) - f_{\min})}{\alpha \mu K \tau} + \frac{\alpha L \sigma_1^2}{\mu \tau} + \frac{\bar{\ell} \sigma_1^2}{\tau} \left(\frac{\alpha L}{\mu} - \frac{1}{\sigma_2^2} \right).$$

where $\mathcal{M}$ is the set of major iterations for which Step 2 of ML-BCD is performed.

The assumptions in Theorem 5.3.2 are standard (see Bottou et al. [2018], for instance), except for the condition $c_\epsilon > \sqrt{2}\sigma_1$. This condition is however reasonable, because it does not make sense to optimize (for a block) beyond the noise level. Note that the last term in the theorem's bound is negative because of the assumption on α . Large values of $\bar{\ell}$ will thus improve the bound, but this improvement is limited because the subproblem may also be terminated because of the ϵ -dependent gradient test.

We note that BCD methods have been considered and analysed in a stochastic setting in Bellavia et al. [2023], in which a convergence result is stated, based on a bound on the average-squared gradients. This result assumes a distributed context with non overlapping blocks, which is slightly less general than the corresponding result for SG method proposed in [Bottou et al., 2018, Th. 4.8]. Interestingly, the condition we

use to select the blocks, allows us to provide a bound that also covers the hierarchical setting. The bound is not directly comparable to that for the standard SG, because it is established considering just the subsequence of the major iterations.

The situation is more involved and the complexity bound worse (when it exists) as soon as the function is not Lipschitz continuous on the path of iterates, and is for instance discussed (for the single block case) in [Jordan et al. \[2022\]](#), [Tian and So \[2021\]](#), [Zhang et al. \[2020\]](#). The recent paper [Kong and Lewis \[2023\]](#) analyzes the complexity of a “first-order” monotonic descent method applied to a wide class of functions². The method assumes that one can evaluate, for any point x and any direction d , the value of $f(x)$, its directional derivative along d $f'(x, d)$ and a “directional subgradient” $G(x, d)$ such that its inner product with d returns $f'(x, d)$. Under these conditions, the method achieves (Goldstein [Goldstein \[1977\]](#)) ϵ -approximate optimality in at most $\mathcal{O}(\epsilon^{-4})$ such evaluations. Unsurprisingly, the convergence proof once more relies on showing that it is possible to obtain “sufficient descent”, in this case given by a multiple of ϵ , albeit at a possible cost of $\mathcal{O}(\epsilon^{-3})$ evaluations. We refer the reader to [Kong and Lewis \[2023\]](#) for details. The monotonic nature of the algorithm then implies (as is the case for the simple proof in appendix) that sufficient descent obtained in the solution of the subproblem before termination translates into sufficient descent on the complete problem, so that the complexity result obtained by [Kong and Lewis \[2023\]](#) for single block minimization extends to the case where there is a (bounded) number of blocks.

Interestingly, Theorem 5.3.1 subsumes and simplifies the convergence theory for RMTR [Briggs et al. \[2000\]](#), [Gratton et al. \[2008\]](#) by recasting this latter method (when used with first-order Galerkin low-level approximations) as a trust-region BCD algorithm in the complete space. Also note that the use of Galerkin approximations in this case avoids the need of a “tau correction” or “first-order coherency” condition [Briggs et al. \[2000\]](#), [Gratton et al. \[2008\]](#) which is typically requested for less structured low-level approximations.

5.4 Application to Physics-Informed Neural Networks

Techniques using neural networks have recently emerged as an alternative to classical methods for solving partial differential equations. In particular, the physics-informed neural networks have raised significant interest [Raissi et al. \[2019\]](#). Their advantage is their ability to exploit physical knowledge to solve PDEs without using simulation data, which motivates our choice to illustrate our multilevel approach in this context. Our presentation proceeds in two stages: after the brief introduction to PINNs given in 2.2.4, we introduce their multilevel version, we first focus on showing the advantage of alternate training of “coarse” and “fine” networks in the hierarchical context of Section 5.2, before showing how a refined use of the frequency content (in a distributed context) may yield further benefits.

²Technically, the objective function must be bounded below, have a bounded directional subgradient map and a finite nonconvexity modulus.

5.4.1 Alternating training in a hierarchical context

Our objective in what follows is to use several PINNs in conjunction with the ML-BCD algorithm, broadly mimicking multigrid methods, in the hope of obtaining similar computational advantages. We define multi-level PINNs (MPINNs) as follows.

Consider a feedforward neural networks with $N - 1$ hidden layers. Let $d_j \in \mathbb{N}$ be the number of hidden neurons in the j -th hidden layer for $j = 1, \dots, N - 1$ and let d_0 and d_N be the number of neurons of the input and the output layers, respectively. Let $W^j \in \mathbb{R}^{d_j \times d_{j-1}}$ be the matrix of weights between the $(j - 1)$ -th and the j -th layers for $j = 1, \dots, N$. We denote the set of all such networks by $H^{N, \{d_j\}}$. Assuming a method is selected to minimize the loss functions (2.5), we may now use the ML-BCD algorithm by selecting our neural network as

$$y(x)(z) = \sum_{i=1}^s y_i(x_i)(z) \quad (5.18)$$

with $y_i \in H^{N_i, \{d_{j,i}\}}$.

We start by considering the question of whether imitating the multigrid approach of alternating between a coarse and a fine grid can be computationally advantageous.

5.4.1.1 Test problems

Inspired by [Rapaka and Samtaney \[2020\]](#), we perform our experiments on several instances of the Poisson problem with source term r in the domain Ω and Dirichlet/Neumann conditions on its boundary Γ . Moreover, we assume that Ω contains a closed embedded boundary Γ_i dividing Ω in Ω_e (the exterior of Ω_i) and Ω_i (the interior of Ω_i), so that the complete boundary is given by $\Gamma = \Gamma_e \cup \Gamma_i$. The problem is thus stated as

$$\begin{cases} \Delta u(z) = r(z) \text{ in } \Omega \\ au(z) + b \frac{\partial u}{\partial \zeta}(z) = g(z) \text{ on } \Gamma_e \\ cu(z) + d \frac{\partial u}{\partial \zeta}(z) = h(z) \text{ on } \Gamma_i \end{cases} \quad (5.19)$$

where $\frac{\partial u}{\partial \zeta} = \nabla u^T \zeta$ with ζ being the boundary normal vector pointing into Ω . The domain for the test problem is the square $\Omega = [-1, 1]^2$ with an embedded circle of radius $R = 0.5$ centered at the origin defining Ω_i . We consider Dirichlet boundary condition on Γ_e and Γ_i . We choose r to ensure that exact solution is

$$u(z) = \cos(\alpha\pi z_1 + \pi z_2) + \cos(\pi z_1 + \beta\pi z_2)$$

where α and β are integers defining the frequency content of the solution.

5.4.1.2 Networks architectures

In this first set of experiments, we simplify (5.18) to

$$y(x)(z) = y_1(x)(z) + y_2(x)(z)$$

where $y_1 \in H^{3,\{d_{j,1}\}}$ is the "fine" network and $y_2 \in H^{3,\{d_{j,2}\}}$ is the "coarse" one. In accordance with our earlier discussion, we choose y_2 smaller than y_1 in the sense that y_2 is an (independent) copy of a subnetwork of y_1 . Thus our setting is hierarchical (in the sense of Section 5.2) and $\mathcal{A}_2 \subset \mathcal{A}_1 = \mathcal{A}_{12}$.

We consider three different MPINNs corresponding to different sizes of the coarse network, while keeping the size of the fine network fixed. We compare them to a standard PINN network of size approximately equal to the total number of parameters in the fine and coarse networks, as well as a network of the same size as the fine network.

Experiment's name	Network(s)' size (d_1, d_2, d_3)		Number of parameters
	Fine	Coarse	
ML1	(140,140,140)	(140,140,140)	40,040 + 40,040
ML2	(140,140,140)	(100,100,100)	40,040 + 20,600
ML3	(140,140,140)	(70,70,70)	40,040 + 10,220
SL1	(140,140,140)		40,040
SL2	(200,200,200)		81,200

Table 5.1: Network architecture for the experiments with alternating training in the hierarchical context

Table 5.1 details the five architectures tested in our experiments and the size of the involved networks: ML1, ML2 and ML3 are the multilevel ones (training two networks using the ML-BCD algorithm) and SL1 and SL2 are single level ones (standard training of a single network) provided for comparison. All hidden layers use the tanh activation function, thereby ensuring the necessary differentiability properties.

5.4.1.3 Training setup

Training points are uniformly sampled using the Latin hypercube sampling in Ω and $\partial\Omega$. Here we have chosen to use the same grid to train the coarse and the fine networks with $N_\Omega = 50000$ points sampled in the domain and $N_\Gamma = 4000$ points sampled on the boundary. These values have been chosen by performing a grid search on the optimization of the network with classical training.

At each epoch we use a random subset of these points composed of 2000 inner points and 500 boundary points. We have chosen the coefficients $\lambda_\Omega = \lambda_\Gamma = 1$ to weight internal and boundary losses.

To evaluate the accuracy of the different models, we consider a set of testing points $\{z^t\}_{t=1}^T$ with $T = 30000$, randomly chosen using the Latin hypercube sampling in Ω_e ,

and consider the mean squared error given by

$$MSE = \frac{1}{T} \sum_{t=1}^T (y(x)(z^t) - u(z^t))^2$$

All networks are trained with Adam optimizer [Kingma \[2014a\]](#). The initial learning rate is set at 2×10^{-4} with an exponential decay of 0.99999 at each epoch for all networks. All our codes are implemented in TensorFlow2 (version 2.2.0) and run with a single NVIDIA GeForce GTX 1080 Ti.

It is important to notice that using an alternate training strategy requires special care when setting the hyperparameters of the optimizer. Indeed, each time we select a sub-network to train, the optimization problem also changes. To ensure a fair and coherent comparison with standard network training procedures, we decided to maintain the current learning rate when restarting the solver. This allowed us to maintain the property that the learning rate gradually decreases over time, which is a common technique used to prevent the model from overshooting the optimal values and enhances convergence. For other parameters, we decide to use Adam as a black box and to restart it each time we change subproblem, that is we do not transfer the specific hyperparameters that are specific to this optimization method, such as momentum. This means that we have to initialize the optimizer from scratch with default hyperparameters (except for the learning rate) for each subproblem. The consequence of such a restart is a peak in the loss at the first iteration of the optimization process, which is a common behavior when initializing an optimizer with random or default values. However it remains an open question to find the best strategy to tune these hyperparameters: is it better to restart the optimizer or to find a good way to transfer the parameters from one subproblem to the other? We also estimate the number of floating-point operations performed as the number of FLOPs required for the matrix-vector product operations during the forward pass. For MPINNs, it takes into account both the operations performed at fine and at coarse levels. For instance, for a network with two layers, d_h neurons in the layers, an output size of 1, and N training points of size d , the number of FLOPs for the forward pass would be computed as:

$$\text{FLOPs} = N \times (2d \times d_h + d_h^2 + d_h \times 1)$$

We select the subnetwork to train at each cycle according to Step 2 of the ML-BCD algorithm. Moreover, we chose to terminate the subproblem training after a fixed number of epochs on each problem (see numerical results). We also chose $\alpha = 2$ and $\beta = 4$. For each experiment, we alternately performed 2000 epochs on y_1 and 2000 epochs on y_2 .

5.4.1.4 Results

The results of the test are reported in Figures 5.1 and 5.2. We see that, at equivalent computational cost, MPINNs (MLs) converge significantly faster than conventional PINNs (SLs). It is worth noting that the PINN with fewer parameters (SL1) converges faster

5.4. Application to Physics-Informed Neural Networks

than the larger one (SL2). The choice of the coarse network's dimension also seems to affect the speed of convergence, smaller coarse networks yielding better results in these tests.

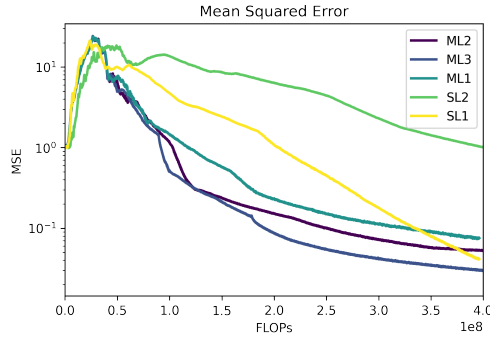


Figure 5.1: Evolution of the MSE as a function of the computational cost for our different models

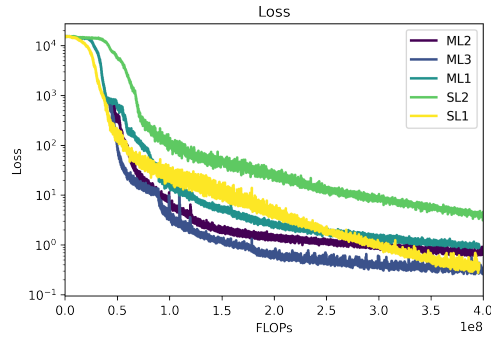


Figure 5.2: Evolution of the loss as a function of the computational cost for our different models

These result may seem encouraging, but this approach remains limited in that it does not overcome a limitation inherent to classical neural network training: high-frequency fitting. This is highlighted in Figure 5.4 where we also test our method on problem (6.2.2) with parameters $\alpha = 2$ and $\beta = 20$.

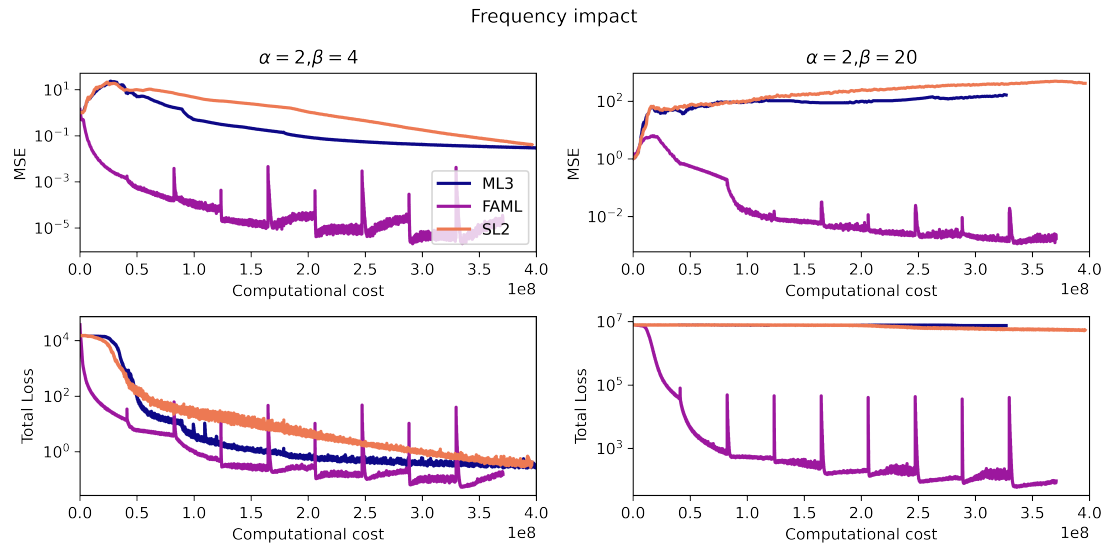


Figure 5.3: Results of our method on the Poisson problem (6.2.2) for different frequencies. The method indicated as FAML (Frequency-Aware-Multilevel) is the method proposed in the forthcoming Section 5.5.

An alternative approach is thus warranted, which we develop in the next section.

5.5 A “frequency-aware” distributed approach

In the hierarchical cases detailed in Section 5.2, the reduction in computational cost is typically directly proportional to the separation of frequencies. For instance, in multigrid methods, switching to a coarse grid helps to target low frequencies that converge slowly in the fine problem. This does not seem to be the case for neural networks. The F-Principle [Xu et al. \[2019\]](#) states that “deep neural networks often fit target functions from low to high frequencies during the training process”, which is problematic in some cases and seems to affect also MPINNs, leading to a slow convergence, as illustrated at the end of the previous section. The simple alternation of coarse and fine problems doesn’t seem to be sufficient in the context of neural networks.

Several papers have addressed the issue related to the F-Principle by proposing architectures that transform high frequencies of the problem into low frequencies, thereby allowing a more efficient use of the neural networks. Inspired by these papers, and after a brief presentation of the F-principle, we propose a frequency-aware architecture that retains the computational cost savings of multilevel training while incorporating the frequency separation aspect of classical methods.

5.5.1 The F-Principle

The spectral-bias ([Rahaman et al. \[2019\]](#)) or frequency principle (F-Principle, [Xu et al. \[2019\]](#)) provide insights on the learning and generalization processes of deep neural networks. As said earlier the F-principle state that “deep neural networks often fit target functions from low to high frequencies during the training process”. The observation made by [Rahaman et al. \[2019\]](#) are based on numerical experiments that reveal the nature of the frequency captured by DNNs during training. The frequency in question is a response frequency, which reflects how variations in the input influence the output of the network. This is a crucial distinction from the conventional notion of input frequency commonly discussed in signal processing. For instance, in tasks like MNIST image classification, the response frequency refers to the variability of the classification function relative to changes in pixel intensity, rather than the spatial frequency of the image itself. To demonstrate this, [Xu et al. \[2019\]](#) trained a DNN to fit a synthetic function $f(x) = \sin(x) + \sin(3x) + \sin(5x)$, which contains three distinct frequency components. At each epoch, they calculated the distance between the DNN output and the true function for each of these frequencies using the discrete Fourier transform. The results, presented in Figure 5.4, illustrate the application of the F-Principle.

Theses observations have been supported by the Neural Tangent Kernel (NTK) theory. This theory introduced by [Jacot et al. \[2018\]](#) study the DNNs in the infinite-width limit connecting the to kernel methods. It state that during the gradient descent, the dynamic of the network function (linking input to output w.r.t parameters) is described by the associate kernel. This kernel is determine by the network depth, the choice of

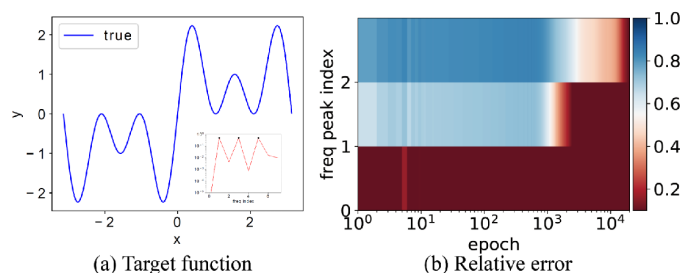


Figure 5.4: (a) The input function and its Fourier coefficients as inset, (b) Relative error between the DNN and the true Fourier coefficients per epoch for the three main frequencies. From [Xu et al. \[2019\]](#)

activation and the variance of the initialisation. The convergence is then linked to the definiteness of the infinite-width Neural Tangent Kernel. Furthermore for least-squares regression loss, the network function follows a linear differential equation in the infinite-width limit, and the eigenfunctions of the Jacobian are the kernel principal components of the input data. This perspective allows for a more tractable mathematical treatment: the evolution of the network’s output function can be described using the NTK, enabling precise characterization of how different function components are learned over time. Within this framework, the NTK theory provides theoretical backing for the Frequency Principle, the empirical observation that DNNs tend to learn low-frequency components of a target function before capturing high-frequency details. Specifically, the convergence rates of various frequency components during training are linked to the eigenvalues of the NTK matrix. Lower frequency components correspond to larger eigenvalues, leading to faster convergence.

Studies have extended the NTK framework to explain why Physics-Informed Neural Networks exhibit spectral bias, which affects their ability to accurately capture high-frequency components. [Wang et al. \[2020\]](#) demonstrated that when PINNs are trained by gradient descent, there is a notable discrepancy in the convergence rates of different components of the loss function due to this bias. In the infinite-width limit, the NTK remains nearly constant, leading to faster convergence for low-frequency components associated with larger eigenvalues compared to high-frequency components. This inherent bias in the NTK can result in challenges when trying to model scenarios with significant high-frequency behavior, causing PINNs to struggle with capturing the complete frequency spectrum effectively.

To address this issue, we explore several architectures proposed in the literature.

5.5.2 A frequency-aware network architecture

Our architecture proposal is mainly inspired by Multi-scale deep neural networks (MscaleDNNs) [Liu et al. \[2020\]](#) and WWP [Wang et al. \[2021b\]](#), which were designed to mitigate the effect of the F-principle in the standard (single level) context. The basic idea is to transform high frequency learning in low frequency learning and to facilitate the separation of

the frequency contents of the target function. As a result, these networks show uniform convergence over multiple scales.

The MscaleDNNs architecture achieves this objective thanks to two main ingredients:

- radial scaling in the frequency domain: the first layer is separated into N parts, each receiving a differently scaled input. Several variants of MscaleDNNs have been proposed, we choose to focus here on the most efficient one, which uses parallel sub-networks, each dedicated to a specific input scaling.
- the use of wavelet-inspired and frequency-located activation functions. These functions, with compact support, have good scale separation properties.

WWP networks were proposed by Wang, Wang, and Perdikaris. They use the same principle of input scaling, combined with soft Fourier mapping (SFM) defined as :

$$\gamma(z) = s \begin{bmatrix} \cos(z) \\ \sin(z) \end{bmatrix}$$

with $s \in [0, 1)$ a relaxation parameter.

A "Fourier features network" was first proposed in [Tancik et al. \[2020\]](#), which uses a random Fourier features mapping γ as a coordinate embedding of the inputs, followed by a conventional fully-connected neural network. This method did mitigate the pathology of spectral bias and helped networks learn high frequencies more effectively. Recent developments have made it possible to model fully connected neural networks (and thus PINNs) as kernel regression. The authors of [Tancik et al. \[2020\]](#) suggested that using Fourier mapping affects the width of the kernels and thus the network’s capacity to capture high frequencies. This idea was extended to PINNs in [Wang et al. \[2021b\]](#) with convincing results, provided that the fixed scalings are too far from the frequencies contained in the solution.

Inspired by their success, we now propose an architecture that combines the positive features of both methods. Our architecture’s output is the sum of several networks that use different scaling vectors, specializing each network for different frequency scales matching the structure presented in our theory in Section 5.2. To mitigate possible convergence problems arising from a bad choice of the fixed scalings, we have chosen to add learnable weights to the input scaling integers using SFM as a classic activation function.

For low frequency resolution, we choose a classical network using hyperbolic tangent activation functions. For the other networks, we use SFM activation functions for the first layer associated with a scaling from a centred normal distribution whose variance grows with the targeted frequencies. The other layers of the networks use hyperbolic tangent activation functions, thus ensuring our differentiability requirements. We refer to this architecture as Parallel-WWP (P-WWP). Unfortunately, the wavelet-based and frequency-based activation functions utilized in the MSCALE are not continuously differentiable, and thus fail to satisfy our theoretical assumptions. We have however included some (successful) experiments conducted using these functions in Appendix 8.4.

5.5. A “frequency-aware” distributed approach

A similar architecture, based on a lower number of subnetworks, has been proposed in Li et al. [2023] for the deep Ritz method Yu et al. [2018], which produces a variational solution to problem (2.4). To the best of our knowledge, this is the first time this architecture has been proposed for PINNs.

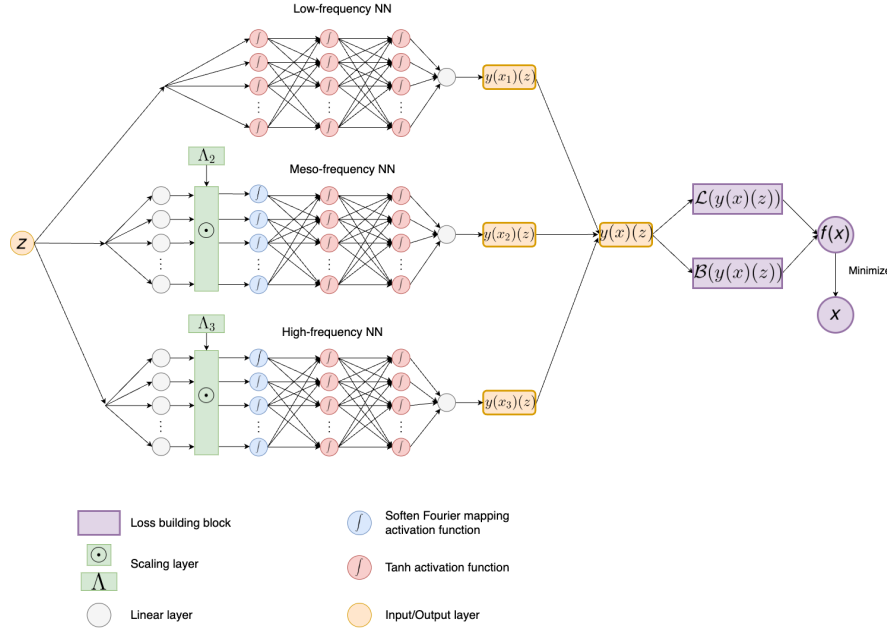


Figure 5.5: An example of P-WWP architecture

An example of our architecture is illustrated in Figure 5.5, which consists of three sub-networks, targeting three different frequency ranges defined by their input scaling Λ_i . The lowest frequency network employs tanh activation functions, while the others use SFM and tanh.

To be effective, this architectures must cover most of the frequencies contained in the a priori unknown PDE’s solution. Therefore, a broad range of frequencies must be covered, employing a large number of neurons, as the target frequencies are defined by the input scalings. As a result, the considerable improvement in accuracy provided by parallel frequency-aware architectures is obtained at the price of a significant increase in the training’s computational cost, making it an ideal candidate for our multilevel approach.

These architectures can be easily incorporated into our framework, where the global network is defined as a sum of y_i sub-networks, each responsible for a frequency range defined by its input scaling. Because of this latter characteristics, the resulting approach now belongs to the distributed context discussed in Section 5.2. In particular, we refer to the combination of the P-WWP network with the ML-BCD algorithm as the FAML (Frequency Aware Multi-Level) method. We use the same selection and termination criteria as in Section 5.4.1. When several sub-networks are available, the network with

the highest ratio $\|\nabla_{x_i}^1 f(x_k)\|/\|\nabla_x^1 f(x_k)\|$ is chosen.

5.6 Numerical results

We illustrate the performance of the proposed method on some Poisson problems and on the heat equation. Further tests performed with a slightly different frequency-aware architecture are presented in Appendix 8.4.

5.6.1 Poisson problems

We now illustrate the efficiency of the FAML method on some examples of the form (6.2.2) defined in Rapaka and Samtaney [2020]. We first describe the test problems themselves, then specify the considered network architectures and the training setup before describing and commenting the obtained results.

5.6.1.1 Test problems

Test problem 1: Circle embedded in a square domain The domain for the first problem is the square $\Omega = [-1, 1]^2$ with an embedded circle of radius $R = 0.5$ centered at the origin defining Ω_i . We consider Neumann boundary conditions on Γ_i and Dirichlet boundary condition on Γ_e , thus $a = c = 1$ and $b = d = 0$.

The source term and boundary term are given by

$$\begin{cases} r(\rho, \omega) = D(k^2 - n^2)\rho^{k-2} \sin(n\omega) \text{ in } \Omega \\ g(\rho, \omega) = -\frac{Dk}{n} \left(\frac{\rho}{R}\right)^{(n-k)} \rho^k \sin(n\omega) + Rq \log \rho \text{ on } \Gamma_e \\ h(\rho, \omega) = 1 \text{ on } \Gamma_i \end{cases} \quad (5.20)$$

where ρ, ω are the polar coordinates in the plane, $n \in \mathbb{Z}$, $z \in \mathbb{N}$ and $D = (\sqrt{2})^{-\max(k,n)}$. We choose $k = 1$ and $n = -5$. The solution is given by $u(\rho, \omega) = -\frac{Dk}{n} \left(\frac{\rho}{R}\right)^{(n-k)} \rho^k \sin(n\omega) + R \log \rho$. The solution and the source terms are depicted in Figure 5.6.

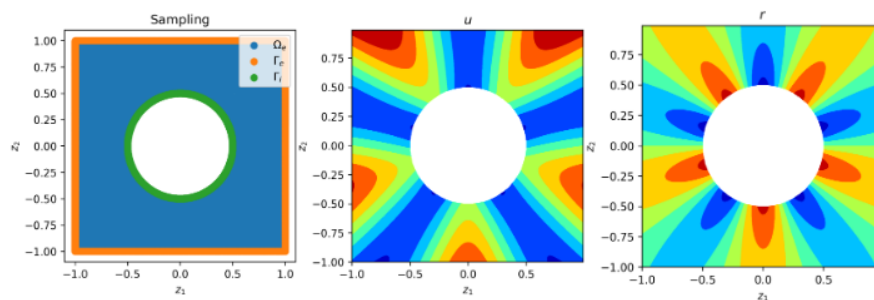


Figure 5.6: Test problem 1 (cf. (6.2.2), (5.20)). Left: the domain Ω and its subdomains. Center: the solution u . Right: the source term r .

Notice the variation in angular frequency as a function of the radius.

Test problem 2: Four-lobe structure The domain for the second problem is the unit square $\Omega = [0, 1]^2$ with an embedded surface defined by $\rho(\omega) = R_m + R_d \cos(4\omega)$ with $R_m = 0.0305$, $R_d = 0.117$. We consider Dirichlet boundary conditions for both Γ_e and Γ_i . The source term and boundary term are given by

$$\begin{cases} r(\rho, \omega) = 12(10\rho^2 - 1)e^{-10\rho^2} + \sum_{k=1}^4 40(10r_k^2 - 1) \text{ in } \Omega \\ g(\rho, \omega) = 0.3e^{-10\rho^2} + \sum_{k=1}^4 e^{-10r_k^2} \text{ on } \Gamma_e \cup \Gamma_i \\ r_k = \sqrt{(x \pm 0.45)^2 + (y \pm 0.45)^2}, \quad k = 1, \dots, 4 \end{cases} \quad (5.21)$$

where ρ and ω are the polar coordinates, and x and y the corresponding Cartesian coordinates. The solution is given by $u(\rho, \omega) = 0.3e^{-10\rho^2} + \sum_{k=1}^4 e^{-10r_k^2}$. The solution and the source terms are depicted in Figure 5.7.

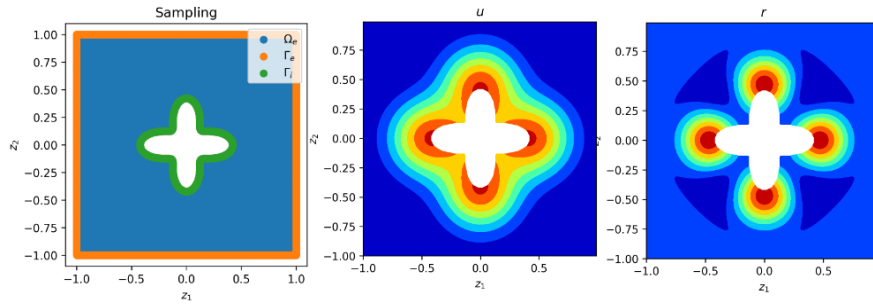


Figure 5.7: Test problem 2 (cf. (6.2.2), (5.21)). Left: the domain Ω and its subdomains. Center: the solution u . Right: the source term r .

Test problem 3: Annulus domain with homogeneous source We now consider a domain defined by an annulus with inner radius $R_i = 0.25$ and outer radius $R_o = 0.75$, and consider a Neumann boundary condition on Γ_i and a Dirichlet boundary condition on Γ_e . The source term and boundary term are given by

$$\begin{cases} r(\rho, \omega) = 0 \text{ in } \Omega \\ g(\rho, \omega) = 0 \text{ on } \Gamma_e \\ h(\rho, \omega) = 1 \text{ on } \Gamma_i \end{cases} \quad (5.22)$$

where ρ and ω are the polar coordinates. The solution is given by $u(\rho, \omega) = R_i \log(\frac{\rho}{R_o})$, which is depicted in Figure 5.8.

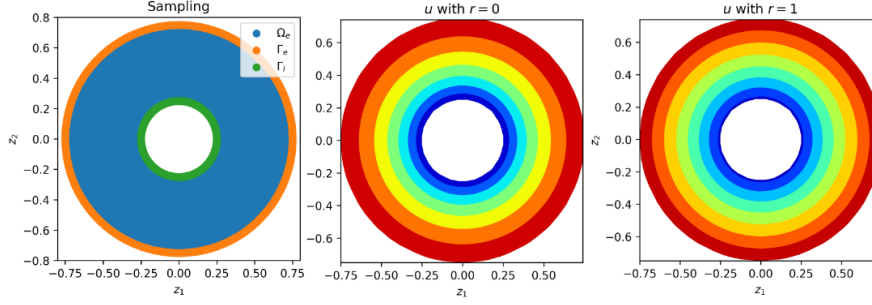


Figure 5.8: Test problems 3 (cf. (6.2.2), (5.22)) and 4 (cf. (6.2.2), (5.23)). Left: the domain Ω and its subdomains. Center: the solution u for Test Problem 3. Right: the solution u for Test Problem 4.

Test problem 4: Annulus domain with inhomogeneous source We finally consider again the domain defined by an annulus with inner radius $R_i = 0.25$ and the outer radius $R_o = 0.75$ with Neumann boundary condition on Γ_i and Dirichlet boundary condition on Γ_e . The source and boundary terms now given by

$$\begin{cases} r(\rho, \omega) = 1 & \text{in } \Omega \\ g(\rho, \omega) = 0 & \text{on } \Gamma_e \\ h(\rho, \omega) = 1 & \text{on } \Gamma_i. \end{cases} \quad (5.23)$$

The solution is then given by $u(\rho, \omega) = \frac{\rho^2 - R_o^2}{4} + R_i(1 - \frac{R_i}{2}) \log(\frac{\rho}{R_o})$.

5.6.1.2 Network architecture

We now give the details of the network architectures used in our second set of experiments. It is important to notice that our method is not just applicable to P-WWP networks, but can be used to train any architecture composed of sum of subnetworks.

	input scaling	First Activation	Other Activations
subnetwork 1	None	tanh	tanh
subnetwork 2	$\mathcal{N}(0, 20)$	SFM(0.5)	tanh
subnetwork 3	$\mathcal{N}(0, 40)$	SFM(0.5)	tanh
subnetwork 4	$\mathcal{N}(0, 60)$	SFM(0.5)	tanh

Each of the subnetworks is composed of 3 hidden layers of 100 neurons each. In order to assess computational performance, we also consider the standard single level training applied on the complete network.

5.6.1.3 Training setup

We will use the same setup as in Section 5.4 except for the coefficients $\lambda_{\Omega_e} = 1$, $\lambda_{\Gamma_e} = 100$ and $\lambda_{\Gamma_i} = 1$ that weight internal and boundary losses. These values were chosen by a

grid search on the optimization of the network with classical training.

We consider as a unit of cost the cost of optimising our complete neural network over an epoch. When a subnetwork is selected in the ML-BCD algorithm, the unselected part of the complete network is cached in memory and does not contribute to the subnetwork optimization cost. Since the cost per iteration is linear in the number of parameters when using first-order methods, the cost of optimizing for an epoch one of four subnetworks of identical sizes is $\frac{1}{4} = 0.25$ cost units. For these experiments we chose to do 1000 epochs in full network cycle and 4000 epochs in sub-network cycle. Thus the cost of the full and partial training is similar. We do a total of 9 cycles with 1000 epochs on the full network to start the training, for a total computational cost of $1000 + 9 \times (1000 + \frac{4000}{4}) = 19000$ units.

For each problem we first compute the curve of the median values of the loss over 10 runs and of the MSE, as a function of epochs and for two different computational budgets (10k and 19k units). We then select the lowest median loss obtained for a given budget and record the associated median MSE.

5.6.1.4 Numerical results

Table 5.2 reports the results just described, obtained with the Frequency-Aware-Multilevel (FAML), cf. also Figure 5.11.

Problem	Budget	MSE	MSE FAML	Loss	Loss FAML
Test pb 1	10000	1.40E-05	2.54E-07	1.64E-01	1.18E-02
	19000	7.70E-06	2.49E-07	6.33E-02	4.89E-03
Test pb 2	10000	1.66E-05	3.99E-07	1.45E-01	1.17E-02
	19000	7.77E-06	2.57E-07	5.38E-02	4.42E-03
Test pb 3	10000	1.05E-05	1.36E-07	1.04E-01	6.76E-03
	19000	3.81E-06	1.11E-07	3.71E-02	2.55E-03
Test pb 4	10000	9.53E-06	1.62E-07	1.02E-01	7.40E-03
	19000	3.58E-06	1.32E-07	3.75E-02	2.63E-03

Table 5.2: Best value of the median loss for the given budget and associated MSE for standard single-level and FAML training (10 independent runs)

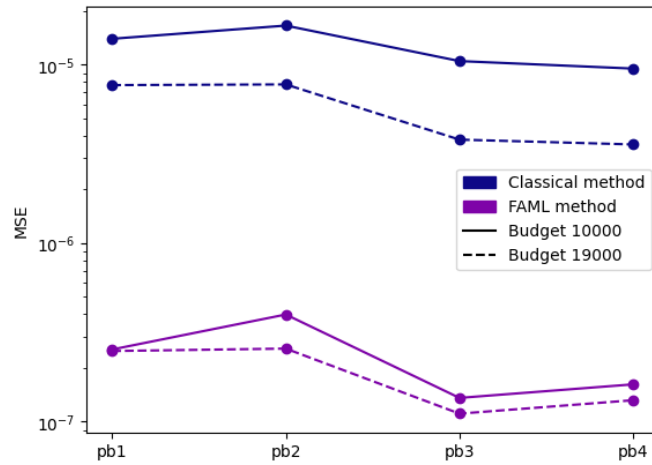


Figure 5.9: Comparison of average MSE values (from Table 5.2)

In each case, the proposed multi-level training yields lower losses than the standard one. The differences are particularly significant for a small computational budget where the improvement is of several orders of magnitude. The multi-level training also always results in a lower associated MSE.

To provide further insight, we finally provide a typical example allowing the comparison of standard and multi-level training: for Test problem 3, we illustrate in Figure 5.10 the decrease of the total loss and of its boundary and interior components as well as that of the MSE as a function of computational cost. The multi-level approach clearly outperforms the standard one. We remind the reader that peaks correspond to the restarts of the optimizer.

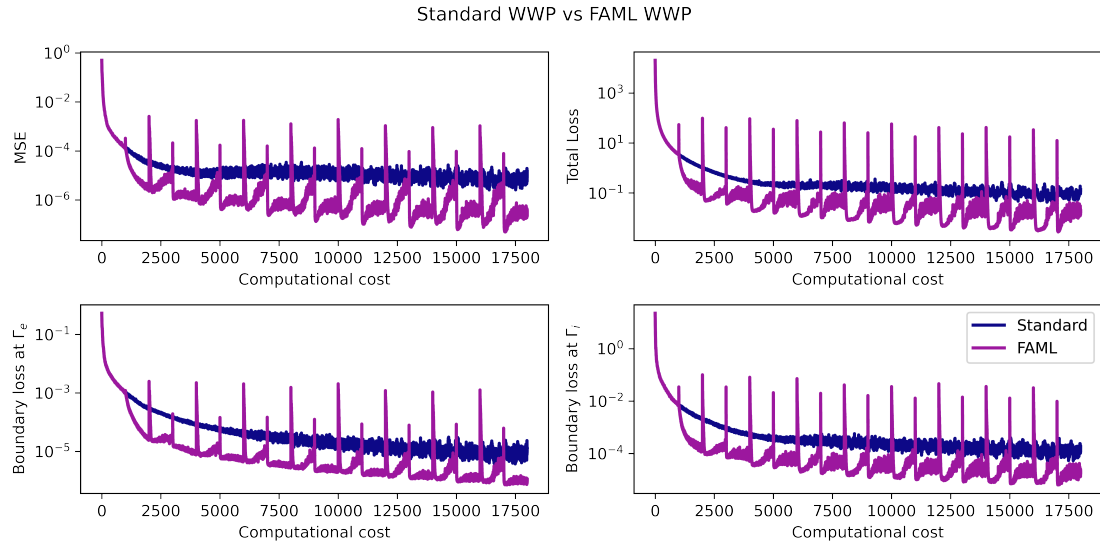


Figure 5.10: Evolution of MSE and loss values for a typical run on Test problem 3

5.6.2 The heat equation

We next consider a non-elliptic heat equation. Given a positive parameter ω , the problem is stated as

$$\begin{cases} \frac{\partial u(z, t)}{\partial t} = \frac{1}{(\omega\pi)^2} \frac{\partial^2 u(z, t)}{\partial z^2} \text{ with } (z, t) \in (0, 1) \times (0, 1), \\ u(z, 0) = \sin(\omega\pi z), \quad z \in [0, 1], \\ u(0, t) = u(1, t) = 0, \quad t \in [0, 1], \end{cases} \quad (5.24)$$

and its exact solution is given by

$$u(z, t) = e^{-t} \sin(\omega\pi z).$$

We consider different test problems for values of $\omega = 1, 20, 50$. As for the Poisson problems, we use P-WWP networks with the following architecture.

	input scaling	First Activation	Other Activations
subnetwork 1	None	tanh	tanh
subnetwork 2	$\mathcal{N}(0, 50)$	SFM(0.5)	tanh
subnetwork 3	$\mathcal{N}(0, 100)$	SFM(0.5)	tanh

Each of the subnetworks is composed of 3 hidden layers of 50 neurons each. The training setup remains the same. In this case, as we have three subnetworks, the cost of one epoch in subnetworks training is 0.33 cost units. We report the result of our method in Table 5.3, cf. also Figure 5.11.

Problem	Budget	MSE	MSE FAML	Loss	Loss FAML
$\omega = 1$	10000	5.64E-06	1.48E-06	1.18E-01	1.65E-02
	19000	8.19E-07	2.89E-07	5.23E-03	4.50E-03
$\omega = 20$	10000	5.50E-04	5.19E-05	1.97E-02	8.68E-03
	19000	1.77E-05	7.97E-06	2.90E-03	2.45E-03
$\omega = 50$	10000	1.13E-01	1.13E-02	1.01E+00	1.10E-01
	19000	2.58E-02	2.55E-04	2.08E-01	1.35E-02

Table 5.3: Best median of the loss for the given budget and associated MSE for standard single-level and FAML training (10 independent runs)

As in the case for the elliptic equation, our method yields a faster decrease of the MSE in the first part of the training as compared to the classical training. This is true for all values of ω . Note that much lower errors are obtained at the end of the training and for high values of ω .

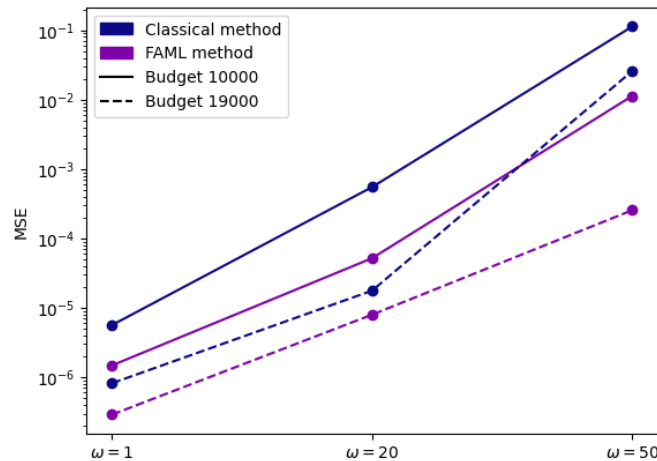


Figure 5.11: Comparison of average MSE values (from Table 5.3)

5.7 Conclusion and perspectives

We first developed a new point of view on multilevel optimization methods, arguing they can be seen as block-coordinate minimization methods in a higher dimensional space. Distinguishing two contexts (hierarchical and distributed), we reformulated a class of multilevel algorithms in block form and showed how convergence results for block-coordinate methods can be applied to the multilevel case.

We next illustrated this approach for the approximate solution of partial differential equations, using physics-informed neural networks as a block solver, and presented numerical experiments with a method using pure alternating training (in the hierarchical context) and a more elaborate frequency-aware technique (in the distributed one) on

complex Poisson and heat propagation problems. On these problems, both proposed multilevel PINNS methods consistently produced lower losses than those obtained using conventionally trained networks. Convergence was also shown to be much faster, resulting in a considerable reduction of the computational cost to obtain good solutions (often better than those obtained by a classical training).

While these initial results are encouraging, more research remains desirable for a better understanding of the algorithms. Their dependence on techniques to transfer algorithmic hyperparameters between levels is of particular interest. Applying our approach to more general operator learning is also worth further investigation. Finally, we plan to develop a more efficient implementation of the method in terms of GPU memory management.

Chapter 6

A coarse space acceleration of deep-DDM

6.1	Introduction	93
6.2	The Deep Domain Decomposition Method (Deep DDM)	94
6.2.1	Basics of domain decomposition	94
6.2.2	Deep DDM	95
6.3	Extension via a Coarse Network	98
6.4	Numerical Results	99
6.5	Conclusion	101

6.1 Introduction

Recent advancements in machine learning, particularly Physics-Informed Neural Networks, have opened new avenues for solving complex Partial Differential Equations. In the context of classical solvers, Domain Decomposition Methods (DDMs) have long been recognized for their efficiency and scalability, especially when applied to large-scale problems. However, the application of these techniques is not straightforward but offer the potential to enhance both the flexibility and accuracy of deep learning approaches.

The combination of DDMs with neural networks represents a significant shift in computational strategies, allowing for the efficient computation of high-dimensional problems. This chapter aims to explore the intersection of these two powerful techniques, focusing on how neural network-based multilevel strategy, particularly through the use of PINNs, can be effectively incorporated into DDM frameworks.

While domain decomposition methods are well-established solvers for PDEs using classical discretizations, the use of neural network-based discretizations, in particular, PINNs has been explored more recently. This concept has been explored in various studies like for example in [Shukla et al. \[2021\]](#), where the authors discuss non-overlapping DDMs for parallel training. Extensions to Schwarz methods for Deep Ritz networks, which integrate the variational form into the loss function, are presented in [Li et al.](#)

[2019]. The coupling in the finite basis PINNs (FBPINNs) approach in Moseley et al. [2023] differs from the aforementioned approaches; the authors introduce an overlapping domain decomposition, and the coupling is performed via a corresponding partition of unity scaling and hard enforcement of boundary conditions. For a broader overview over the combination of domain decomposition and machine learning methods, we refer to the review Klawonn et al. [2024].

Our focus is on employing PINNs as the subdomain solver in a classical Schwarz approach iteration; this is known as the Deep Domain Decomposition Method (Deep-DDM) introduced in Li et al. [2020a]. Given that the subdomain problems can be solved sufficiently accurately, the convergence properties are the same as for the classical Schwarz iteration. As a result, the method is not numerically scalable when increasing the number of subdomains. In this paper, we will incorporate a coarse level, in order to retain numerical scalability. A related two-level Schwarz approach, which uses a different coupling between the two levels of a classical Schwarz iteration has been presented in Jang et al. [2024]. Other related multilevel approaches are, for instance, multilevel FBPINNs Dolean et al. [2022, 2024] and multilevel optimization methods using frequency-aware networks Gratton et al. [2024].

6.2 The Deep Domain Decomposition Method (Deep DDM)

6.2.1 Basics of domain decomposition

Domain decomposition methods originate from the work of Hermann Schwarz Schwarz [1870] on the solution of the Poisson equation. Before this work, the existence of a solution was only proven for simple geometries. Hermann Schwarz had the idea to consider a domain of complex geometry as a union of simple geometries as described in Figure 6.1. From there he sets up an iterative algorithm: local solutions of the Poisson equation are performed on subdomains taking for boundary condition the value of the solution on the neighboring subdomains. In the case of 2 subdomains, Schwarz's methods writes:

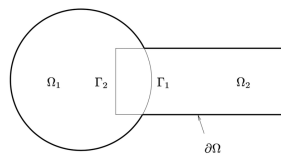


Figure 6.1: The diagram used by Schwarz in 1870.

$$\begin{cases} \Delta u_1^{k+1} = f \text{ in } \Omega_1 \\ u_1^{k+1} = g \text{ on } \Gamma \cap \partial\Omega_1 \\ u_1^{k+1} = u_2^k \text{ on } \Gamma_1 \end{cases} \quad \text{and} \quad \begin{cases} \Delta u_2^{k+1} = f \text{ in } \Omega_2 \\ u_2^{k+1} = g \text{ on } \Gamma \cap \partial\Omega_2 \\ u_2^{k+1} = u_1^{k+1} \text{ on } \Gamma_2. \end{cases} \quad (6.1)$$

After showing the convergence of this algorithm he obtains, in the limit, a solution of the Poisson equation on the composite geometry. With the emergence of numerical simulation, these methods have regained interest because they can be modified into a parallel algorithm. Indeed, with few modifications, we can obtain an algorithm whose solutions on the subdomains can be performed concurrently as follows:

$$\begin{cases} \Delta u_1^{k+1} = f & \text{in } \Omega_1 \\ u_1^{k+1} = g & \text{on } \Gamma \cap \partial\Omega_1 \\ u_1^{k+1} = u_2^k & \text{on } \Gamma_1 \end{cases} \quad \text{and} \quad \begin{cases} \Delta u_2^{k+1} = f & \text{in } \Omega_2 \\ u_2^{k+1} = g & \text{on } \Gamma \cap \partial\Omega_2 \\ u_2^{k+1} = u_1^k & \text{on } \Gamma_2. \end{cases} \quad (6.2)$$

Note the method we described so far is mainly theoretical, as it involves the solutions of continuous problems. To solve numerically the problem it is necessary to apply a discretization scheme on each subdomain, using for example a method such as the finite element method. The central idea of the article [Li et al. \[2020a\]](#) is to replace these numerical solvers by neural networks and more precisely by PINN.

6.2.2 Deep DDM

Let the computational domain Ω be decomposed into S overlapping subdomains $\Omega_1, \dots, \Omega_S$. Then, we consider the classical Schwarz iteration [Schwarz \[1870\]](#), which involves an independent problem on each subdomain Ω_s : find u_s

$$\begin{cases} \mathcal{N}(u_s) = f & \text{in } \Omega_s, \\ \mathcal{B}(u_s) = g & \text{on } \partial\Omega_s \setminus \Gamma_s, \\ \mathcal{D}(u_s) = \mathcal{D}(u_r) & \text{on } \Gamma_s, \end{cases} \quad (6.3)$$

where $\mathcal{D}$ is an operator for the transmission conditions (e.g., Dirichlet, Neumann, or Robin) on the subdomain boundary, and Γ_s is the interface between the subdomain Ω_s and the neighboring subdomains Ω_r , with $\Omega_r \cap \Omega_s \neq \emptyset$. For convenience, we define the multiple overlapping subdomains as a single subdomain Ω_r with a corresponding solution network u_r . In order to train a PINN model to solve 6.3, we incorporate an additional term accounting for the transmission conditions into the loss function 2.5. In particular, the loss term is computed by sampling points on the interface denoted $\{\tilde{x}_i\}_{i=1}^{N_\Gamma} \subset \partial\Omega_r$. A visualization of all sampling points is shown in 6.2. For the transmission to Ω_r , with $\Omega_r \cap \Omega_s \neq \emptyset$, we incorporate the loss term:

$$\mathcal{M}_\Gamma(w) = \frac{1}{N_\Gamma} \sum_{i=1}^{N_\Gamma} |\mathcal{D}(u_s(\tilde{x}_k)) - W_i|^2 \quad (6.4)$$

In this term, we minimize the difference between the trained network u_s and the neighboring networks u_r on the interface with respect to the transfer operator $\mathcal{D}$. Here,

$$W_i = \mathcal{D}(u_r(\tilde{x}_k)) \quad (6.5)$$

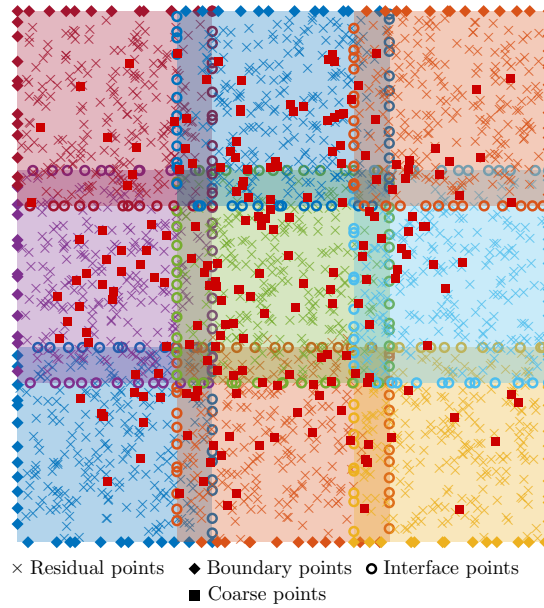


Figure 6.2: Sampling points for the two-level Deep-DDM method

is defined based on the network u_r from the previous outer Schwarz iteration. Once all the local subnetworks have been trained up to a certain stopping criterion (e.g., number of iterations or tolerance), the interface values W_i are being updated, and we proceed to the next outer iteration.

The one-level Deep DDM algorithm, without the red parts, is shown in 1. Initially, two stopping criteria were used to assess convergence: differences in the network solution between subsequent iterations, both in the interior and at the boundary. In addition, a loss criterion variation was used to terminate the training of each network, alongside a maximum epoch limit. This method made comparisons between runs difficult as the number of epochs varied. Here we simplify by using a fixed number of epochs per training and fixed outer iterations. For further details on the original criteria, refer to Li et al. [2020a].

As mentioned in the introduction, the convergence of the one-level method does not scale with the number subdomains. We consider two types of scalability:

- *Strong scalability*: Strong scalability is defined as how the solution time varies with the number of cores for a fixed total problem size. Ideally, the elapsed time is inversely proportional to the number of processing units.
- *Weak scalability*: Weak scalability is defined as how the solution time varies with the number of cores for a fixed problem size per core. Ideally, the elapsed time is constant for a fixed ratio between the size of the problem and the number of processing units.

Algorithm 1 Two-level DeepDDM; *coarse level*

- 1: Sampling the fine *and the coarse* collocation points
 - 2: Initialization of the network parameters w_s^0 *and* w_c
 - 3: Initialization of interface values $W = [W_1, \dots, W_s]$
 - 4: *Initialization of weights λ_f and λ_c*
 - 5: **tant que** Iteration limits not reached **faire**
 - 6: Local network training
 - 7: *Coarse network training*
 - 8: *Compute $\sum_{s=1}^S E_s(\chi_s u_s(x_{i,coarse}))$ for each coarse points*
 - 9: Update of W_k values at interfaces with 6.4 *or* 6.6
 - 10: *Update λ_f and λ_c*
 - 11: **fin tant que**
-

Since these terms are not clearly defined for of deep learning, we will assume that the size of our problem is the number of points sampled to solve the problem, and the number of processing units is the number of PINNs models used to solve the problem (and therefore the number of subdomains).

We consider a Poisson equation with Dirichlet boundary conditions:

$$\begin{cases} \Delta u = r(x) & \text{in } \Omega = [0, 1] \times [0, 1], \\ u = g(x) & \text{on } \partial\Omega. \end{cases}$$

We choose r and g such that the exact solution is $u(x) = \sin(\omega_1 \pi x_1) \sin(\omega_1 \pi x_2) + \sin(\omega_2 \pi x_1) \sin(\omega_2 \pi x_2)$, in this paper we will perform tests for several values of ω_1 and ω_2 . To investigate the scalability of the one-level method we choose $\omega_1 = \omega_2 = 1$ and we perform 1500 epochs per training (other settings are the same as in 6.4). Here, we test the strong scalability on regular rectangle domain decomposition. A regular rectangle domain decomposition with overlap divides $\Omega = [L_{x_0}, L_{x_1}] \times [L_{y_0}, L_{y_1}]$ into $N_x \times N_y$ subdomains, each overlapping by $\alpha_x \Delta x$ and $\alpha_y \Delta y$. The total number of points sampled remains the same in each experiment, corresponding to our problem size, while we increase the number of subdomains and thus the number of processing units. In 6.3, the mean squared error over outer Schwarz iterations for several numbers of subdomains are plotted. We observe that the convergence deteriorates when increasing the number of subdomains. Therefore, in the next section, we will introduce a coarse level for the Deep-DDM, which facilitates fast global transport of information, to retain scalability.

6.3 Extension via a Coarse Network

Our coarse level corresponds to training a neural network acting on the entire domain, which we will denote as the *coarse network*. The convergence of the two-level method will then depend on the coarse network solution as well as the exchange of information with the local networks on the first level of the method. In particular, as the coarse network, we train a classical PINN model on the global domain and add an additional

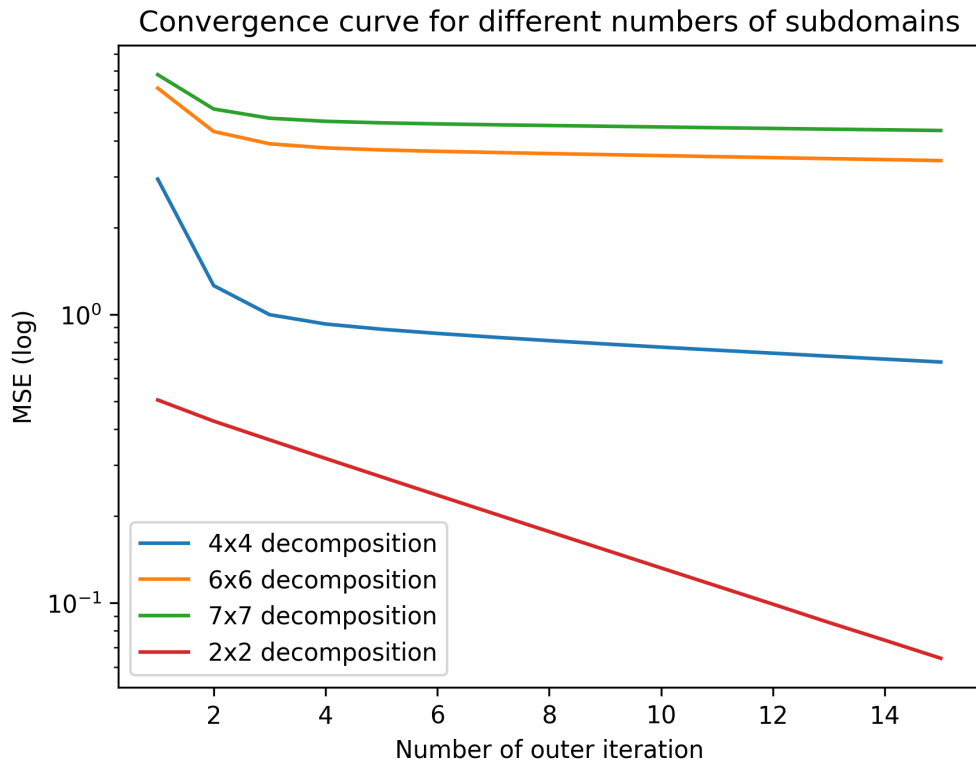


Figure 6.3: Sampling points for the two-level Deep-DDM method

loss term incorporating the local subdomain networks. Conversely, after training the coarse network, we will incorporate the coarse network into the loss function for the training of the local networks.

Extension operators and a partition of unity **Dolean et al. [2015]**: Let us define an extension operator

$$E_s(w_s) = \begin{cases} w_s & \text{in } \Omega_s \\ 0 & \text{otherwise.} \end{cases}$$

Here, w_s is a function defined on Ω_s . Moreover, we define partition of unity functions χ_s with $\chi_s \geq 0$, $\text{supp}(\chi_s) \subset \overline{\Omega_s}$, $\chi_s(x) = 0$ for $x \in \partial\Omega_s \setminus \partial\Omega$, and

$$w = \sum_{s=1}^S E_s(\chi_s w|_{\Omega_s})$$

for any function w defined on Ω . We then add the term

$$\mathcal{M}_{fine}(w_c) = \frac{1}{N_{\Omega_c}} \sum_{k=1}^{N_{\Omega_c}} |u_c(x_k^c) - \sum_{s=1}^S E_s(\chi_s u_s(x_k^c))|^2,$$

to the loss function 2.5 for the training of the coarse network. Here, $\{x_k^c\}_{k=1}^{N_{\Omega_c}}$ are the sampling points for the coarse network Ω related loss (example in 6.2), and u_c and w_c are the coarse network and its network parameters, respectively. This term which transfer information from the fine solution to the coarse network is weighted with λ_f . As we aim to train the coarse network concurrently with the fine networks, making our algorithm fully parallelizable, we use the u_s from the previous outer iteration. This information is unavailable for the first iteration, so in this case, we set λ_f to 0.

In order to transfer information from the coarse to the local networks, we introduce a loss term into the M_Γ loss of the local networks via the interface term 6.5:

$$W_i = \lambda_c \mathcal{D}(u_r(\tilde{x}_i)) + (1 - \lambda_c) \mathcal{D}(u_c(\tilde{x}_i)), \quad (6.6)$$

where $\lambda_c \in [0, 1]$ is a weight balancing the impact of the coarse network on the local networks.

Note that, since the accuracy of the local networks is often poor in the first iterations, we adjust the corresponding weight in the loss function λ_f during the Schwarz iteration. The final two-level algorithms is given in 1, including the red parts.

6.4 Numerical Results

In this section, we will conduct tests on the previously defined Poisson's problem using the following settings. The collocation points are sampled using Latin hypercube sampling in Ω and on $\partial\Omega$ and Γ . For our strong scaling tests, we fix $N_\Omega = 30\,000$ and $N_{\partial\Gamma} = N_\Gamma = 16\,000$ for the whole problem while increasing the number of subdomains. All networks are trained using the Adam optimizer with an initial learning rate of 2×10^{-4}

	2500 epochs	5000 epochs	7500 epochs
One-level method	66 min	148 min	190 min
Two-level method	71 min	136 min	201 min

Table 6.1: Median of the wall time for the experiments of the figure 6.7

and an exponential decay of 0.999 every 100 epochs. Each neural network is composed of two hidden layers with 30 neurons. The overlap is set to 30% of the subdomain larger side. In each Schwarz iteration, each local and coarse network is trained for 2 500 epochs. The weight controlling the impact of the coarse network on the local networks is set to $\lambda_c = 1 \times 0.9^I$ where I is the index of the Schwarz iteration. This increases the impact of the coarse network during the course of the outer Schwarz iteration. The coefficient λ_f controlling the impact of the fine networks on the coarse network is set to a fixed value of 0.5. All weights λ_* have been optimized using a rough grid search to obtain good performance.

Our implementation uses TensorFlow2 (version 24.02) and runs numerical experiments on a single Nvidia A100-80 GPU. Both the fine and coarse network training processes run concurrently on the GPU using multiprocessing. Although this setup is suboptimal due to the unaddressed GPU load and the complex interaction between TensorFlow and multiprocessing, it allows us to compare the wall time between one-level and two-level methods. The results presented are the median of three independent training runs, each initialized with different seeds.

We focus on investigating the impact of our two-level domain decomposition on the *spectral bias* or *f-principle* [Xu et al. \[2019\]](#), [Rahaman et al. \[2019\]](#) of neural networks; this refers to the observation low-frequency components of the target functions are learned much faster than the high-frequency components. The Deep-DDM method tackles this issue by splitting the global problem into smaller subproblems, allowing for better approximation of high frequency components, while the coarse network is supposed to learn the low frequency components.

In order to investigate this, we consider two different pairs of coefficients $(\omega_1, \omega_2) \in \{(1, 3); (1, 6)\}$.

For the lower frequency pair of coefficients, that is, $(\omega_1, \omega_2) = (1, 3)$, we observe a noticeable improvement in convergence with the addition of a coarse level; cf. 6.4. In particular, the convergence seems to be independent of the number of subdomains. However, when we increase the frequency of the solution, that is, $(\omega_1, \omega_2) = (1, 6)$, we notice that it takes longer for the two-level method before it converges; cf. 6.6. We observed that this problem can be easily solved by improving hyper parameter settings. In particular, for 6×6 subdomains, we observe that we can significantly improve the scalability by increasing the number of epochs for each subproblem; cf. 6.7. Notably, with 5 000 and 7 500 epochs per subproblem, the two-level methods clearly outperforms the one-level Deep-DDM.

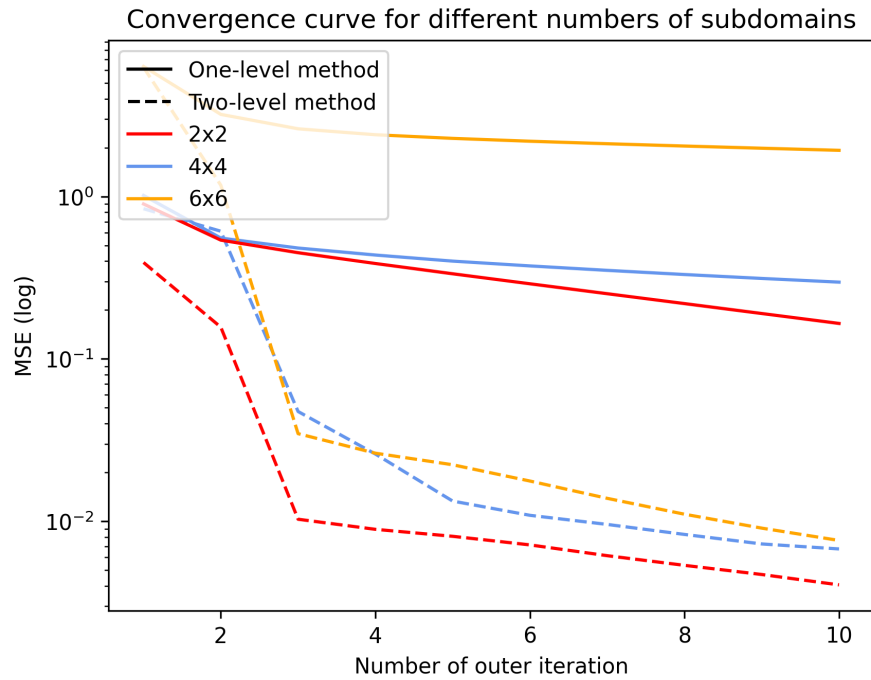
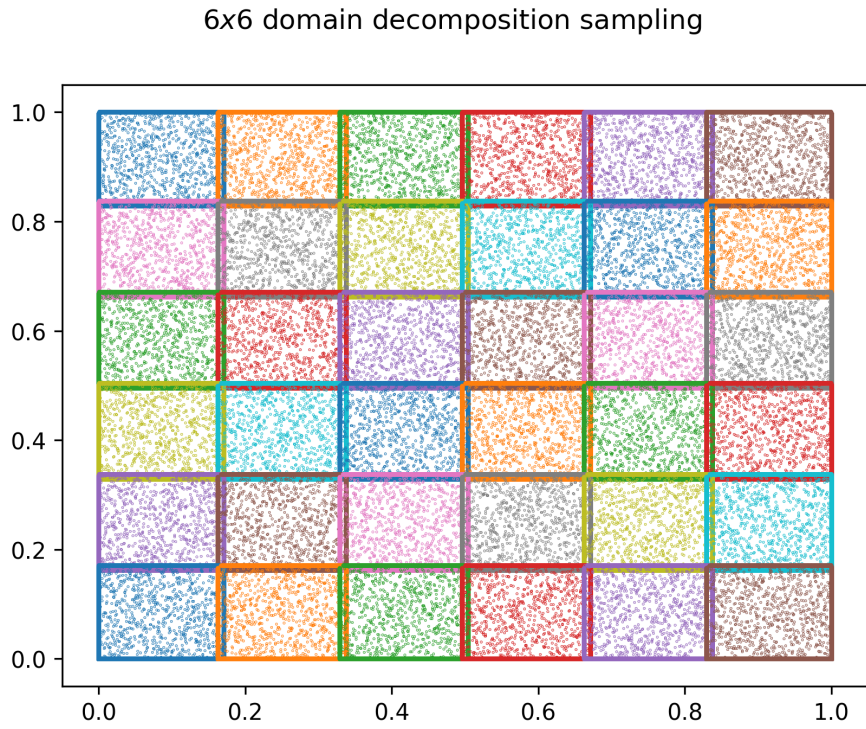
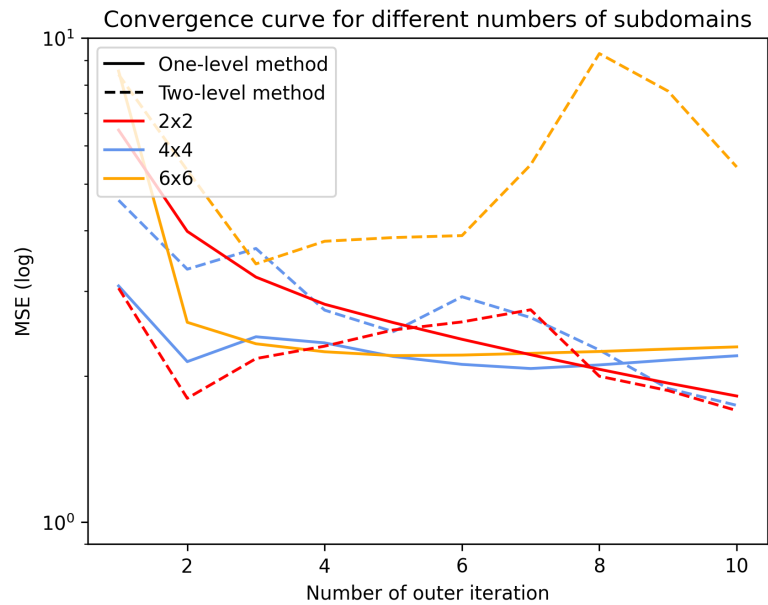


Figure 6.4: Strong scalability test for test problem with $\omega_1 = 1, \omega_2 = 3$: Test with 2 500 epoch

6.5 Conclusion

We have presented a two-level approach to improve the convergence of the one-level Deep-DDM method . The additional coarse networks facilitates faster global transport of information and enhances the scalability of the Deep-DDM method to larger numbers of subdomains. The cost of training the coarse network is relatively low compared with the total cost of the method and the method is well-suited for a parallel implementation with only small differences in wall time (6.1) with a non optimized parallelization.

Figure 6.5: An example of sampling with a 6×6 decompositionFigure 6.6: Strong scalability test for test problem with $\omega_1 = 1, \omega_2 = 6$ with 2 500 epoch

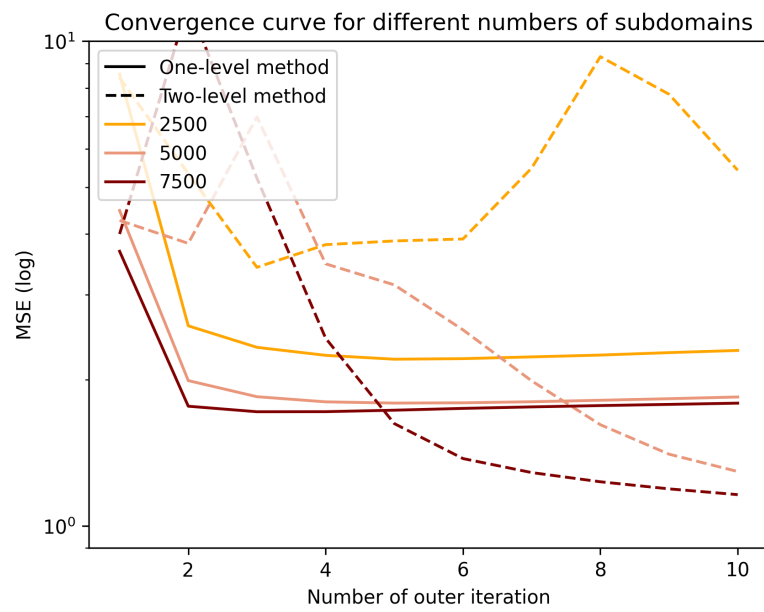


Figure 6.7: Testor test problem with $\omega_1 = 1, \omega_2 = 6$ on 6×6 decomposition with variation in the number of epochs

Conclusion

In the last two chapters we have discussed the significant contributions that have been made to the development and application of physics informed neural networks (PINNs). Our work has focused on improving the efficiency of PINNs through advanced multi-level optimisation techniques, enabling more efficient computation, distribution and targeting of multi-frequency problems. These improvements have demonstrated significant gains in reducing computational costs and improving convergence rates, particularly in the context of solving simpler PDEs such as the Poisson equation and the heat equation.

However, despite these advances, there are still significant limitations to the methods we have developed. In particular, the computational time required by these improved PINNs is still far from achieving our original goal of real-time forecasting, especially when dealing with more complex hyperbolic PDEs such as the Barré de Saint-Venant equations that model flood dynamics. The computational requirements of these more complicated PDEs, combined with the limitations of the current PINN framework, make real-time flood forecasting impractical at this stage.

Recognising these limitations, we have shifted our focus to exploring alternative approaches that can better meet the demands of real-time flood forecasting. Taking advantage of more efficient frameworks and state-of-the-art GPUs, we have investigated the potential of surrogate models. These models offer a promising way to overcome the computational challenges currently faced by PINNs, and provide a more viable solution for real-time flood modelling applications.

Part III

Flood-prediction

Chapter 7

Graph neural networks for flood alert system

7.1	Introduction	105
7.2	Dataset Creation	106
7.2.1	Initial and boundary conditions	107
7.2.2	Generating Realistic Floods	108
7.3	Graph neural networks for flood alert system	108
7.3.1	Model basis	108
7.3.2	Multimesh creation	112
7.3.3	Boundary conditions	112
7.3.4	Loss Function	113
7.3.5	General training Setup	113
7.3.6	Metrics	114
7.4	Numerical results	115
7.4.1	Multimesh MeshGraphNet	115
7.4.2	Overflow detection	121
7.4.3	Frequential analysis	126
7.5	Conclusions and Perspectives	129
7.5.1	Future Directions	129

This chapter presents the work undertaken to develop substitute models for traditional simulation software used in flood management. More specifically, the goal is to propose a real-time model capable of rapidly predicting flood-prone areas while avoiding the high computational costs associated with classical methods. To achieve this, we created a dedicated dataset and employed methodologies based on graph neural networks to develop the model. We then analyze its strengths and limitations before proposing potential improvements.

The work presented in this chapter has led to the forthcoming publication of an article on [Nvidia technical blog](#). Additionally, a research paper is currently in preparation.

7.1 Introduction

Floods are one of the most common and dangerous natural disasters, affecting around 1.5 billion people worldwide. Each year, they cause as much as 25 billion in damage to economies (STA). Predicting floods accurately is very important for preparing for disasters and reducing risks. Reliable forecasts can save lives, protect important infrastructure, and reduce economic losses.

Traditionally, flood forecasting relies on physics-based simulations, such as those created by software like Telemac 2D (Hervouet [2007]). Engineers, including those at BRLi, use these tools to study different rainfall scenarios. They provide local authorities with valuable information about flood risks, how to reduce them, and how to plan for emergencies. However, these methods take a lot of time and computing power. Simulating a single flood can take several hours, even with advanced processors. This makes them unsuitable for real-time forecasting when quick decisions are needed, delaying critical warnings.

Recently, deep learning has shown great potential to provide real-time flood forecasts. For example, Pianforini et al. [2024] used a video prediction model to forecast water levels up to 3 hours in advance in under a minute on an NVIDIA A100 80 GPU, achieving accurate results for dam-break scenarios. Similarly, Bentivoglio et al. [2023a] tested graph neural networks (GNNs) for simulating dam breaks on regular meshes. These models worked well for predicting how water spreads but were limited by their ability to scale to larger or more irregular areas due to high computing needs.

To address these issues, Bentivoglio et al. [2024] introduced a multiscale GNN model inspired by the work of Fortunato et al. [2022], Lino et al. [2022]. This approach extended the use of GNNs to irregular meshes, achieving a $700\times$ speedup compared to traditional methods with an error margin of only 0.15m. However, these improvements were mainly limited to dam-break scenarios and relatively small meshes (up to 20,000 cells).

The real-world challenges faced by BRLi engineers require the use of irregular meshes that can accurately model small structures significantly affecting physical processes, while covering large areas. This leads to meshes containing several hundred thousand to even millions of mesh nodes. Furthermore, the scenarios addressed in our work differ from the commonly studied dam-break cases in the literature, where water flows over dry plains. Instead, we focus on a more frequent and practical phenomenon in riverine contexts: river floods. These events are characterized by the gradual overflow of a river's banks due to upstream discharge increases, involving specific hydraulic dynamics and distinct modeling constraints.

In this chapter, we explore whether graph-based methods can tackle these large-scale, real-world flood scenarios.

To test this, we created a high-quality dataset using a real case study of the Têt River near Perpignan, France. The study area includes the river, its nearby waterways, and urban zones like Saint-Estève, Perpignan, Bompas, Clair, Villelongue-de-la-Salanque, Torrelles, Sainte-Marie-la-Mer, and Canet-en-Roussillon, which are vulnerable to flooding.

We then built a model based on the MeshGraphNet architecture (Pfaff et al. [2020]), improving it with a multi-mesh creation process inspired by GraphCast (Lam et al. [2022]). This approach allowed us to expand the model’s capacity to process large areas while keeping the computational effort manageable. In our experiments, the model produced accurate flood maps with a six-hour prediction range in under one second on an NVIDIA A100 80 GPU. In comparison, Telemac 2D required 25 minutes to simulate the same scenario on 24 CPUs. However, challenges like detecting sudden flood overflows remain, showing areas where further improvements are needed.

7.2 Dataset Creation

To create a reliable dataset for the Têt River, we used the numerical simulation software Télémac 2D (3.3). The first step was to design a mesh, which is a detailed grid that models the study area. This process followed the techniques described in 3.4.1, with specific density zones created based on their importance. We also used softlines to better represent structures like highway embankments (see Fig. 7.1).



Figure 7.1: Density map used to create the mesh used for simulations on the river Têt.

The mesh was carefully designed to capture key features of the region:

- **Main riverbed and embankments:** These zones have the highest density, with an average distance between nodes of 5 to 10 metres, to accurately capture small-scale water dynamics.

- **Urban areas:** Zones near populated areas had a high density, with an average distance between nodes of 20 metres, to improve accuracy in regions most at risk of flooding.
- **General area:** Other parts of the study area used have an average distance between nodes of 50 metres.
- **Specific study zone (Chenal Vert):** This area received an high density, with an average distance between nodes of 5 meters.

Softlines were used to refine the modeling of key structures, such as embankments near the riverbed, helping the model better capture flood behavior. Using this configuration, we obtain a mesh of 412,844 nodes and 2,013,374 elements. This mesh is interpolated onto the topographical surveys conducted in the area to obtain the bed elevation and friction values.

7.2.1 Initial and boundary conditions

To model the flood, upstream liquid boundary conditions are applied with an imposed water flow in cubic meters per second in the form of a hydrograph. This hydrograph models the characteristics of the flood. Downstream, another liquid boundary condition with imposed water height is used to model the sea, and rigid wall conditions are applied elsewhere. Initially, our domain is empty, which would result in an unrealistic filling phase. To achieve a proper initialization, we first dug an artificial lake upstream and filled it with water to avoid errors caused by the sudden appearance of water on a dry surface. We then run an initialization simulation with an upstream flow rate of $100 \text{ m}^3/\text{s}$ and a downstream imposed water level gradually rising to 1.5 m. This simulation runs for 100 h until the domain stabilizes.

7.2.2 Generating Realistic Floods

To create a realistic database, we based our approach on a historical analysis of floods to generate hydrographs. Specifically, we selected the 20 most significant historical floods in terms of peak discharge. We then calculated the shape coefficient (K_c), which is determined between the peak of the hydrograph and the daily mean discharge. This daily mean discharge is computed to be maximized over the 24-hour window containing the flood peak. The higher the shape coefficient, the sharper the flood hydrograph, indicating a shorter duration and smaller volume.

We used this index to cluster the hydrograph types into four distinct groups. Each group was then normalized and averaged, resulting in four representative hydrographs 7.2. Once these typical hydrographs were obtained, we scaled them by multiplying with peak discharges ranging between that of the 2020 flood ($1000 \text{ m}^3/\text{s}$) and that of the 1940 flood ($3600 \text{ m}^3/\text{s}$), starting the hydrographs at $100 \text{ m}^3/\text{s}$ to match the initialization discharge. This scaling was performed across 14 value intervals.

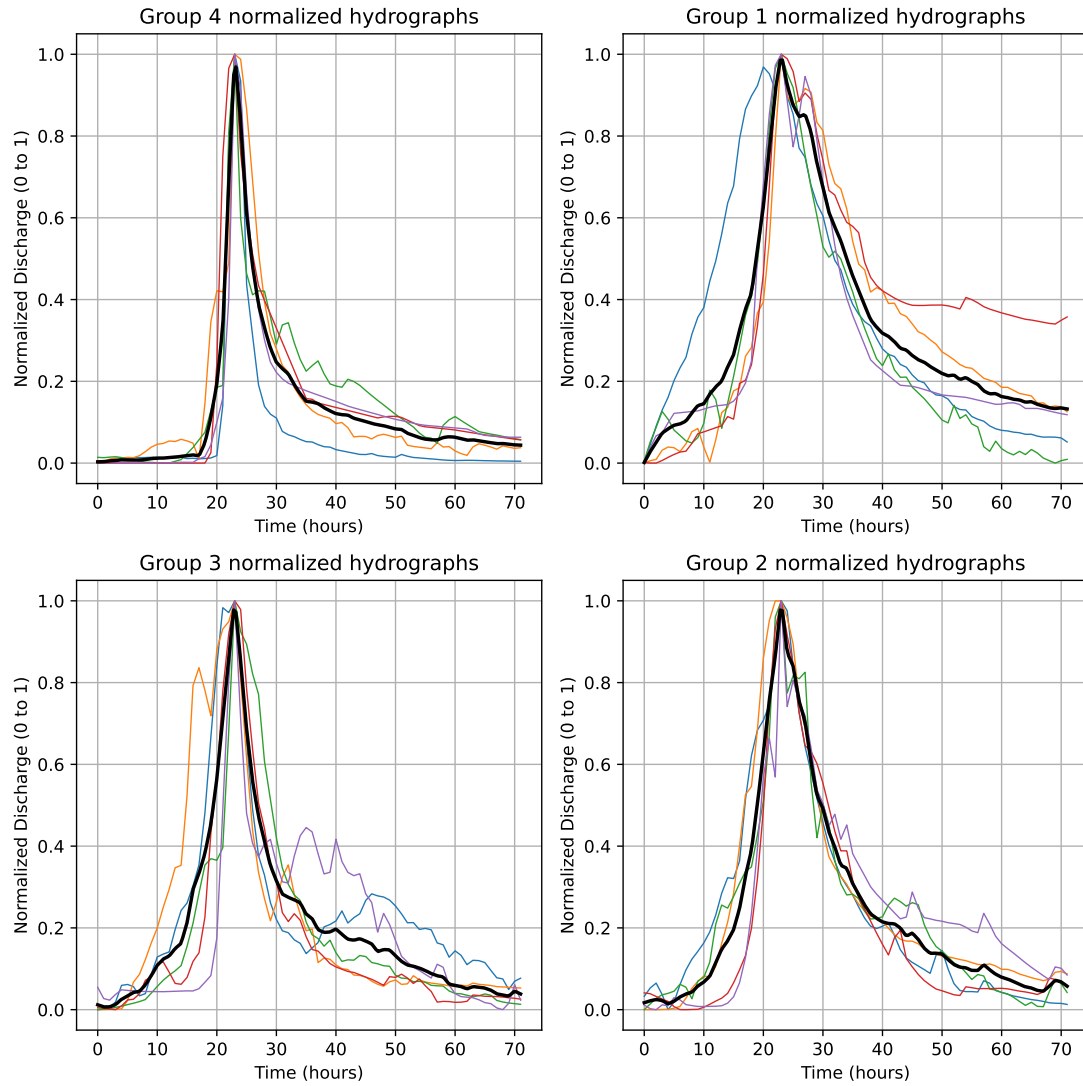


Figure 7.2: The four groups of hydrographs obtained by clustering with the shape coefficient. In color the normalized historical hydrographs and in black the mean hydrogram.

Using the generated meshes and the defined initial and boundary conditions, we utilized Telemac2D (3.3) to generate our dataset. Simulations were performed on 28 CPUs with a time step of 1 second. The specific options and configurations used in the .cas file are detailed in 8.5. As a result, we generated $14 \times 4 = 56$ hydrographs, each with a duration of 40 hours, forming our database. We obtain a result in Selafin format, which contains at each node the water height and velocities, as well as static bed elevation and friction values. We chose to have an output every 30 minutes in our database.

7.3 Graph neural networks for flood alert system

7.3.1 Model basis

The model used in this work is based on the MeshGraphNet architecture (Pfaff et al. [2020]), which is an encoder-processor-decoder framework utilizing graph neural networks. The core idea is to represent the simulation mesh as a graph after certain transformations. By applying message passing on this graph, we create local updates based on each node's neighborhood and then map these updated values back into the simulation domain.

To achieve this, we transform the values at the nodes of the simulation mesh. The positions of the nodes in the x, y plane are converted into relative data on the edges of the graph. Specifically, for two nodes X_1 and X_2 , we obtain a vector that concatenates the relative position between the two nodes $X_1 - X_2$ and the norm of this vector. Since the edges are bidirectional, this information depends on which node is being updated. The other informations : water height (h), velocities (u and v), strickler coefficient (k) and the elevation (z) stays on the nodes (7.3). We add to the node information a one-hot encoded vector indicating whether the node is a boundary condition node (inflow, imposed height, or wall) or a classical internal node.

Using this graph representation, two separate fully connected neural network encoders are used to encode the node and edge features, respectively, into a latent space. In the processor phase, a network computes updated edge features by considering the latent representations of the two connected nodes and the current edge features. For each node, the connected edge values are aggregated (in our case by summation) and concatenated with the node's latent representation. This combined vector is then fed into another network that computes the new node latent representation. These processing operations are repeated across multiple message passing layers, effectively increasing the radius of influence for each node update. At the end, the updated latent representations are decoded back to the simulation mesh to produce the final predictions 7.4. In practice, the model learns to predict the changes in the dynamic variables (such as water height Δh and velocities Δu and Δv) rather than their absolute values. These values are predicted for internal nodes and only Δu and Δv are predicted for imposed height nodes. For more details please refer to 2.2.3 or Pfaff et al. [2020].

The number of message-passing layers we use directly affects how much information a node can gather from its neighbors. With each additional layer, the receptive field

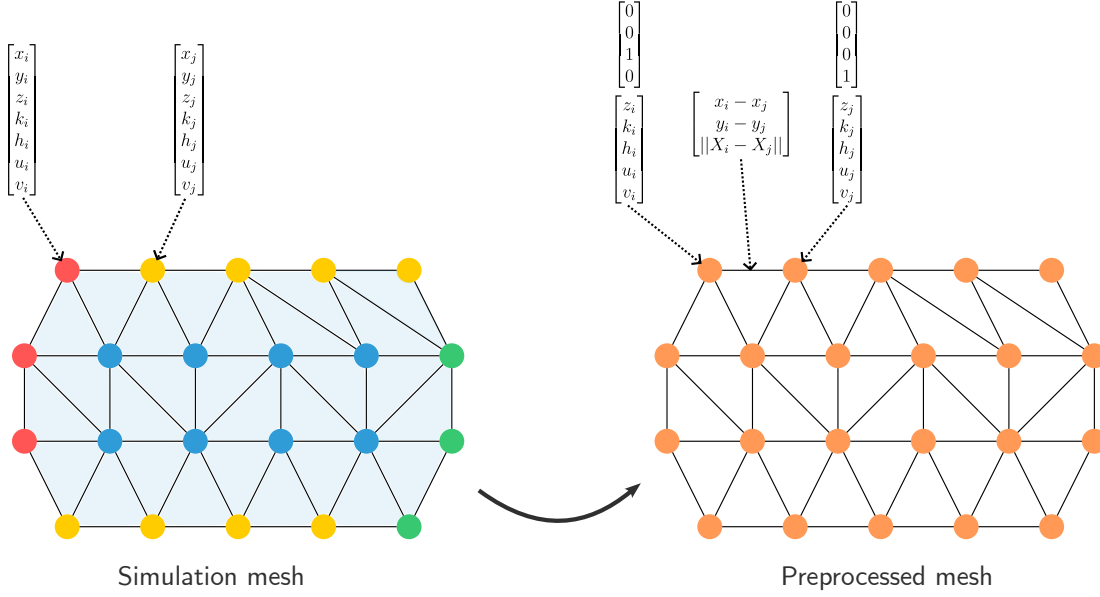


Figure 7.3: Transforming Telemac mesh data into data that can be interpreted by our MeshGraphNet. The colours on the nodes indicate different types of node (various boundary conditions or interior nodes).

(the part of the graph a node can pull data from) gets bigger. However, increasing the number of layers comes at a significant cost because it makes the model much more computationally demanding. This is especially a challenge for our graph, which is based on the simulation mesh. In the most detailed part of the mesh, the riverbed, we have elements spaced every five meters to capture small-scale dynamics. Even if we use a high number of layers, like twelve (which is common in the literature), the receptive field would only extend to a 60-meter radius. This isn't enough to capture the changes that happen over 30 minutes in an area that spans several kilometers.

Adding more message-passing layers makes the computation more intensive, as each layer requires processing more information for every node and edge in the graph. The data handled by the model grows exponentially, leading to longer training times and higher memory demands. In our case, the fine resolution of the mesh, particularly in the riverbed, results in a large number of nodes and edges. Therefore, increasing the receptive field by adding more layers quickly becomes computationally unfeasible, limiting the model's ability to account for the long-range interactions needed to accurately simulate flood dynamics over large areas.

7.3.2 Multimesh creation

To address this problem, we took inspiration from Graphcast (see 4.1.2 or Lam et al. [2022]) and created a multimesh. The goal is to add edges to the graph in order to capture long-range dependencies. Unlike Graphcast, where the mesh is regular and the

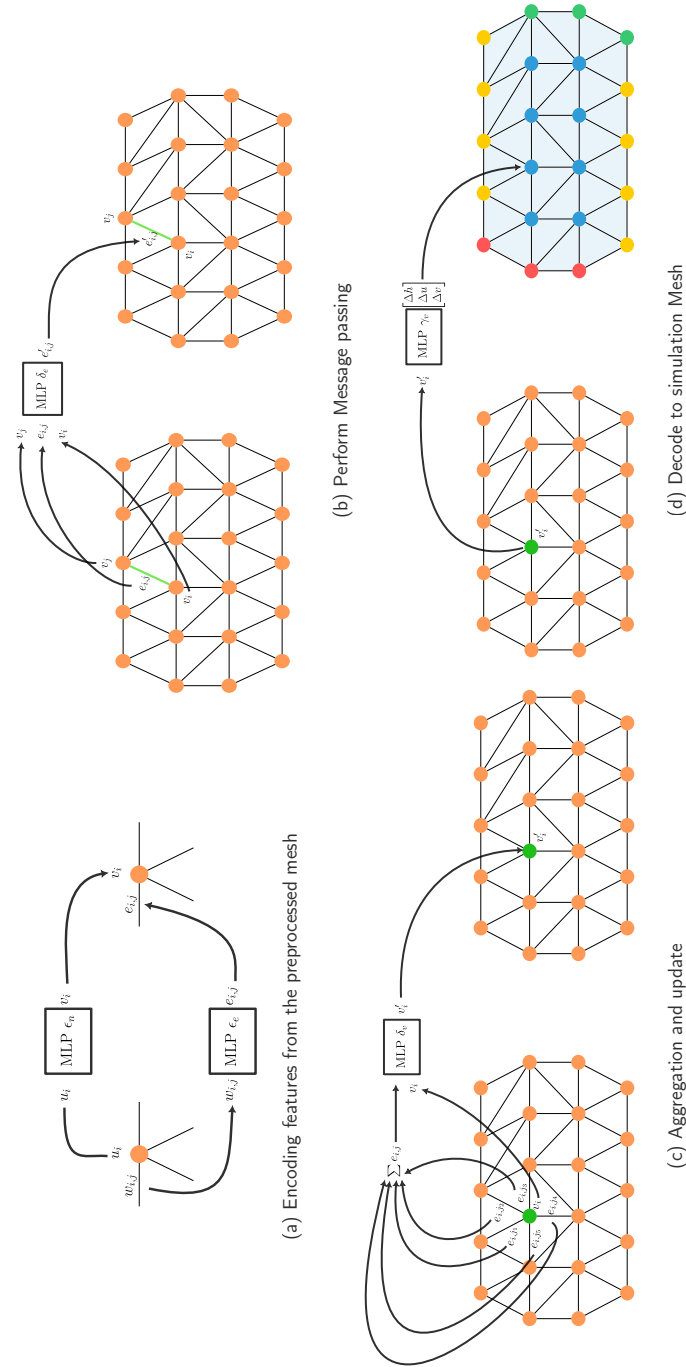


Figure 7.4: Summary of the different stages of the MeshGraphNet method. Steps (a) encoding data to a latent space on nodes and edges using two separate MLPs. Steps (b) and (c) message passing, aggregation and updating of values on the nodes, these steps are repeated as many times as there are message passing layers. Step (d) decoding values at nodes to physical space and updating.

multimesh can be built incrementally, we are working with a "custom" mesh that was designed by engineers for this specific application. To create this multimesh, we use our meshing software again and multiply the densities of the different zones by factors of 2, 4, 8, 16, and 32. This results in progressively coarser meshes 7.5. For each of these coarser meshes, we employ a KD-Tree algorithm to identify pairs of the closest points within the original, finer mesh, using the distribution of the coarsened mesh as a guide. We then add edges directly between these closest points in the fine mesh. This approach allows us to significantly increase the receptive field of our model without increasing the number of message passing layers. By incorporating multiple levels of mesh coarseness, we effectively expand the neighborhood each node can influence within the same computational budget. The coarser meshes capture long-range interactions by connecting nodes that are farther apart, compensating for the limited receptive field imposed by computational constraints on the number of message passing layers 7.1.

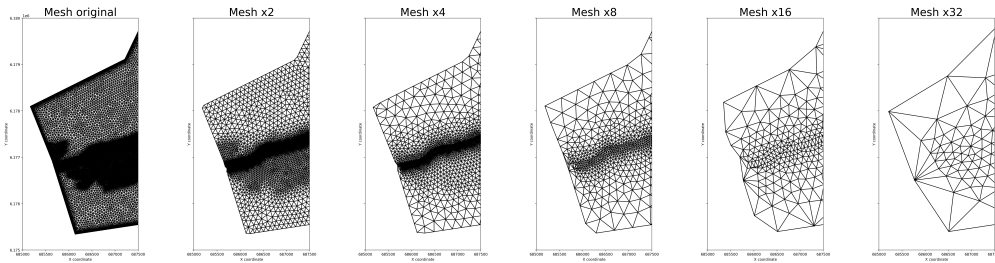


Figure 7.5: Visualization of the various meshes generated for multi-mesh creation, focused on a zoomed-in section of the domain.

	Original	x2	x4	x8	x16	x32
Normal	min: 0.750	min: 1.520	min: 3.014	min: 5.898	min: 12.005	min: 66.464
	mean: 15.743	mean: 27.939	mean: 48.407	mean: 81.025	mean: 141.169	mean: 282.536
	max: 82.968	max: 159.055	max: 319.864	max: 630.155	max: 1148.238	max: 2301.499
	nodes: 412,844	nodes: 128,050	nodes: 43,271	nodes: 16,222	nodes: 6,166	nodes: 1,830
Multimesh	min: 0.750	min: 1.520	min: 3.014	min: 5.898	min: 12.005	
	mean: 24.788	mean: 43.346	mean: 71.475	mean: 110.437	mean: 171.562	-
	max: 2293.732	max: 2288.121	max: 2363.003	max: 2521.218	max: 2410.404	
	nodes: 412,844	nodes: 128,050	nodes: 43,271	nodes: 16,222	nodes: 6,166	

Table 7.1: Minimum, average and maximum distance in metres between two connected points on the graph with the number of mesh nodes. The multi-meshes of a chosen mesh contain the edges created by all the coarser meshes according to the detailed method.

7.3.3 Boundary conditions

To inject the boundary conditions in the model we use dedicated nodes which are not learned. Inflow nodes give for each time step the height and velocities values at the next time step and imposed height nodes give the height at the next time step.

7.3.4 Loss Function

The model was trained using a one-step-ahead loss function, focusing on predicting changes in the hydraulic variables (Δh , Δu , Δv) for the next timestep based on the current state. The loss was computed using the Mean Squared Error as follows:

$$\mathcal{L} = \|\Delta \hat{y}^{t+1} - \Delta y^{t+1}\|_2, \quad (7.1)$$

where $\Delta y^{t+1} = (\Delta h^{t+1}, \Delta u^{t+1}, \Delta v^{t+1})$ represents the ground truth changes in water height, horizontal velocity, and vertical velocity at the next timestep, and $\Delta \hat{y}^{t+1}$ represents the model's predictions for these values.

This one-step approach ensures that the model learns to predict short-term changes accurately while keeping the training process computationally efficient. Although the formulation could be extended to multistep forecasting, the scope of this work focused solely on single-step predictions, with multistep forecasts explored only during inference.

7.3.5 General training Setup

Numerous training sessions were conducted in this chapter; therefore, we detail here the constants used in each training. All models were trained using an implementation based on Nvidia Modulus [Modulus Contributors] in PyTorch [Paszke et al., 2019] and the Direct Graph Library [Wang et al., 2019]. The training process employed the Adam optimization algorithm with an initial learning rate of 0.0005, which decayed by a factor of 0.999995 at each epoch. To reduce computational costs and memory footprint, training was conducted using Automatic Mixed Precision for Deep Learning (AMP). Enabling mixed precision involves two steps: first, adapting the model to use half-precision data types where appropriate; second, applying loss scaling to preserve small gradient values. For more details, see Nvidia [2024].

In terms of hardware, we had access to the Turpan supercomputer, which allowed us to use up to six nodes, each equipped with two Nvidia A100-80 GPUs. The Modulus codebase enabled data parallelism across the nodes. On the full training dataset (47 flood events), we performed speedup tests to validate the efficiency of multi-node usage (Fig. 7.6).

All data were normalized during training by subtracting the mean and dividing by the variance and were subsequently de-normalized for testing.

7.3.6 Metrics

We assessed the models' performance using the L_1 norm for each or every hydraulic variable x over the entire simulation period, calculated as:

$$L_1(x) = \|\hat{x} - x\|,$$

We analyzed the spatio-temporal error distribution of water depth using the critical success index (CSI) for thresholds (typically 5cm), following the methodology of Bentivoglio

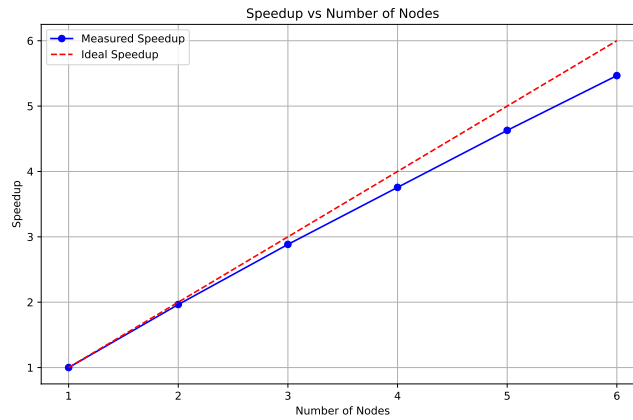


Figure 7.6: Speedup of the training code with respect to the number of nodes used on the entire training dataset (47 flood events).

et al. [2024, 2023a]. The CSI quantifies the spatial accuracy in detecting specific classes (e.g., flooded or non-flooded areas) for a given threshold and is defined as:

$$\text{CSI} = \frac{\text{TP}}{\text{TP} + \text{FP} + \text{FN}} \quad (7.2)$$

where TP refers to true positives (cells correctly identified as flooded by both the model and simulations), FP denotes false positives (cells incorrectly predicted as flooded by the model), and FN represents false negatives (cells where the model fails to detect flooding). The CSI ranges between 0 and 1, providing a normalized measure of performance. A CSI value of 0 indicates a complete failure to identify flooded areas correctly, while a value of 1 represents perfect agreement between the model predictions and reference data. However, it is important to note that when the number of true positives is low, the CSI becomes sensitive to noise, as even small variations in false positives or false negatives can significantly impact the score.

7.4 Numerical results

7.4.1 Multimesh MeshGraphNet

To assess our approach under controlled conditions, we began by focusing on a single subset of hydrographs (Group 3). Within this group, we randomly selected 11 flood events for training and retained 3 events for testing. These test floods feature peak discharges of $1600 \text{ m}^3/\text{s}$, $2200 \text{ m}^3/\text{s}$, and $3200 \text{ m}^3/\text{s}$, covering a wide range of flood intensities.

We then trained our model for 500 epochs using the Adam optimizer on the finest multi-mesh configuration described earlier. Model performance was evaluated using two main metrics:

7.4. Numerical results

- the L1 error computed on denormalized water height h and velocity components u, v (velocity components can be found in the appendix in Fig. 8.2)
- the Critical Success Index for nodes with water depth above 5 cm.

Overall, the network demonstrated low L1 errors and high CSI 7.2 values for one-step-ahead forecasts, implying strong short-term predictive capability (Fig. 7.7).

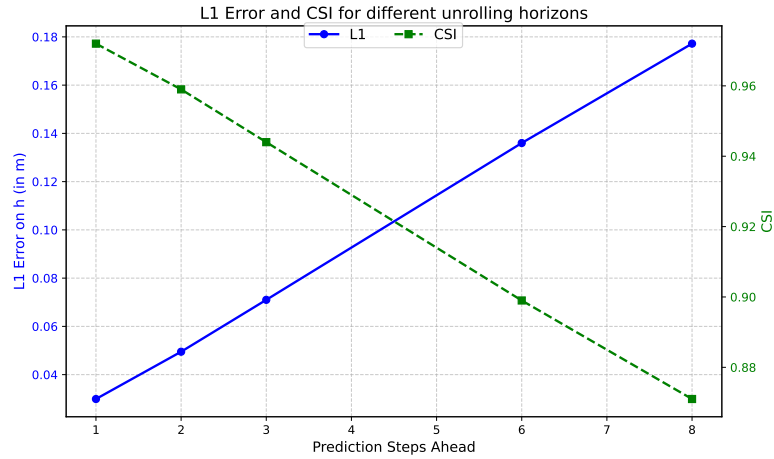


Figure 7.7: Evolution of L1 error on h (blue) and CSI (green) on test floods as a function of the number of unrolling steps

However, when the model was unrolled to make predictions several steps into the future, the error inevitably accumulated, although at a rate suggesting it remains useful for short lead times. A closer examination of these multi-step forecasts revealed a failure in capturing the onset of flooding. As shown in Fig. 7.8 for the 2200 m³/s test flood, the model missed the point in time when the water overflows the river bed.

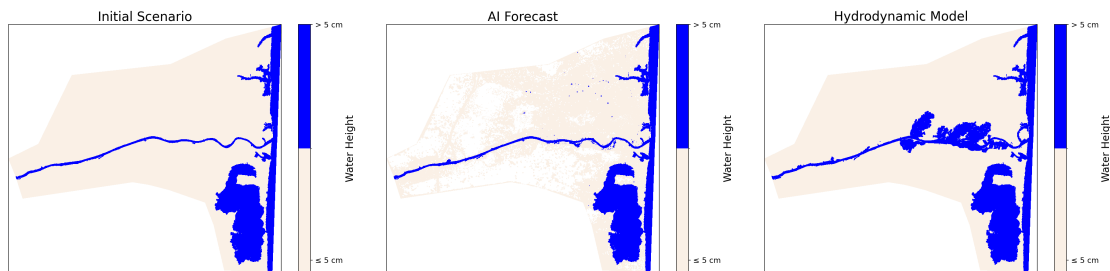


Figure 7.8: An example of a three-hour prediction from our model where the overflow is missed. Here we show a flood map with the areas with a water height greater than 5cm in blue. From left to right we have the starting point of the prediction, the prediction of our model and our groundtruth generated with Telemac 2D.

To better characterize the model’s tendency to miss overflow onset, we first defined a mask capturing the initial wetted area. This mask includes any region where the water depth exceeds 5 cm, at the start of the flood encompassing the main riverbed, lakes, and sea basins that do not typically flood (Fig. 7.9).

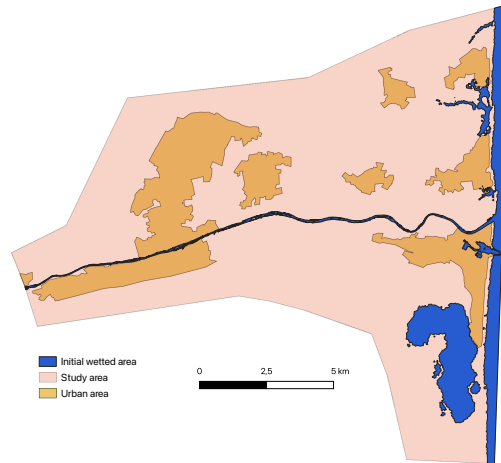


Figure 7.9: Initial wetted zone polygon used as a mask.

Using this mask, we define a new metric: the CSI overflow, written as CSI_{over} . This metric is the CSI calculated for points outside the mask of the initially wetted region. It helps us describe how well we detect flood overflow.

$$CSI_{over} = CSI|_{x \notin \text{initilly wetted region}} \quad (7.3)$$

We tested this metric on floods with a prediction horizon of six hours (corresponding to 12 unrolled steps) for each starting point in the floods, using a threshold of 5cm. We compared this to the same metric applied to the identity, which checks the groundtruth values at time t and at time $t + 6$, hours. For the trained model, the CSI_{over} at a given time $t = 1360$ represents the metric computed between the model’s prediction, unrolled 12 steps forward from $t = 1000$, and the ground truth at $t = 1360$. For the identity baseline, the CSI_{over} at the same point reflects the metric computed between the ground truth values at $t = 1000$ and $t = 1360$. For each flood, we also added a graph above the one showing CSI_{over} . This upper graph displays the wetted area at the prediction time and includes illustrations of the flooded area. These results are shown in Figs. 7.10–7.12.

As expected, the identity curve matches the slope of the wetted area curve. When the flood starts with small overflows (a small positive slope) or decreases (a small negative slope), our model performs better than the identity. However, during times of strong flooding and large overflows (a steep positive slope), our model does not perform better than the identity.

7.4. Numerical results

These measurements confirm what was illustrated in Fig. 7.8. Our model captures small flows changes but does not capture the main flood overflow.

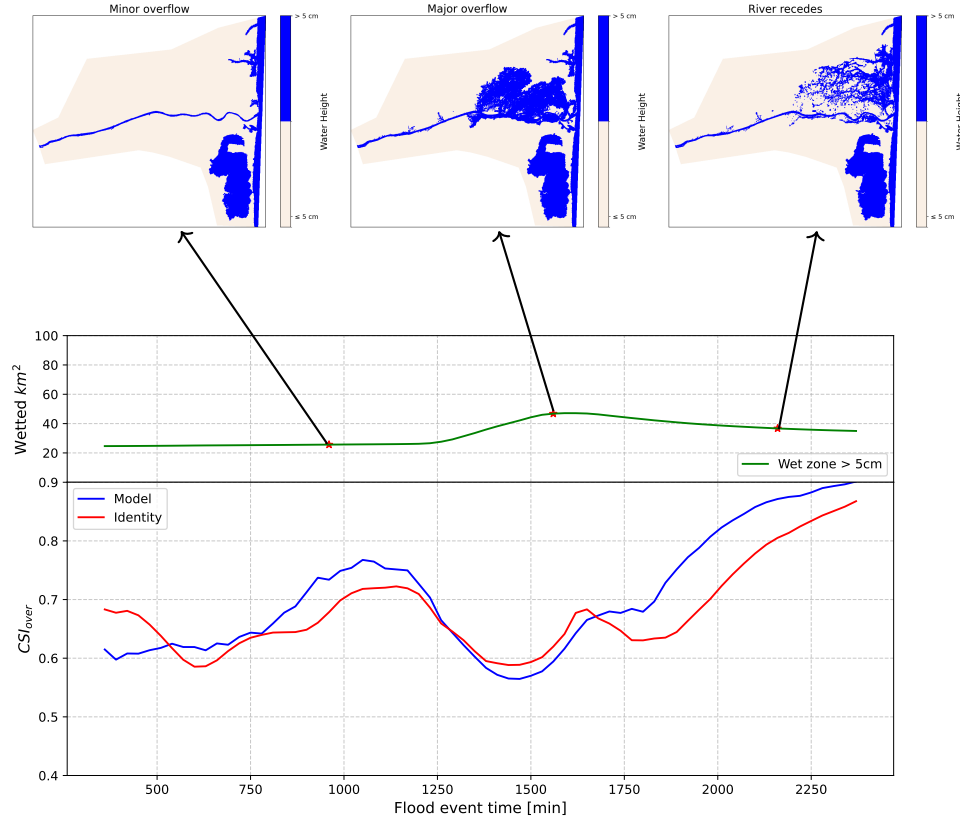


Figure 7.10: From bottom to top: CSI_{over} calculated at a prediction horizon of six hours at all possible time steps of the test flood with a peak flow of $1600 \text{ m}^3/\text{s}$. Change in wetted area of at least five centimetres for each of the time steps at which the CSI_{over} is calculated. Illustrations of the area flooded by at least 5 cm of water at different times during the flood.

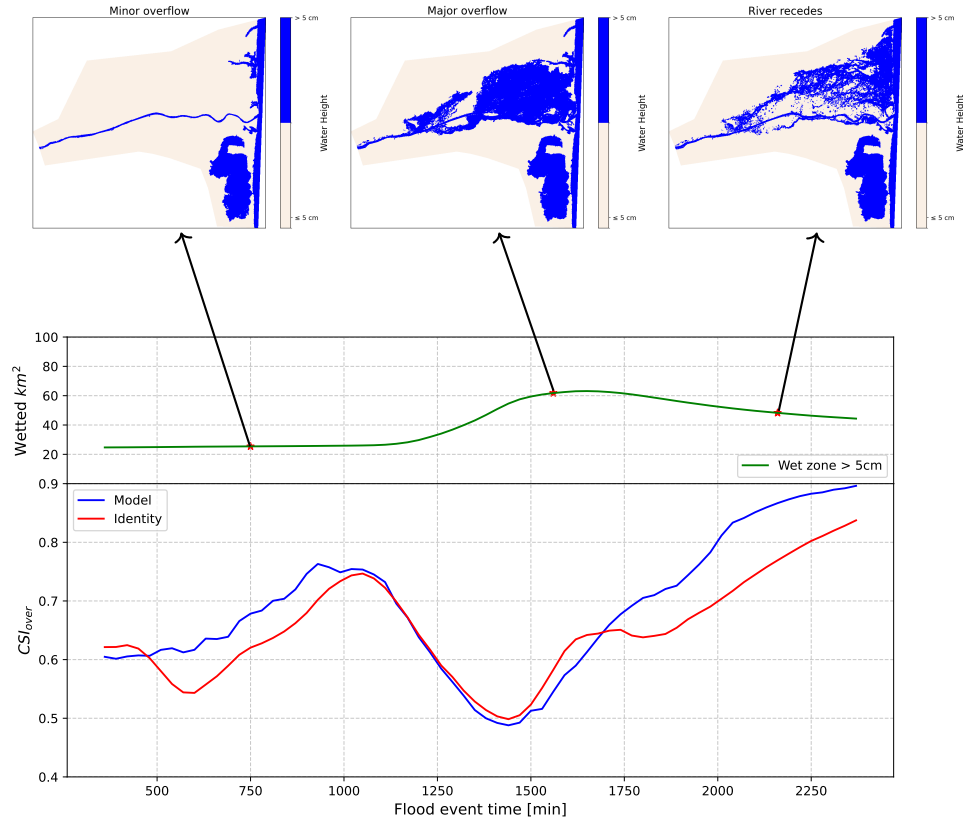


Figure 7.11: From bottom to top: CSI_{over} calculated at a prediction horizon of six hours at all possible time steps of the test flood with a peak flow of $2200 \text{ m}^3/\text{s}$. Change in wetted area of at least five centimetres for each of the time steps at which the CSI_{over} is calculated. Illustrations of the area flooded by at least 5 cm of water at different times during the flood.

7.4. Numerical results

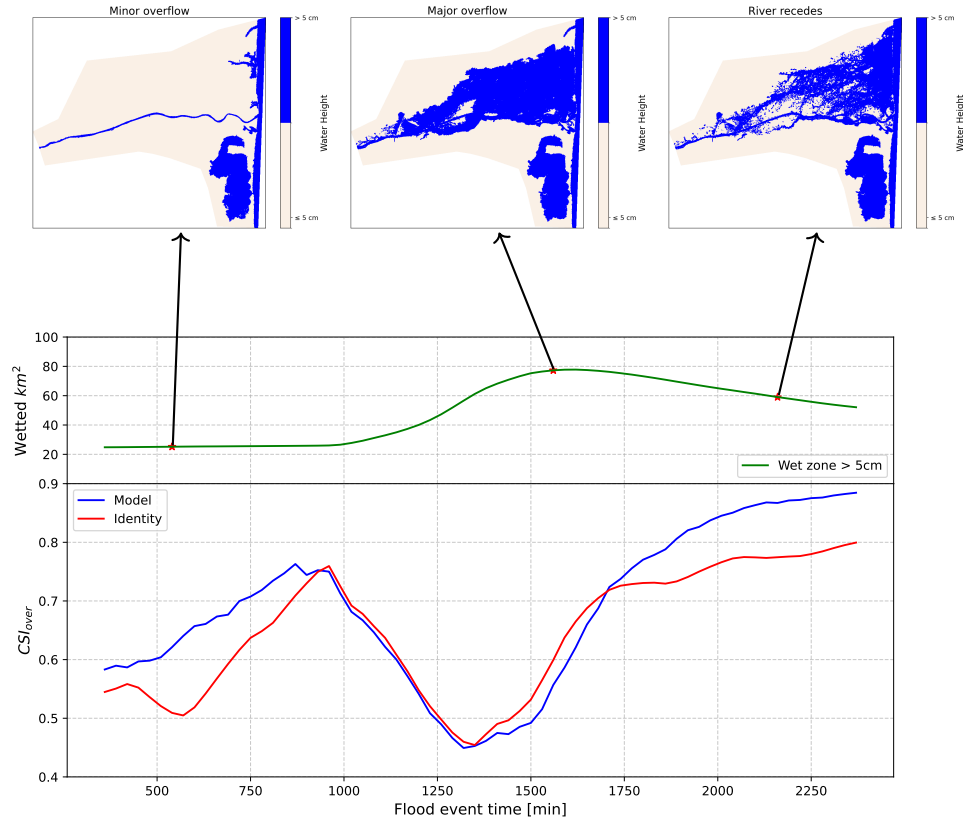


Figure 7.12: From bottom to top: CSI_{over} calculated at a prediction horizon of six hours at all possible time steps of the test flood with a peak flow of $3200 \text{ m}^3/\text{s}$. Change in wetted area of at least five centimetres for each of the time steps at which the CSI_{over} is calculated. Illustrations of the area flooded by at least 5 cm of water at different times during the flood.

7.4.2 Overflow detection

In order to investigate this inability to detect the main flood overflow, we refined our dataset by focusing on time steps closer to the onset of flooding, targeting those near the hydrograph peak. Under these revised conditions, each flood event was reduced to 31 timesteps spaced at 30-minute intervals.

In Fortunato et al. [2022] the authors introduce the principle of high-accuracy labels. The idea is to use high-resolution simulation results and interpolate them onto coarser meshes where we train our model. Using this concept, we projected our fine results in a bi-linear fashion onto the coarse meshes used to construct the multimesh. We thus obtain four new datasets on the $\times 2$, $\times 4$, $\times 8$ and $\times 16$ meshes. When interpolating our high-resolution data onto a coarser mesh, we inevitably face the risk of spectrum aliasing. However, the flood data is relatively smooth, as shown by the Fourier transforms of the test floods 7.18, which are dominated by low-frequency components. Moreover, our bilinear interpolation scheme imparts a mild smoothing effect (though it is not a perfect low-pass filter), which further mitigates aliasing. We therefore chose not to apply any additional filtering, although implementing a dedicated filtering step could still be beneficial. Using these interpolated datasets, we trained one model per dataset utilizing the available multimesh with edges created from all the coarser meshes. Each training was conducted for 2000 epochs under the same hyperparameter settings 7.3.5. For consistency in comparing model outputs, we then interpolated the predictions to a 700×700 regular grid, retaining only values within the domain.

The difference between the resolutions used by the models leads to a first effect: a large difference in inference times for predictions 7.2.

	x16	x8	x4	x2	x1
Inference Speed (s)	0.015	0.026	0.043	0.120	0.380

Table 7.2: Inference speed (in seconds) for a one-step (30 minutes) prediction, for each model resolution.

As can be seen from the global metrics in figures (Figs. 7.13a, 7.13b), the network trained on the finest mesh still has a clear advantage when it comes to one-step forecasting. However, the longer the prediction horizon, the less obvious this advantage becomes (figures of L_1 error for u and v can be found in the appendix in Figs. 8.3 and 8.4). We even observe better results in the global CSI on the model with a mesh 8 times coarser than on the fine mesh (Fig. 7.13b).

This impression is further supported by examining the flood maps (for water depths greater than 5cm) generated by each model over a 9-hour window, starting from the first time step of the flood with a peak flow of $3200 \text{ m}^3/\text{s}$ (Fig. 7.14). The coarser models more accurately capture the initial overflows during the first 5 hours, but their predictions become less stable beyond that point.

7.4. Numerical results

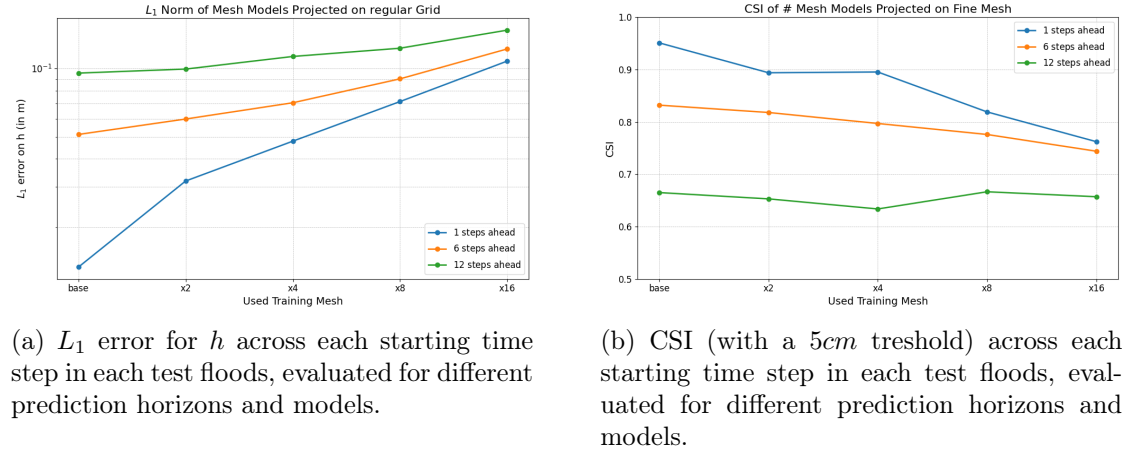


Figure 7.13: Metrics for each model on a regular grid

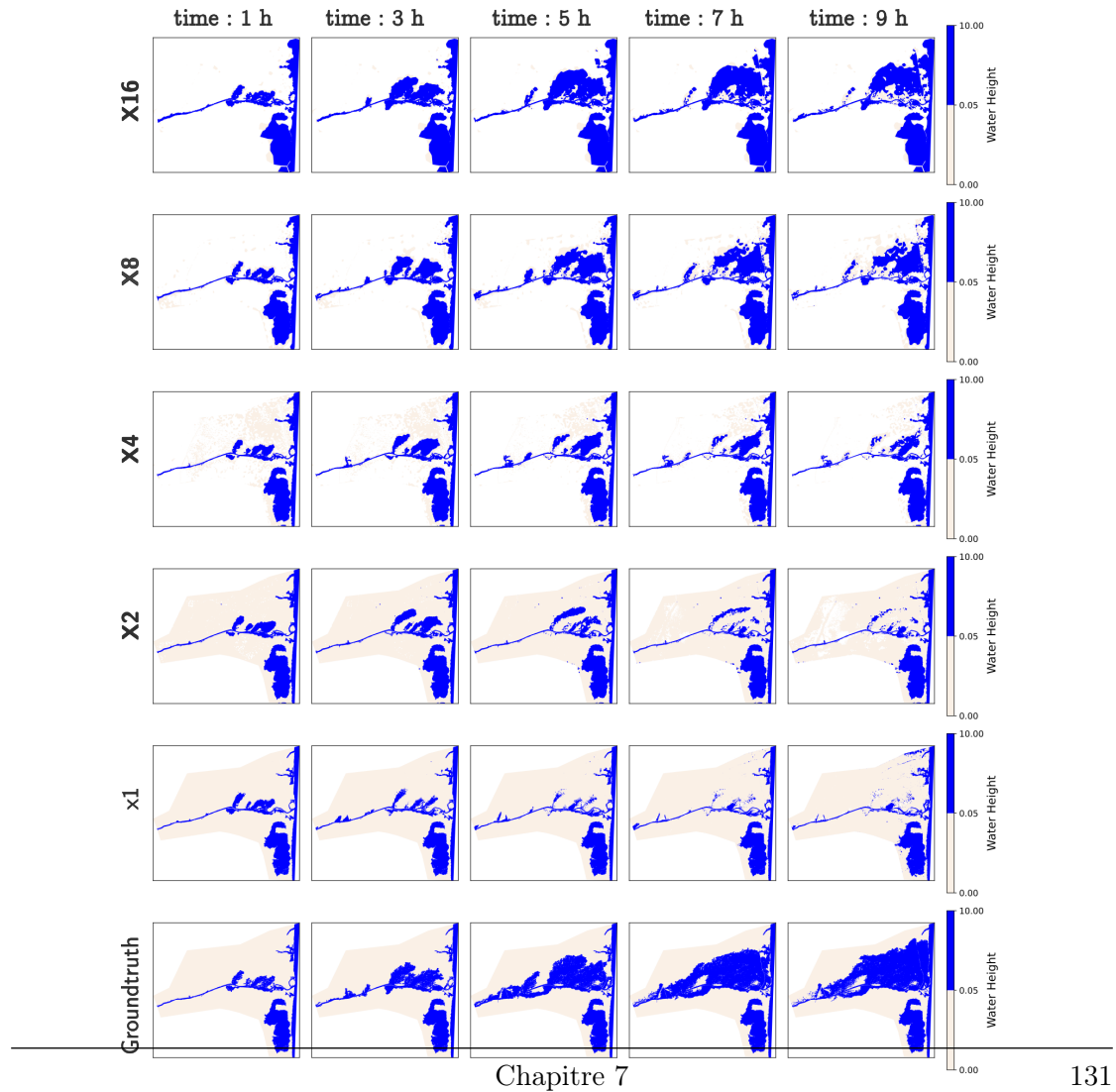


Figure 7.14: Comparison of floodmaps produced by each model and the ground-truth reference for test flood with a peak flow of $3200 \text{ m}^3/\text{s}$. All predictions are unrolled starting from the first timestep of the reduce dataset.

As before, we use the mask of the initially wetted area (5cm water depth) to delimit the approximate flood zone (area outside the mask). The CSI_{over} is then calculated in order to assess the ability of our model to predict the overall dynamics of the flood. These results are presented in Figs. 7.15, 7.16 and 7.17.

For networks trained on the finest meshes ($\times 1$, $\times 2$, even $\times 4$), the same limitations observed previously reappear: when the identity has poor performance due to large changes, these models tend to obtain results similar to those of the identity, thus struggling to correctly predict large variations. On the other hand, when the changes are more moderate, these models perform very well.

In contrast, for models using coarser meshes ($\times 8$, $\times 16$), performance is significantly better than that of the identity when changes are marked (even if they are not perfect). However, their relative effectiveness decreases compared to the finer models when the changes become less frequent, as can be seen in the test flood with a small peak (Fig. 7.15), where their superiority is less significant.

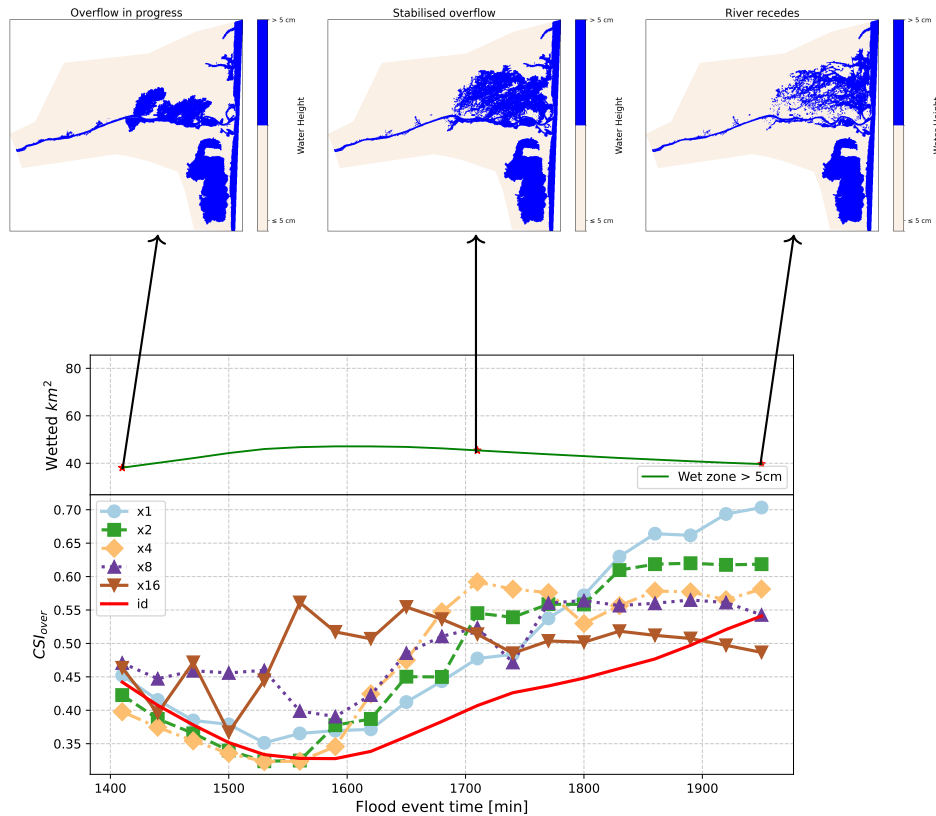


Figure 7.15: From bottom to top: CSI_{over} calculated for all the trained models at a prediction horizon of six hours at all possible time steps of the test flood with a peak flow of $1600 \text{ m}^3/\text{s}$. Change in wetted area of at least five centimetres for each of the time steps at which the CSI_{over} is calculated. Illustrations of the area flooded by at least 5 cm of water at different times during the flood.

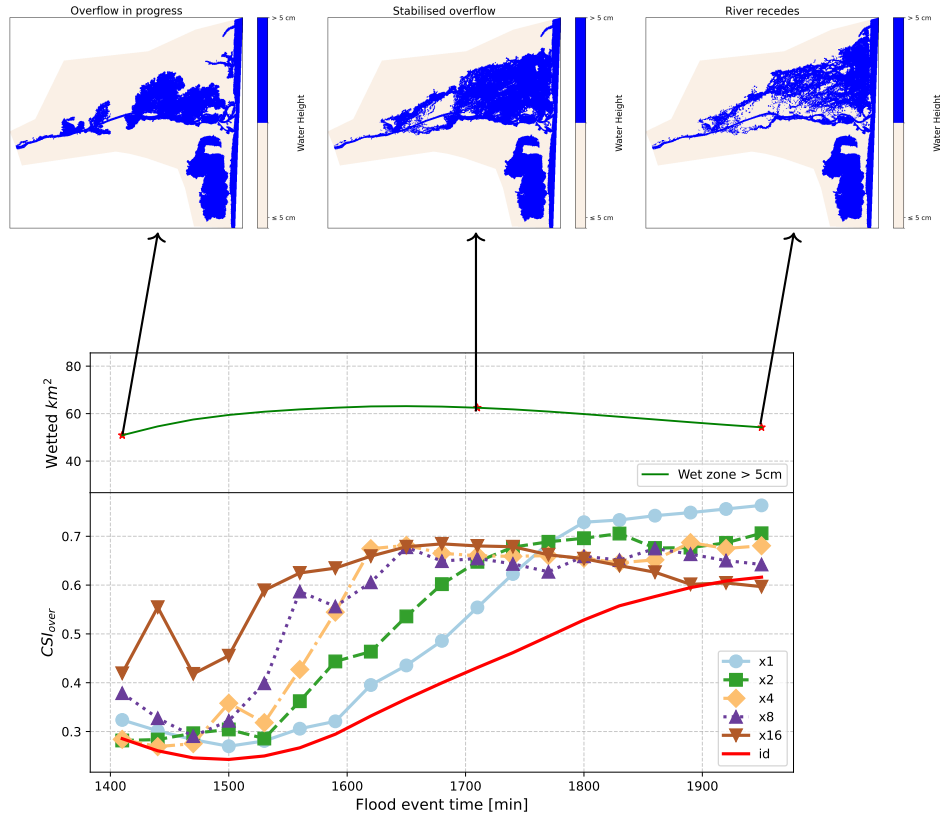


Figure 7.16: From bottom to top: CSI_{over} calculated for all the trained models at a prediction horizon of six hours at all possible time steps of the test flood with a peak flow of $2200 \text{ m}^3/\text{s}$. Change in wetted area of at least five centimetres for each of the time steps at which the CSI_{over} is calculated. Illustrations of the area flooded by at least 5 cm of water at different times during the flood.

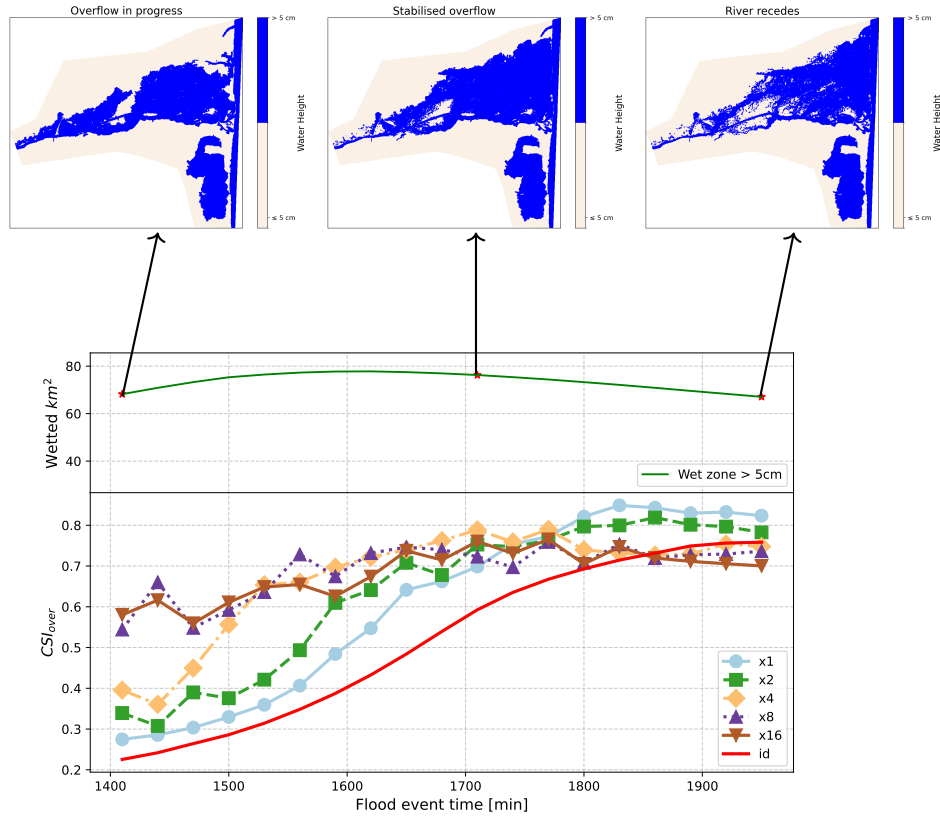


Figure 7.17: From bottom to top: CSI_{over} calculated for all the trained models at a prediction horizon of six hours at all possible time steps of the test flood with a peak flow of $3200 \text{ m}^3/\text{s}$. Change in wetted area of at least five centimetres for each of the time steps at which the CSI_{over} is calculated. Illustrations of the area flooded by at least 5 cm of water at different times during the flood.

7.4.3 Frequential analysis

To broaden our analysis, we conducted a frequency study on the model outputs for the $t + 6$ hours forecasts in the test flood scenarios. Specifically, we projected each model's predictions onto a regular grid, where the distance between consecutive points in both the x and y directions was set to match the size of the smallest edge in the chosen output resolution. The construction of this regular grid proceeded as follows:

1. We first determined the bounding box of the original computational domain along the x and y axes.
2. Next, a regular grid was created using a spacing equal to the minimum edge size.
3. The model outputs were then mapped onto this grid (using bilinear interpolation) for all points falling within the convex envelope of the domain; any points lying outside this envelope were assigned a value of zero.

To mitigate boundary artifacts, we applied a Hann window to the gridded data before performing a Fast Fourier Transform (FFT), thereby obtaining the frequency spectra of the model's predictions. Averaging the resulting power spectra for water height over all timesteps in the test set yields the plot in Fig. 7.18. We observe that most of the spectral magnitude is concentrated at low frequencies, with the model's spectra closely matching the distribution of the reference solution. As expected, finer-mesh models capture higher-frequency components more accurately than coarser meshes. It is worth noting that the finer model accurately captures high-frequency components up to the resolution limit imposed by the mesh representation.

To further examine how spectral characteristics evolve over time, we divided the test flood with a peak flow of $3200 \text{ m}^3/\text{s}$ into three distinct phases (early, middle, and late) and computed separate average for each phase. The resulting plots, shown in Fig. 7.19, display a semi-log scale (logarithmic on the x -axis only). Consistent with our previous visual observations, the coarser-mesh models capture low-frequency variations more effectively during the early (rapid) overflow period, while the finer models retain considerable high-frequency detail throughout the flood's progression.

Comparison of Radial Spectrum Magnitudes of the predicted water height (time average)

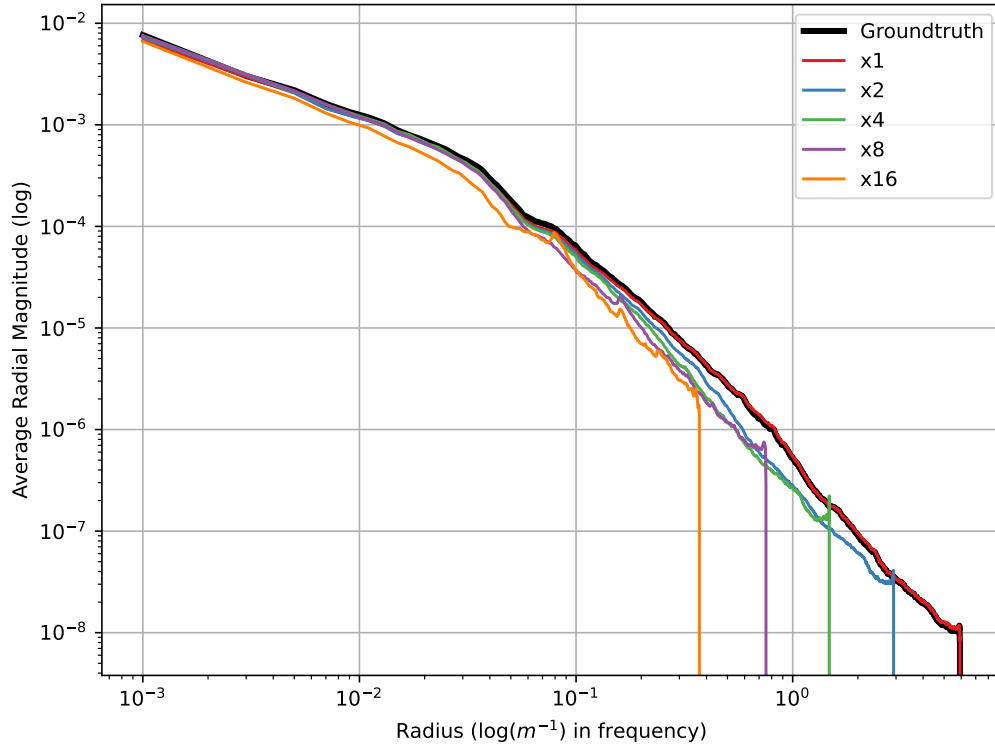


Figure 7.18: Mean frequency spectra of the predicted water heights, averaged over all timesteps in the test set.

7.4. Numerical results

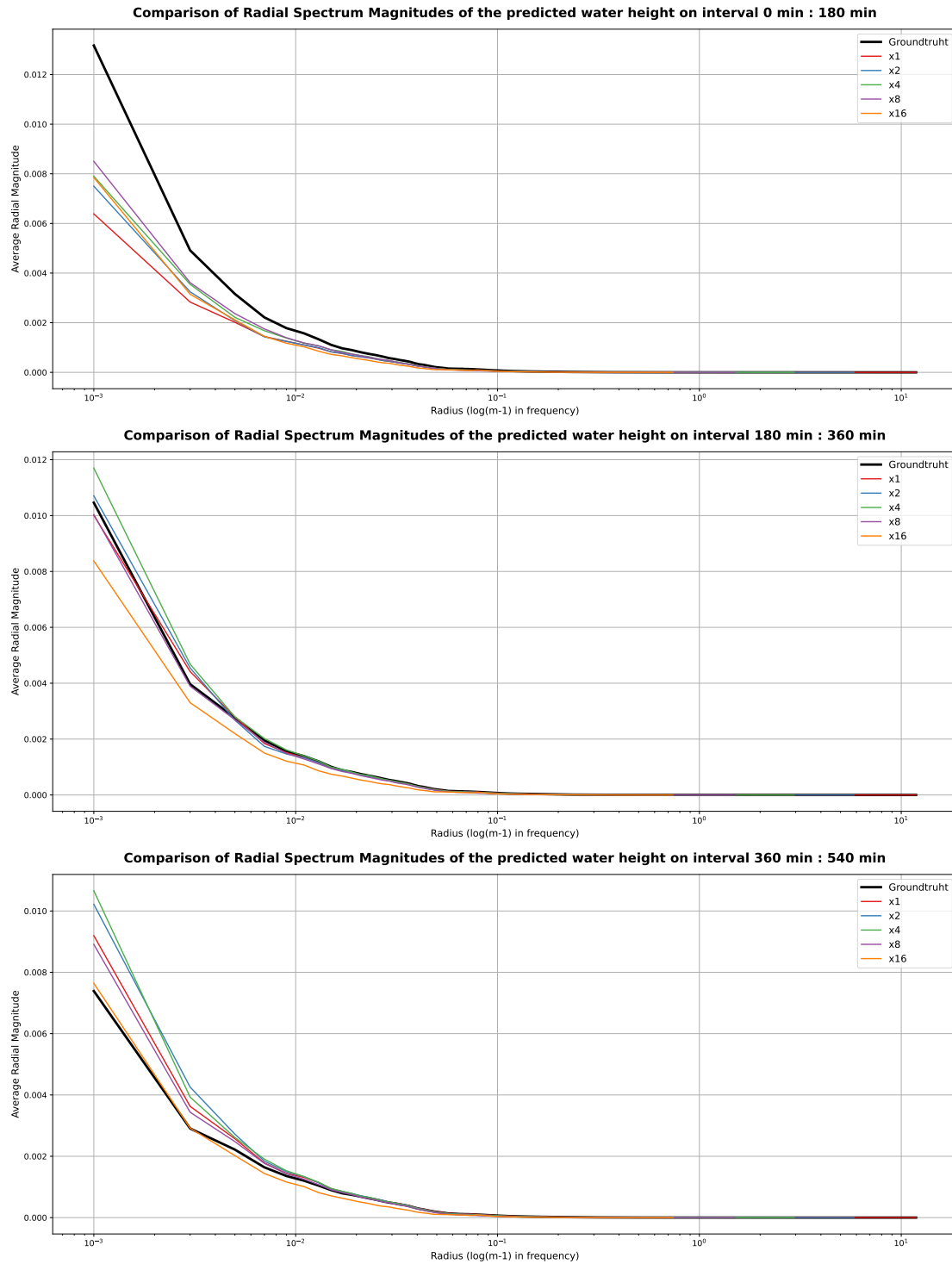


Figure 7.19: Semi-log FTT for different time slices of the $3200 \text{ m}^3/\text{s}$ peak flow test flood.

7.5 Conclusions and Perspectives

In this chapter, we demonstrated how a state-of-the-art graph neural network (Mesh-GraphNet) can be adapted for real-time flood forecasting in an industrial-scale setting. Our work included two major contributions: the development of a high-quality dataset tailored to a real-world case study of the Têt River and the adaptation of the Mesh-GraphNet approach using a multimesh strategy inspired by GraphCast. This allowed the model to efficiently handle large, irregular meshes while preserving the ability to capture complex flood dynamics.

Our initial brute-force approach, focused on fine-resolution models, showed strong performance in detecting small changes in water levels. However, it struggled to predict significant overflow events, especially during rapid flooding. By investigating this limitation, we discovered that coarser models, which aggregate information over larger areas, are better suited for detecting and predicting overflow dynamics. These coarser models demonstrated their capability to produce accurate flood maps for medium-range predictions (up to six hours ahead) in less than one second of computation time on an NVIDIA A100 80 GPU.

Despite these successes, we observed trade-offs between the fine and coarse models. Fine-resolution models are better at capturing small-scale changes and local dynamics, whereas coarser models excel in detecting and predicting larger-scale overflow events. This observation suggests that a combined multilevel approach, such as those explored in Fortunato et al. [2022], Lino et al. [2022], Bentivoglio et al. [2024], could effectively balance these strengths. Such a framework could allow the model to simultaneously capture fine details and large-scale patterns, improving overall flood forecasting accuracy.

Another important consideration is the stability of long-range predictions. While our approach achieved promising results for six-hour forecasts, extending this to longer timescales will require further refinement. Autoregressive training techniques, as proposed in Bentivoglio et al. [2024], Lam et al. [2022], could help stabilize the model during extended prediction periods and ensure consistency across multiple timesteps.

Finally, it is important to acknowledge that this study used only a small portion of the available dataset for training and evaluation. Leveraging a larger and more diverse set of flood scenarios could further enhance the model's robustness and generalizability, especially under varying boundary conditions and extreme flood events.

7.5.1 Future Directions

To build on the progress made in this work, we propose the following future directions:

- **Multilevel Modeling:** Implementing a hybrid model that combines the strengths of fine and coarse meshes to better capture both small-scale changes and large-scale overflow events.
- **Improved Long-Range Stability:** Exploring autoregressive training strategies to maintain stability and accuracy in predictions over extended forecasting periods.

- **Dataset Expansion:** Using the full dataset and additional real-world scenarios to improve model robustness and generalizability across diverse flood types and intensities.
- **Broader Testing:** Evaluating the approach across a wider variety of geographical and hydrological contexts to assess its adaptability and scalability.

By addressing these challenges, graph neural networks like MeshGraphNet could become a powerful tool for real-time flood forecasting, offering communities and stakeholders timely and accurate information to mitigate the impacts of flooding.

Chapter 8

Conclusion and perspectives

8.1 Conclusion on the contributions of this thesis

During this thesis, we investigated methods for creating deep learning-based models for flood prediction. The literature evolved significantly during this period, as highlighted in Chapters 2 and 4, with a growing interest in deep learning-based surrogate models for predicting complex physical phenomena. Two main approaches were evaluated in this work.

First, we focused on Physics-Informed Neural Networks, physics-constrained models that ensure consistency with partial differential equations. Following the framework of solving PDEs through classical methods, we introduced ideas such as multi-level optimization strategies. We proposed an original perspective on multi-level optimization methods by reinterpreting them as block-minimization methods (Chapter 5). This reformulation allowed convergence results from block methods to be adapted to the context of PINNs. We developed an alternating training strategy within a hierarchical context that synergized with frequency-aware network architectures. This approach proved effective on complex problems such as the Poisson and heat equations, reducing losses and computational costs while improving convergence. Next, we enhanced the Deep Domain Decomposition Method by introducing a coarse level to address its lack of numerical scalability (Chapter 6). This coarse network enabled faster global information propagation, improving the convergence and performance of Deep-DDM for complex configurations. Numerical experiments demonstrated that this method reduced training costs while maintaining high accuracy and was compatible with efficient parallelization. These innovations overcame computational cost limitations and positioned PINNs as a valuable tool for solving canonical PDEs such as the Poisson and heat equations.

In the second phase, we explored the use of Graph Neural Networks to design surrogate models capable of predicting floods with operational efficiency. This work, detailed in Chapter 7, relied on a database created during this thesis from numerical simulations applied to the Têt River basin in southern France. The GNN models developed demonstrated remarkable computational efficiency, reducing computation times by several orders of magnitude compared to traditional numerical methods. These models also

proved capable of capturing gradual variations in water levels as well as sudden flood events, making them particularly suited to scenarios requiring real-time responses, such as emergency management.

8.2 Perspectives for future research

Multilevel Model In our work presented in Section 7.4, we trained multiple models at different resolutions with varying capacities. High-resolution networks efficiently capture high-frequency details, whereas coarser-resolution models focus on large-scale overflow trends during unrolling. The logical next step is to combine them. Several studies have already explored these multi-resolution model combinations and propose directly applicable techniques Fortunato et al. [2022], Lino et al. [2022], Bentivoglio et al. [2024].

Unrolling Stability Ensuring stability during the unrolling process is a key challenge in developing our deep learning models, especially because our current forecasts are limited to a few hours. Several methods can help maintain stability over time, but they come with different computational and memory costs. For example, [Bentivoglio et al., 2024, Lam et al., 2022] introduce a cost function applied across multiple time steps, which improves stability but requires more memory. This approach also makes training more expensive, as it needs an initial single-step training phase and carefully adjusted learning rates. Another interesting approach is the pushforward trick [Brandstetter et al., 2023], which stabilizes the unrolling process by using a non-differentiable correction step instead of relying only on full backpropagation. This method helps reduce gradient divergence and lowers computational costs while improving long-term accuracy. Because of this, it is a promising strategy to extend forecast duration without significantly increasing the computational burden.

Data, Transfer Learning, and Computational Cost At present, we utilize only a fraction of the dataset generated for the Têt River. Increasing both the quantity and quality of data, as well as diversifying it (by incorporating various types of floods and different rivers), is the most evident approach to enhancing the robustness and performance of our models. This strategy also facilitates their transferability to other river systems. However, excessive computational costs should be avoided for both financial and practical reasons. Indeed, a company like BRLi, whose primary focus is not Deep Learning, cannot afford such investments and a path of low cost model training should be explored. The recent case of DeepSeek-R1 DeepSeek-AI et al. [2025] illustrates this path of the global deep learning research well: they manage to compete with ChatGPT on certain benchmarks at a fraction of the training cost. In AI for physics, several strategies are conceivable: for instance, Cheon et al. [2024] presents a global weather forecasting model trained at significantly lower resolutions than Lam et al. [2022] or Bi et al. [2023] while achieving results superior to traditional methods and close to those of high-resolution networks, thanks to architectural optimizations. Furthermore, alternative optimization methods, such as the one presented in Chapter 5, could enable more efficient training

by selectively targeting the most important parts of the network. Finally, GEOPIN [Xu et al., 2024], discussed in the state-of-the-art review, integrates physical constraints into surrogate models for flood forecasting. Such an approach could facilitate and accelerate retraining, particularly when the dataset or simulation conditions evolve.

Part IV

Annexes

Appendix of Paper 1

8.3 Proofs of convergence theorems

8.3.1 Proof of Theorem 5.3.1

Consider iteration k of ML-BCD and suppose that this iteration occurs in the minimization of subproblem $\mathcal{A}_i$ on the variables given by x_i . From the Lipschitz continuity of the gradient and (5.15), we obtain that

$$\begin{aligned} f(x_{k+1}) &\leq f(x_k) - \alpha \nabla_x f(x_k)^T \overline{\nabla_{x_i}^1 f(x_k)} + \frac{\alpha^2 L}{2} \|\overline{\nabla_{x_i}^1 f(x_k)}\|^2 \\ &= f(x_k) - \alpha \|\nabla_{x_i}^1 f(x_k)\|^2 + \frac{\alpha^2 L}{2} \|\nabla_{x_i}^1 f(x_k)\|^2 < f(x_k) - \frac{\alpha}{2} \|\nabla_{x_i}^1 f(x_k)\|^2. \end{aligned}$$

Since the minimization of subproblem $\mathcal{A}_i$ has not yet terminated, we must have that $\|\nabla_{x_i}^1 f(x_k)\| \geq \frac{\epsilon}{\sqrt{2}}$, and thus

$$f(x_{k+1}) < f(x_k) - \frac{\alpha}{2} \epsilon^2.$$

Summing this inequality on all iterations, we deduce that, for all k before termination of the ML-BCD algorithm,

$$f(x_0) - f_{\text{low}} \geq f(x_0) - f(x_{k+1}) = \sum_{j=0}^k \left(f(x_j) - f(x_{j+1}) \right) \geq \frac{(k+1)\alpha}{2} \epsilon^2.$$

This implies that the algorithm cannot take more than

$$\frac{2(f(x_0) - f_{\text{low}})}{\alpha \epsilon^2} - 1$$

iterations before it terminates, proving the theorem with

$$\kappa_* = \frac{2(f(x_0) - f_{\text{low}})}{\alpha} > 2L(f(x_0) - f_{\text{low}}).$$

□

8.3.2 Proof of Theorem 5.3.2

Consider iteration k of ML-BCD and suppose that this iteration occurs in the minimization of subproblem $\mathcal{A}_i$ on the variables given by x_i . From the Lipschitz continuity, using the SG update of the selected block (5.16) we obtain:

$$\mathcal{E}_{\xi_k}[f(x_{k+1})] \leq f(x_k) - \alpha \nabla_x^1 f(x_k)^T \mathcal{E}_{\xi_k}(\overline{g_i(x_k, \xi_k)}) + \frac{L\alpha^2}{2} \mathcal{E}_{\xi_k}(\|\overline{g_i(x_k, \xi_k)}\|^2). \quad (8.1)$$

Using assumption (5.17) and the bound on α we deduce that

$$\begin{aligned} \mathcal{E}_{\xi_k}[f(x_{k+1})] - f(x_k) &\leq -\alpha\mu \|\nabla_{x_i}^1 f(x_k)\|^2 + \frac{\alpha^2 L\sigma_1^2}{2} + \frac{\alpha^2 L\sigma_2^2}{2} \|\nabla_{x_i}^1 f(x_k)\|^2 \\ &= \alpha \left(\frac{\alpha L\sigma_2^2}{2} - \mu \right) \|\nabla_{x_i}^1 f(x_k)\|^2 + \frac{\alpha^2 L\sigma_1^2}{2} \\ &\leq -\frac{\alpha\mu}{2} \|\nabla_{x_i}^1 f(x_k)\|^2 + \frac{\alpha^2 L\sigma_1^2}{2}, \end{aligned}$$

and, if k is a major iteration, we have that

$$\mathcal{E}_{\xi_k}[f(x_{k+1})] - f(x_k) \leq -\frac{\alpha\mu\tau}{2} \|\nabla_x^1 f(x_k)\|^2 + \frac{\alpha^2 L\sigma_1^2}{2}. \quad (8.2)$$

Taking the total expectation gives that

$$\mathcal{E}[f(x_{k+1}) - f(x_k)] \leq -\frac{\alpha\mu}{2} \mathcal{E}(\|\nabla_{x_i}^1 f(x_k)\|^2) + \frac{\alpha^2 L\sigma_1^2}{2}. \quad (8.3)$$

Consider now a major iteration k , denoting by $x_{k+\ell}$ (for $\ell \in \{1, \dots, \ell_k\}$ and $\ell_k \leq \bar{\ell}$) the iterations performed on the selected subproblem. We deduce from (8.3) that

$$\begin{aligned} \mathcal{E}[f(x_{k+\ell_k}) - f(x_k)] &= \sum_{\ell=1}^{\ell_k-1} \mathcal{E}(f(x_{k+\ell+1}) - f(x_{k+\ell})) + \mathcal{E}(f(x_{k+1}) - f(x_k)) \\ &\leq \sum_{\ell=1}^{\ell_k-1} \left(-\frac{\alpha\mu}{2} \mathcal{E}(\|\nabla_{x_i}^1 f(x_{k+\ell})\|^2) + \frac{\alpha^2 L\sigma_1^2}{2} \right) - \frac{\alpha\mu}{2} \mathcal{E}(\|\nabla_{x_i}^1 f(x_k)\|^2) + \frac{\alpha^2 L\sigma_1^2}{2}. \end{aligned}$$

If the minimization of the subproblem is not stopped, this means that $\|g_i(x_{k+\ell}, \xi_{k+\ell})\| \geq c_\epsilon$, so that $\mathcal{E}_{\xi_{k+\ell}}(\|g_i(x_{k+\ell}, \xi_{k+\ell})\|^2) \geq c_\epsilon^2$. From (5.17), we have that

$$\|\nabla_{x_i}^1 f(x_{k+\ell})\|^2 \geq \frac{\mathcal{E}_{\xi_{k+\ell}}(\|g_i(x_{k+\ell}, \xi_{k+\ell})\|^2) - \sigma_1^2}{\sigma_2^2} \geq \frac{c_\epsilon^2 - \sigma_1^2}{\sigma_2^2}$$

and, since k is a major iteration, from the condition for the choice of the blocks we have $\|\nabla_{x_i}^1 f(x_k)\| > \tau \|\nabla_x^1 f(x_k)\|$. Thus

$$\mathcal{E}[f(x_{k+\ell_k}) - f(x_k)] \leq -\frac{\alpha\mu\bar{\ell}}{2} \frac{c_\epsilon^2 - \sigma_1^2}{\sigma_2^2} - \frac{\alpha\mu\tau}{2} \mathcal{E}(\|\nabla_x^1 f(x_k)\|^2) + \frac{\alpha^2 L\sigma_1^2(\bar{\ell} + 1)}{2}.$$

Taking into account the fact that f is bounded from below, summing up K major iterations and using the assumption $c_\epsilon > 2\sigma_1^2$ we obtain:

$$\begin{aligned} f_{\min} - f(x_1) &\leq \sum_{k=1}^K \mathcal{E}[f(x_{k+\ell_k}) - f(x_k)] \\ &\leq -\frac{\alpha\mu\tau}{2} \sum_{k=1}^K \mathcal{E}(\|\nabla_x^1 f(x_k)\|^2) + \frac{K(\bar{\ell} + 1)\alpha^2 L\sigma_1^2}{2} - \frac{\alpha\mu\bar{\ell}K}{2} \frac{\sigma_1^2}{\sigma_2^2}. \end{aligned}$$

Thus

$$\mathcal{E}\left(\sum_{k=1}^K \frac{\|\nabla_x^1 f(x_k)\|^2}{K}\right) \leq \frac{2(f(x_1) - f_{\min})}{\alpha\mu K\tau} + \frac{\alpha L\sigma_1^2}{\mu\tau} + \frac{\bar{\ell}\sigma_1^2}{\tau} \left(\frac{\alpha L}{\mu} - \frac{1}{\sigma_2^2}\right) \quad (8.4)$$

$$\leq \frac{2(f(x_1) - f_{\min})}{\alpha\mu K\tau} + \frac{\alpha L\sigma_1^2}{\mu\tau} \quad (8.5)$$

where the last inequality follows from the fact that $\bar{\ell}\sigma_1^2 \left(\frac{\alpha L}{\mu} - \frac{1}{\sigma_2^2}\right) < 0$ by the assumption on α . Taking $\bar{\ell}$ large we can thus improve the bound, as long as ϵ remains larger than the variance. $\square$

8.4 A Mscale FAML variant and its performance

8.4.1 Another frequency-aware architecture

This appendix reports on tests conducted using a frequency-aware network architecture based on MscaleDNN defined in [Liu et al. \[2020\]](#), instead of the WWP networks used in Section 5.5. A large part of the success of Mscale networks is due to their wavelet-inspired activation functions. These compactly-supported have good scale separation properties and are constructed so that the bandwidth of their Fourier transform increases with their input scaling. Several activation functions have been proposed in [Liu et al. \[2020\]](#), but the most efficient one from a practical point of view has been introduced in [Li et al. \[2020b\]](#) and is given by

$$\text{s2ReLU}(z) = \sin(2\pi z)\text{ReLU}(-(z-1))\text{ReLU}(z), \quad (8.6)$$

where z is the input scaling. This is a continuous function which decays faster and has better localization property in the frequency domain than the previously proposed sRelu. The amplitude peaks of the differently scaled s2ReLU in the frequency domain are well separated in the Fourier domain. They are indicated by black stars in Figure 8.1, and we observe their expected monotonic growth with scaling.

8.4. A Mscale FAML variant and its performance

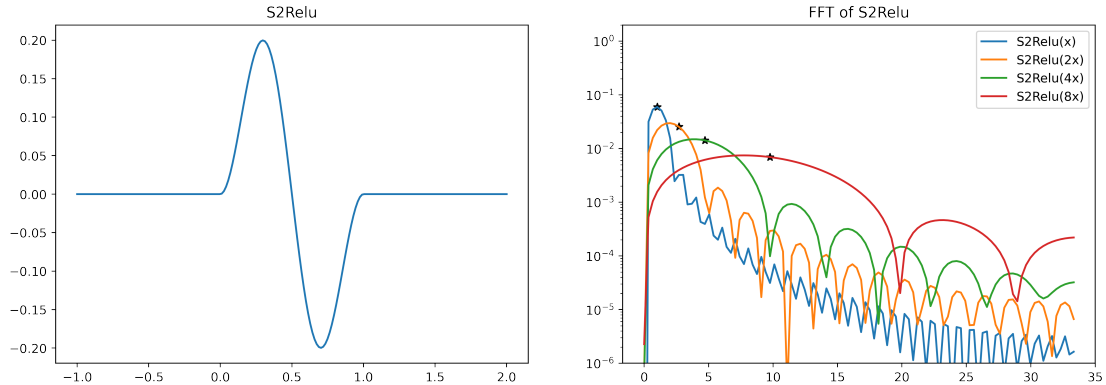


Figure 8.1: The s2Relu activation function (left) and its Fourier transform for several scalings (right). The black stars highlight the amplitude peaks.

As in Section 5.5, we use these functions to create a neural network composed of a sum of subnetworks each targeting a different frequency range (analogously to Figure 5.5). In this modified architecture, the lower frequency networks still use tanh activation functions while the higher frequency networks now use (8.6). For each subnetwork, the first activation function is a *SFM* function associated with a fixed input scaling, as in standard Mscales networks. The details of this architecture are given in Table 8.1. Each of the subnetworks is composed of 3 hidden layers of 100 neurons each.

	input scaling	First Activation	Other Activations
subnetwork 1	(0.5,1,1.5,...,10)	SFM(1.0)	tanh
subnetwork 2	(11,12,...,29,30)	SFM(0.5)	s2ReLU
subnetwork 3	(31,32,...,50,51)	SFM(0.5)	s2ReLU
subnetwork 4	(51,52,...,69,70)	SFM(0.5)	s2ReLU

Table 8.1: Details of the parallel Mscale architecture

Associated with the ML-BCD algorithm (exactly as in Section 5.5), this modified architecture defines an Mscale variant of the FAML approach.

8.4.2 Numerical results

We tested this approach using the same experimental setup and methodology as that of Section 5.5 and again compared its performance to that the standard single level training applied on the complete network. The results are reported in Table 8.2.

8.4. A Mscale FAML variant and its performance

Problem	Budget	MSE	MSE FAML	Loss	Loss FAML
Test Pb 1	10000	1.33E-05	2.76E-06	2.49E-01	7.98E-03
	19000	1.85E-06	2.51E-06	4.53E-03	2.50E-03
Test Pb 2	10000	1.22E-05	3.87E-07	1.73E-01	8.14E-04
	19000	6.11E-07	2.74E-08	1.22E-05	3.87E-07
Test Pb 3	10000	1.06E-05	1.85E-07	2.80E-01	1.06E-03
	19000	4.09E-07	1.23E-08	6.17E-04	2.09E-04
Test Pb 4	10000	1.56E-05	1.98E-07	2.84E-01	1.04E-03
	19000	5.24E-07	1.21E-08	6.26E-04	1.85E-04

Table 8.2: Best value of the median loss and associated MSE for standard single-level and Mscale-FAML training (10 independent runs)

As was the case when using P-WWP networks, the Mscale FAML variant produced lower losses than the standard training in each case. The differences are particularly significant for a small computational budget where the improvement is of several orders of magnitude. This is also almost always the case for the associated MSE. These results thus remain excellent, despite the fact that our complexity theory does not formally cover the Mscale FAML variant because of the non-smoothness of (8.6).

Appendix of Chapter 4

8.5 Telemac .cas file

```
1 /-----
2 / TELEMAC2D Version v7p2 Jun 10, 2022
3 / nom inconnu
4 /-----
5
6 /-----
7 / ENVIRONNEMENT DE CALCUL
8 /-----
9
10 TITRE : 'Group_1_peak_1000'
11
12 FICHIER DES PARAMETRES           ='cas'
13 FICHIER DE GEOMETRIE             ='maillage_3.slf'
14 FICHIER DES CONDITIONS AUX LIMITE='cli'
15 FICHIER DES FRONTIERES LIQUIDES  ='hydrographs_Group_1_peak_1000.liq'
16 /FICHIER FORTRAN                 ='fortran2.f'
17 FICHIER DES RESULTATS            ='Group_1_peak_1000.res'
18 PROCESSEURS PARALLELES           =28
19
20 /-----
21 / HYDRO
22 /-----
23
24 DEBITS IMPOSES                   =0;0
25 OPTION POUR LES FRONTIERES LIQUIDES =1;1
26 PROFILS DE VITESSE               =1;4
27 COTES IMPOSEES                   =0;0
28
29 /-----
30 / ENTREES-SORTIES, GRAPHIQUES ET LISTING
31 /-----
32
33 VARIABLES POUR LES SORTIES GRAPHIQUES =U,V,S,H,MAXZ
34 PAS DE TEMPS VARIABLE             =NON
35 PAS DE TEMPS                       =1
36 PERIODE DE SORTIE LISTING          =1800
37 DUREE DU CALCUL                    =144000
38 PERIODE POUR LES SORTIES GRAPHIQUES =1800
39 NOMBRE DE TABLEAUX PRIVES         =1
40
41 /-----
42 / CONDITIONS INITIALES
```

```

43 /-----
44
45 FICHIER DU CALCUL PRECEDENT          ='Tet_init.res'
46 SUITE DE CALCUL                     =OUI
47 REMISE A ZERO DU TEMPS              =OUI
48
49 /-----
50 /                                OPTIONS NUMERIQUES
51 /-----
52
53 BILAN DE MASSE                      =OUI
54 LOI DE FROTTEMENT SUR LE FOND        =3
55 COEFFICIENT DE DIFFUSION DES VITESSES =1.E-4
56 MODELE DE TURBULENCE                =1
57 COEFFICIENT DE FROTTEMENT            =15
58 PRECISION DU SOLVEUR                 =1.E-8
59 FORME DE LA CONVECTION                =1;5
60 TRAITEMENT DU SYSTEME LINEAIRE        =2
61 NOMBRE DE COURANT SOUHAITE            =1
62 EQUATIONS                            ='SAINT-VENANT EF'
63 IMPLICITATION POUR LA HAUTEUR         =1
64
65 /-----
66 /                                SOLVEUR
67 /-----
68
69 SOLVEUR                              =1
70 MAXIMUM D'ITERATIONS POUR LE SOLVEUR =500
71
72 /-----
73 /                                BANCS DECOUVRANT
74 /-----
75
76 BANCS DECOUVRANTS                    =OUI
77 OPTION DE TRAITEMENT DES BANCS DECOUVRANTS =1
78 TRAITEMENT DES HAUTEURS NEGATIVES        =2
79 CORRECTION DE CONTINUITE                 =OUI
80 OPTION DE SUPG                           =0;0;0;0
81 COMPATIBILITE DU GRADIENT DE SURFACE LIBRE =0.8
82 MASS-LUMPING SUR H                       =1
83 MASS-LUMPING SUR LA VITESSE               =1
84 PROFONDEUR LIMITE POUR PROCEDURE DE RESSUYAGE =0.1
85 /
86 &ETA
87 &FIN

```

8.6 Speed measurements

8.6.1 Multimesh MeshGraphNet

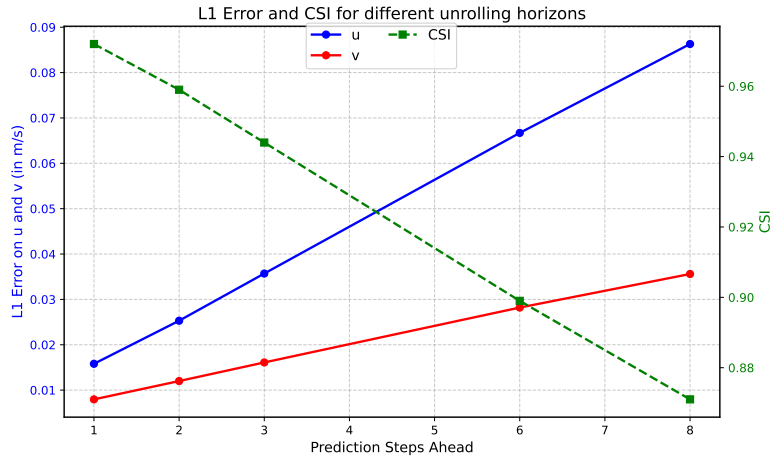


Figure 8.2: Evolution of L1 error on u (blue), v (red) and CSI (green) on test floods as a function of the number of unrolling steps under the same conditions as Fig. 7.7

8.6.2 Overflow detection

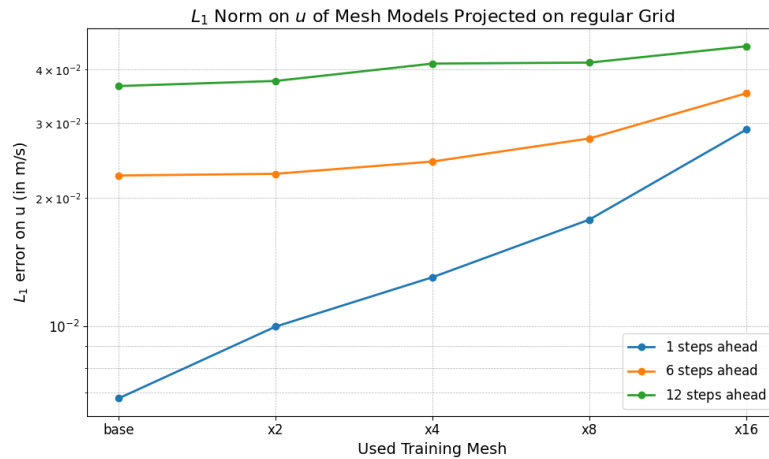


Figure 8.3: L_1 error for u across each starting time step in each test floods, evaluated for different prediction horizons and models under the same conditions as Fig. 7.13.

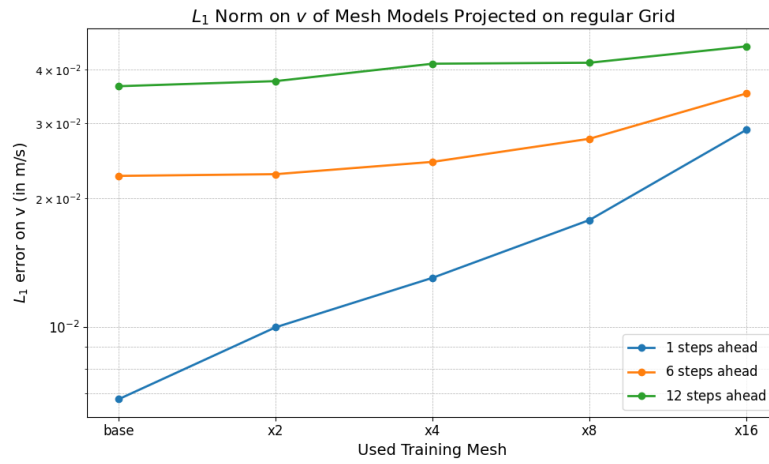


Figure 8.4: L_1 error for v across each starting time step in each test floods, evaluated for different prediction horizons and models under the same conditions as Fig. 7.13.

Bibliography

Economic insured losses caused by floods worldwide from 2013 to 2023. URL <https://www.statista.com/statistics/1326526/economic-losses-floods-worldwide-by-type/>.

Grégoire Allaire. *Numerical analysis and optimization: an introduction to mathematical modelling and numerical simulation*. OUP Oxford, 2007.

VS Amaral, Roberto Andreani, Ernesto G Birgin, Diaulas S Marcondes, and José Mario Martínez. On complexity and convergence of high-order coordinate descent algorithms for smooth nonconvex box-constrained minimization. *Journal of Global Optimization*, 84(3):527–561, 2022.

Dzmitry Bahdanau. Neural machine translation by jointly learning to align and translate. *arXiv preprint arXiv:1409.0473*, 2014.

Yohai Bar-Sinai, Stephan Hoyer, Jason Hickey, and Michael P Brenner. Learning data-driven discretizations for partial differential equations. *Proceedings of the National Academy of Sciences*, 116(31):15344–15349, 2019.

Klaus-Jürgen Bathe. *Finite element procedures*. Klaus-Jurgen Bathe, 2006.

Peter W Battaglia, Jessica B Hamrick, Victor Bapst, Alvaro Sanchez-Gonzalez, Vini-
cius Zambaldi, Mateusz Malinowski, Andrea Tacchetti, David Raposo, Adam Santoro,
Ryan Faulkner, et al. Relational inductive biases, deep learning, and graph networks.
arXiv preprint arXiv:1806.01261, 2018.

Stefania Bellavia, Francesco Della Santa, and Alessandra Papini. Alternate training of shared and task-specific parameters for multi-task neural networks. *arXiv preprint arXiv:2312.16340*, 2023.

Yoshua Bengio, Patrice Simard, and Paolo Frasconi. Learning long-term dependencies with gradient descent is difficult. *IEEE transactions on neural networks*, 5(2):157–166, 1994.

Roberto Bentivoglio, Elvin Isufi, Sebastian Nicolaas Jonkman, and Riccardo Taormina. Deep learning methods for flood mapping: a review of existing applications and future

- research directions. *Hydrology and Earth System Sciences Discussions*, 2022:1–50, 2022.
- Roberto Bentivoglio, Elvin Isufi, Sebastiaan Nicolas Jonkman, and Riccardo Taormina. Rapid spatio-temporal flood modelling via hydraulics-based graph neural networks. *Hydrology and Earth System Sciences*, 27(23):4227–4246, 2023a.
- Roberto Bentivoglio, Elvin Isufi, Sebastiaan Nicolas Jonkman, and Riccardo Taormina. Rapid spatio-temporal flood modelling via hydraulics-based graph neural networks. *Hydrology and Earth System Sciences*, 27(23):4227–4246, 2023b.
- Roberto Bentivoglio, Elvin Isufi, Sebastiaan Nicolas Jonkman, and Riccardo Taormina. Multi-scale hydraulic graph neural networks for flood modelling. *EGUsphere*, 2024: 1–28, 2024.
- Kaifeng Bi, Lingxi Xie, Hengheng Zhang, Xin Chen, Xiaotao Gu, and Qi Tian. Accurate medium-range global weather forecasting with 3d neural networks. *Nature*, 619(7970): 533–538, 2023.
- Christopher M Bishop. *Pattern recognition and machine learning by Christopher M. Bishop*. Springer Science+ Business Media, LLC, 2006.
- Alfio Borzì and Karl Kunisch. A globalization strategy for the multigrid solution of elliptic optimal control problems. *Optimisation Methods and Software*, 21(3):445–459, 2006.
- Léon Bottou, Frank E Curtis, and Jorge Nocedal. Optimization methods for large-scale machine learning. *SIAM review*, 60(2):223–311, 2018.
- Johannes Brandstetter, Daniel Worrall, and Max Welling. Message passing neural pde solvers, 2023. URL <https://arxiv.org/abs/2202.03376>.
- William L Briggs, Van Emden Henson, and Steve F McCormick. *A multigrid tutorial*. SIAM, 2000.
- Tom B Brown. Language models are few-shot learners. *arXiv preprint arXiv:2005.14165*, 2020.
- Gary W Brunner. Hec-ras river analysis system. hydraulic reference manual. version 1.0. 1997.
- Henri Calandra, Serge Gratton, Elisa Riccietti, and Xavier Vasseur. On high-order multilevel optimization strategies. *SIAM Journal on Optimization*, 31(1):307–330, 2021.
- Coralia Cartis, Nicholas IM Gould, and Philippe L Toint. *Evaluation Complexity of Algorithms for Nonconvex Optimization: Theory, Computation and Perspectives*. SIAM, 2022.

- Lei Chen, Xiaohui Zhong, Feng Zhang, Yuan Cheng, Yinghui Xu, Yuan Qi, and Hao Li. Fuxi: A cascade machine learning forecasting system for 15-day global weather forecast, 2023. URL <https://arxiv.org/abs/2306.12873>.
- Tianping Chen and Hong Chen. Universal approximation to nonlinear operators by neural networks with arbitrary activation functions and its application to dynamical systems. *IEEE transactions on neural networks*, 6(4):911–917, 1995.
- Minjong Cheon, Yo-Hwan Choi, Seon-Yu Kang, Yumi Choi, Jeong-Gil Lee, and Daehyun Kang. Karina: An efficient deep learning model for global weather forecast, 2024. URL <https://arxiv.org/abs/2403.10555>.
- Haibo Chu, Wenyan Wu, Quan J Wang, Rory Nathan, and Jiahua Wei. An ann-based emulation modelling framework for flood inundation modelling: Application, challenges and future directions. *Environmental Modelling & Software*, 124:104587, 2020.
- Junyoung Chung, Sungjin Ahn, and Yoshua Bengio. Hierarchical multiscale recurrent neural networks. *arXiv preprint arXiv:1609.01704*, 2016.
- Richard Courant, Kurt Friedrichs, and Hans Lewy. Über die partiellen differenzengleichungen der mathematischen physik. *Mathematische annalen*, 100(1):32–74, 1928.
- Richard Courant et al. Variational methods for the solution of problems of equilibrium and vibrations. *Bulletin Of The American Mathematical Society*, pages 1–1, 1943.
- George Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of control, signals and systems*, 2(4):303–314, 1989.
- F Moukalled L Mangani M Darwish. *The finite volume method in computational fluid dynamics*. 2016.
- Tim De Ryck, Samuel Lanthaler, and Siddhartha Mishra. On the approximation of functions by tanh neural networks. *Neural Networks*, 143:732–750, 2021.
- Tim De Ryck, Ameya D Jagtap, and Siddhartha Mishra. Error estimates for physics-informed neural networks approximating the navier–stokes equations. *IMA Journal of Numerical Analysis*, 44(1):83–119, 2024.
- DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, Bing Xue, Bingxuan Wang, Bochao Wu, Bei Feng, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Qu, Hui Li, Jianzhong Guo, Jia Shi Li, Jiawei Wang, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, J. L. Cai, Jiaqi Ni, Jian Liang, Jin Chen, Kai

- Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Liang Zhao, Litong Wang, Liyue Zhang, Lei Xu, Leyi Xia, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Meng Li, Miaojun Wang, Mingming Li, Ning Tian, Panpan Huang, Peng Zhang, Qiancheng Wang, Qinyu Chen, Qiushi Du, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, R. J. Chen, R. L. Jin, Ruyi Chen, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shengfeng Ye, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, S. S. Li, Shuang Zhou, Shaoqing Wu, Shengfeng Ye, Tao Yun, Tian Pei, Tianyu Sun, T. Wang, Wangding Zeng, Wanjia Zhao, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, W. L. Xiao, Wei An, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, X. Q. Li, Xiangyue Jin, Xiaojin Shen, Xiaosha Chen, Xiaowen Sun, Xiaoxiang Wang, Xinnan Song, Xinyi Zhou, Xianzu Wang, Xinxia Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Yang Zhang, Yanhong Xu, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Yu, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yuan Ou, Yudian Wang, Yue Gong, Yuheng Zou, Yujia He, Yunfan Xiong, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Y. X. Zhu, Yanhong Xu, Yanping Huang, Yaohui Li, Yi Zheng, Yuchen Zhu, Yunxian Ma, Ying Tang, Yukun Zha, Yuting Yan, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhicheng Ma, Zhigang Yan, Zhiyu Wu, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Zizheng Pan, Zhen Huang, Zhipeng Xu, Zhongyu Zhang, and Zhen Zhang. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning, 2025. URL <https://arxiv.org/abs/2501.12948>.
- Michaël Defferrard, Xavier Bresson, and Pierre Vandergheynst. Convolutional neural networks on graphs with fast localized spectral filtering. *Advances in neural information processing systems*, 29, 2016.
- Victorita Dolean, Pierre Jolivet, and Frédéric Nataf. *An introduction to domain decomposition methods: algorithms, theory, and parallel implementation*. SIAM, 2015.
- Victorita Dolean, Alexander Heinlein, Siddhartha Mishra, and Ben Moseley. Finite basis physics-informed neural networks as a schwarz domain decomposition method. In *International Conference on Domain Decomposition Methods*, pages 165–172. Springer, 2022.
- Victorita Dolean, Alexander Heinlein, Siddhartha Mishra, and Ben Moseley. Multilevel domain decomposition-based architectures for physics-informed neural networks. *Computer Methods in Applied Mechanics and Engineering*, 429:117116, 2024.
- Alexey Dosovitskiy. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2020.
- Ghazal Farhani, Alexander Kazachek, and Boyu Wang. Momentum diminishes the effect

- of spectral bias in physics-informed neural networks. *arXiv preprint arXiv:2206.14862*, 2022.
- Meire Fortunato, Tobias Pfaff, Peter Wirsberger, Alexander Pritzel, and Peter Battaglia. Multiscale meshgraphnets. *arXiv preprint arXiv:2210.00612*, 2022.
- Kai Fukami, Koji Fukagata, and Kunihiro Taira. Super-resolution reconstruction of turbulent flows with machine learning. *Journal of Fluid Mechanics*, 870:106–120, 2019.
- Kai Fukami, Koji Fukagata, and Kunihiro Taira. Machine-learning-based spatio-temporal super resolution reconstruction of turbulent flows. *Journal of Fluid Mechanics*, 909:A9, 2021.
- M Gharbi, A Soualmia, D Dartus, and L Masbernat. Comparison of 1d and 2d hydraulic models for floods simulation on the medjerda river in tunisia. *J. Mater. Environ. Sci*, 7(8):3017–3026, 2016.
- Justin Gilmer, Samuel S Schoenholz, Patrick F Riley, Oriol Vinyals, and George E Dahl. Neural message passing for quantum chemistry. In *International conference on machine learning*, pages 1263–1272. PMLR, 2017.
- Allen A Goldstein. Optimization of lipschitz continuous functions. *Mathematical Programming*, 13:14–22, 1977.
- Ian Goodfellow. Deep learning, 2016.
- Somdatta Goswami, Minglang Yin, Yue Yu, and George Em Karniadakis. A physics-informed variational deepoNet for predicting crack path in quasi-brittle materials. *Computer Methods in Applied Mechanics and Engineering*, 391:114587, 2022.
- Serge Gratton. Second-order convergence properties of trust-region methods using incomplete curvature information, with an application to multigrid optimization. *Journal of Computational Mathematics*, pages 676–692, 2006.
- Serge Gratton, Annick Sartenaer, and Philippe Toint. On recursive multiscale trust-region algorithms for unconstrained minimization. In *Oberwolfach Reports: Optimization and Applications*. 2005.
- Serge Gratton, Annick Sartenaer, and Philippe L Toint. Recursive trust-region methods for multiscale nonlinear optimization. *SIAM Journal on Optimization*, 19(1):414–444, 2008.
- Serge Gratton, Valentin Mercier, Elisa Ricciotti, and Philippe L Toint. A block-coordinate approach of multi-level optimization with an application to physics-informed neural networks. *Computational Optimization and Applications*, 89(2):385–417, 2024.

- Daniel Greenfeld, Meirav Galun, Ronen Basri, Irad Yavneh, and Ron Kimmel. Learning to optimize multigrid pde solvers. In *International Conference on Machine Learning*, pages 2415–2423. PMLR, 2019.
- Christian Groß and Rolf Krause. On the globalization of aspin employing trust-region control strategies—convergence analysis and numerical examples. *arXiv preprint arXiv:2104.05672*, 2021.
- Haiwen Guan, Troy Arcomano, Ashesh Chattopadhyay, and Romit Maulik. Lucie: A lightweight uncoupled climate emulator with long-term stability and physical consistency for $\mathcal{O}(1000)$ -member ensembles, 2024. URL <https://arxiv.org/abs/2405.16297>.
- John Guibas, Morteza Mardani, Zongyi Li, Andrew Tao, Anima Anandkumar, and Bryan Catanzaro. Adaptive fourier neural operators: Efficient token mixers for transformers. *arXiv preprint arXiv:2111.13587*, 2021.
- Eldad Haber and Lars Ruthotto. Stable architectures for deep neural networks. *Inverse problems*, 34(1):014004, 2017.
- Eldad Haber, Lars Ruthotto, Elliot Holtham, and Seong-Hwan Jun. Learning across scales—multiscale methods for convolution neural networks. In *Proceedings of the AAAI conference on artificial intelligence*, volume 32, 2018.
- Wolfgang Hackbusch. *Multi-grid methods and applications*, volume 4. Springer Science & Business Media, 1995.
- Zhongkai Hao, Jiachen Yao, Chang Su, Hang Su, Ziao Wang, Fanzhi Lu, Zeyu Xia, Yichi Zhang, Songming Liu, Lu Lu, et al. Pinnacle: A comprehensive benchmark of physics-informed neural networks for solving pdes. *arXiv preprint arXiv:2306.08827*, 2023.
- Trevor Hastie, Robert Tibshirani, and Jerome Friedman. An introduction to statistical learning. 2009.
- Jacob Helwig, Xuan Zhang, Cong Fu, Jerry Kurtin, Stephan Wojtowytsch, and Shuiwang Ji. Group equivariant fourier neural operators for partial differential equations. *arXiv preprint arXiv:2306.05697*, 2023.
- Leon Herrmann and Stefan Kollmannsberger. Deep learning in computational mechanics: a review. *Computational Mechanics*, pages 1–51, 2024.
- Jean-Michel Hervouet. *Hydrodynamics of free surface flows: modelling with the finite element method*. John Wiley & Sons, 2007.
- JM Hervouet. The weak form of the method of characteristics, an amazing advection scheme. *XXth TELEMAC-MASCARET*, pages 1–6, 2013.

- Magnus Rudolph Hestenes, Eduard Stiefel, et al. *Methods of conjugate gradients for solving linear systems*, volume 49. NBS Washington, DC, 1952.
- Langwen Huang, Lukas Gianinazzi, Yuejiang Yu, Peter D Dueben, and Torsten Hoefler. Diffda: a diffusion model for weather-scale data assimilation. *arXiv preprint arXiv:2401.05932*, 2024.
- Arthur Jacot, Franck Gabriel, and Clément Hongler. Neural tangent kernel: Convergence and generalization in neural networks. *Advances in neural information processing systems*, 31, 2018.
- Ameya D Jagtap, Kenji Kawaguchi, and George Em Karniadakis. Locally adaptive activation functions with slope recovery for deep and physics-informed neural networks. *Proceedings of the Royal Society A*, 476(2239):20200334, 2020a.
- Ameya D Jagtap, Kenji Kawaguchi, and George Em Karniadakis. Adaptive activation functions accelerate convergence in deep and physics-informed neural networks. *Journal of Computational Physics*, 404:109136, 2020b.
- Deok-Kyu Jang, Kyungsoo Kim, and Hyea Hyun Kim. Partitioned neural network approximation for partial differential equations enhanced with lagrange multipliers and localized loss functions. *Computer Methods in Applied Mechanics and Engineering*, 429:117168, 2024.
- Michael I Jordan, Tianyi Lin, and Manolis Zampetakis. On the complexity of deterministic nonsmooth and nonconvex optimization. *arXiv preprint arXiv:2209.12463*, 2022.
- I-Feng Kao, Jia-Yi Liou, Meng-Hsin Lee, and Fi-John Chang. Fusing stacked autoencoder and long short-term memory for regional multistep-ahead flood inundation forecasts. *Journal of Hydrology*, 598:126371, 2021.
- Tsung-Wei Ke, Michael Maire, and Stella X Yu. Multigrid neural architectures. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 6665–6673, 2017.
- Jacob Devlin Ming-Wei Chang Kenton and Lee Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of naacL-HLT*, volume 1, page 2. Minneapolis, Minnesota, 2019.
- Ehsan Kharazmi, Zhongqiang Zhang, and George Em Karniadakis. Variational physics-informed neural networks for solving partial differential equations. *arXiv preprint arXiv:1912.00873*, 2019.
- Diederik P Kingma. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014a.

- Diederik P Kingma. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014b.
- Thomas N Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907*, 2016.
- Axel Klawonn, Martin Lanser, and Janine Weber. Machine learning and domain decomposition methods-a survey. *Computational Science and Engineering*, 1(1):2, 2024.
- Dmitrii Kochkov, Jamie A Smith, Ayya Alieva, Qing Wang, Michael P Brenner, and Stephan Hoyer. Machine learning-accelerated computational fluid dynamics. *Proceedings of the National Academy of Sciences*, 118(21):e2101784118, 2021.
- Siyu Kong and AS Lewis. The cost of nonconvexity in deterministic nonsmooth optimization. *Mathematics of Operations Research*, 2023.
- Tugrul Konuk and Jeffrey Shragge. Physics-guided deep learning using fourier neural operators for solving the acoustic vti wave equation. In *82nd EAGE Annual Conference & Exhibition*, volume 2021, pages 1–5. European Association of Geoscientists & Engineers, 2021.
- Alena Kopanicáková and Rolf Krause. Globally convergent multilevel training of deep residual networks. *SIAM Journal on Scientific Computing*, 45(3):S254–S280, 2022.
- Nikola Kovachki, Zongyi Li, Burigede Liu, Kamyar Azizzadenesheli, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Neural operator: Learning maps between function spaces with applications to pdes. *Journal of Machine Learning Research*, 24(89):1–97, 2023.
- Alex Krizhevsky. One weird trick for parallelizing convolutional neural networks. *arXiv preprint arXiv:1404.5997*, 2014.
- Akshay Kulkarni, Adarsha Shivananda, Anoosh Kulkarni, and Dilip Gudivada. The chatgpt architecture: An in-depth exploration of openai’s conversational language model. In *Applied Generative AI for Beginners: Practical Knowledge on Diffusion Models, ChatGPT, and Other LLMs*, pages 55–77. Springer, 2023.
- Remi Lam, Alvaro Sanchez-Gonzalez, Matthew Willson, Peter Wirsberger, Meire Fortunato, Ferran Alet, Suman Ravuri, Timo Ewalds, Zach Eaton-Rosen, Weihua Hu, et al. Graphcast: Learning skillful medium-range global weather forecasting. *arXiv preprint arXiv:2212.12794*, 2022.
- Simon Lang, Mihai Alexe, Matthew Chantry, Jesper Dramsch, Florian Pinault, Baudouin Raoult, Mariana C. A. Clare, Christian Lessig, Michael Maier-Gerber, Linus Magnusson, Zied Ben Bouallègue, Ana Prieto Nemesio, Peter D. Dueben, Andrew Brown, Florian Pappenberger, and Florence Rabier. Aifs – ecmwf’s data-driven forecasting system, 2024. URL <https://arxiv.org/abs/2406.01465>.

- Samuel Lanthaler, Roberto Molinaro, Patrik Hadorn, and Siddhartha Mishra. Nonlinear reconstruction for operator learning of pdes with discontinuities. *arXiv preprint arXiv:2210.01074*, 2022.
- Corentin J Lapeyre, Antony Misdariis, Nicolas Cazard, Denis Veynante, and Thierry Poinso. Training convolutional neural networks to estimate turbulent sub-grid scale reaction rates. *Combustion and Flame*, 203:255–264, 2019.
- Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.
- Jae Yong Lee, Sung Woong Cho, and Hyung Ju Hwang. Hyperdeeponet: learning operator with complex target function space using the limited resources via hypernetwork. *arXiv preprint arXiv:2312.15949*, 2023.
- Ke Li, Kejun Tang, Tianfan Wu, and Qifeng Liao. D3m: A deep domain decomposition method for partial differential equations. *Ieee Access*, 8:5283–5294, 2019.
- Wuyang Li, Xueshuang Xiang, and Yingxiang Xu. Deep domain decomposition method: Elliptic problems. In *Mathematical and Scientific Machine Learning*, pages 269–286. PMLR, 2020a.
- Xi-An Li, Zhi-Qin John Xu, and Lei Zhang. A multi-scale dnn algorithm for nonlinear elliptic equations with multiple scales. *arXiv preprint arXiv:2009.14597*, 2020b.
- Xi-An Li, Zhi-Qin John Xu, and Lei Zhang. Subspace decomposition based dnn algorithm for elliptic type multi-scale pdes. *Journal of Computational Physics*, 488:112242, 2023.
- Zijie Li and Amir Barati Farimani. Graph neural network-accelerated lagrangian fluid simulation. *Computers & Graphics*, 103:201–211, 2022.
- Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Fourier neural operator for parametric partial differential equations. *arXiv preprint arXiv:2010.08895*, 2020c.
- Zongyi Li, Nikola Kovachki, Chris Choy, Boyi Li, Jean Kossaifi, Shourya Otta, Mohammad Amin Nabian, Maximilian Stadler, Christian Hundt, Kamyar Azizzadenesheli, et al. Geometry-informed neural operator for large-scale 3d pdes. *Advances in Neural Information Processing Systems*, 36, 2024a.
- Zongyi Li, Hongkai Zheng, Nikola Kovachki, David Jin, Haoxuan Chen, Burigede Liu, Kamyar Azizzadenesheli, and Anima Anandkumar. Physics-informed neural operator for learning partial differential equations. *ACM/JMS Journal of Data Science*, 1(3): 1–27, 2024b.
- Mario Lino, Stathi Fotiadis, Anil A. Bharath, and Chris D. Cantwell. Multi-scale rotation-equivariant graph neural networks for unsteady eulerian fluid dynamics. *Physics of Fluids*, 34(8):087110, 08 2022. ISSN 1070-6631. doi: 10.1063/5.0097679. URL <https://doi.org/10.1063/5.0097679>.

- Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. Swin transformer: Hierarchical vision transformer using shifted windows. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 10012–10022, 2021.
- Ziqi Liu, Wei Cai, and Zhi-Qin John Xu. Multi-scale deep neural network (mscalednn) for solving poisson-boltzmann equation in complex domains. *arXiv preprint arXiv:2007.11207*, 2020.
- Callum Lock, Oubay Hassan, Ruben Sevilla, and Jason Jones. Meshing using neural networks for improving the efficiency of computer modelling. *Engineering with Computers*, 39(6):3791–3820, 2023.
- Lu Lu, Pengzhan Jin, Guofei Pang, Zhongqiang Zhang, and George Em Karniadakis. Learning nonlinear operators via deepoanet based on the universal approximation theorem of operators. *Nature machine intelligence*, 3(3):218–229, 2021.
- Ilay Luz, Meirav Galun, Haggai Maron, Ronen Basri, and Irad Yavneh. Learning algebraic multigrid using graph neural networks. In *International Conference on Machine Learning*, pages 6489–6499. PMLR, 2020.
- Siddhartha Mishra and Roberto Molinaro. Estimates on the generalization error of physics-informed neural networks for approximating pdes. *IMA Journal of Numerical Analysis*, 43(1):1–43, 2023.
- Modulus Contributors. Nvidia modulus: An open-source framework for physics-based deep learning in science and engineering. URL <https://github.com/NVIDIA/modulus>.
- Ben Moseley, Andrew Markham, and Tarje Nissen-Meyer. Finite basis physics-informed neural networks (fbpinns): a scalable domain decomposition approach for solving differential equations. *Advances in Computational Mathematics*, 49(4):62, 2023.
- C Moulinec, C Denis, N Durand, RW Barber, DR Emerson, XJ Gu, E Razafindrakoto, R Issa, and JM Hervouet. Coupling hpc and numerical validation: accurate and efficient simulation of largescale hydrodynamic events. In *Proc. 2nd Int. Conf. on Parallel, Distributed, Grid and Cloud Computing for Engineering, BHV Topping and P. Iványi (Eds.), Civil-Comp Press, Stirlingshire, Scotland*, 2011.
- Arsha Nagrani, Shan Yang, Anurag Arnab, Aren Jansen, Cordelia Schmid, and Chen Sun. Attention bottlenecks for multimodal fusion. *Advances in neural information processing systems*, 34:14200–14213, 2021.
- Stephen G Nash. A multigrid approach to discretized optimization problems. *Optimization Methods and Software*, 14(1-2):99–116, 2000.
- Yu Nesterov. Efficiency of coordinate descent methods on huge-scale optimization problems. *SIAM Journal on Optimization*, 22(2):341–362, 2012.

- Yurii Nesterov. *Introductory lectures on convex optimization: A basic course*, volume 87. Springer Science & Business Media, 2013.
- Amin Heyrani Nobari, Justin Rey, Suhas Kodali, Matthew Jones, and Faez Ahmed. Conformal predictions enhanced expert-guided meshing with graph neural networks. *arXiv preprint arXiv:2308.07358*, 2023.
- Nvidia. Automatic mixed precision for deep learning, 2024. URL <https://developer.nvidia.com/automatic-mixed-precision>.
- Andrea Nathaly Rodriguez Orozco, Nathalie Bertrand, Lucie Pheulpin, Antonin Migaud, and Morgan Abily. Comparison between hec-ras and telemac-2d hydrodynamic models of the loire river, integrating levee breaches. In *SimHydro*, pages 41–55. Springer, 2023.
- Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems 32*, pages 8024–8035. Curran Associates, Inc., 2019. URL <http://papers.neurips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library.pdf>.
- Jaideep Pathak, Shashank Subramanian, Peter Harrington, Sanjeev Raja, Ashesh Chattopadhyay, Morteza Mardani, Thorsten Kurth, David Hall, Zongyi Li, Kamyar Aziz-zadenesheli, et al. Fourcastnet: A global data-driven high-resolution weather model using adaptive fourier neural operators. *arXiv preprint arXiv:2202.11214*, 2022.
- Azazkhan Ibrahimkhan Pathan, Prasit Girish Agnihotri, Dhruvesh Patel, and Cristina Prieto. Mesh grid stability and its impact on flood inundation through (2d) hydrodynamic hec-ras model with special use of big data platform—a study on punna river of navsari city. *Arabian Journal of Geosciences*, 15(7):659, 2022.
- Mathis Peyron. *Assimilation de données en espace latent par des techniques de deep learning*. PhD thesis, 2024. URL <http://www.theses.fr/2024TLSEP074>. Thèse de doctorat dirigée par Gürol, Selime et Gratton, Serge Informatique et Télécommunications Université de Toulouse (2023-....) 2024.
- Tobias Pfaff, Meire Fortunato, Alvaro Sanchez-Gonzalez, and Peter W Battaglia. Learning mesh-based simulation with graph networks. *arXiv preprint arXiv:2010.03409*, 2020.
- Matteo Pianforini, Susanna Dazzi, Andrea Pilzer, and Renato Vacondio. Real-time flood maps forecasting for dam-break scenarios with a transformer-based deep learning model. *Journal of Hydrology*, 635:131169, 2024.

- Michael JD Powell. On search directions for minimization algorithms. *Mathematical programming*, 4:193–201, 1973.
- Nasim Rahaman, Aristide Baratin, Devansh Arpit, Felix Draxler, Min Lin, Fred Hamprecht, Yoshua Bengio, and Aaron Courville. On the spectral bias of neural networks. In *International conference on machine learning*, pages 5301–5310. PMLR, 2019.
- Maziar Raissi, Paris Perdikaris, and George E Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational physics*, 378: 686–707, 2019.
- Narsimha Reddy Rapaka and Ravi Samtaney. An efficient poisson solver for complex embedded boundary domains using the multi-grid and fast multipole methods. *Journal of Computational Physics*, 410:109387, 2020.
- Pu Ren, Chengping Rao, Yang Liu, Jian-Xun Wang, and Hao Sun. Phycrnet: Physics-informed convolutional-recurrent network for solving spatiotemporal pdes. *Computer Methods in Applied Mechanics and Engineering*, 389:114399, 2022.
- Peter Richtárik and Martin Takáč. Iteration complexity of randomized block-coordinate descent methods for minimizing a composite function. *Mathematical Programming*, 144(1):1–38, 2014.
- Basri Ronen, David Jacobs, Yoni Kasten, and Shira Kritchman. The convergence rate of neural networks for learned functions of different frequencies. *Advances in Neural Information Processing Systems*, 32, 2019.
- Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation. In *Medical image computing and computer-assisted intervention—MICCAI 2015: 18th international conference, Munich, Germany, October 5–9, 2015, proceedings, part III 18*, pages 234–241. Springer, 2015.
- Johannes Sappl, Laurent Seiler, Matthias Harders, and Wolfgang Rauch. Deep learning of preconditioners for conjugate gradient solvers in urban water related problems. *arXiv preprint arXiv:1906.06925*, 2019.
- Hermann Amandus Schwarz. *Ueber einen Grenzübergang durch alternirendes Verfahren*. Zürcher u. Furrer, 1870.
- Jacob Seidman, Georgios Kissas, Paris Perdikaris, and George J Pappas. Nomad: Non-linear manifold decoders for operator learning. *Advances in Neural Information Processing Systems*, 35:5601–5613, 2022.
- Yeonjong Shin, Jerome Darbon, and George Em Karniadakis. On the convergence of physics informed neural networks for linear second-order elliptic and parabolic type pdes. *arXiv preprint arXiv:2004.01806*, 2020.

- Yeonjong Shin, Zhongqiang Zhang, and George Em Karniadakis. Error estimates of residual minimization using neural networks for linear pdes. *Journal of Machine Learning for Modeling and Computing*, 4(4), 2023.
- Khemraj Shukla, Ameya D Jagtap, and George Em Karniadakis. Parallel physics-informed neural networks via domain decomposition. *Journal of Computational Physics*, 447:110683, 2021.
- Xiuyu Sun, Xiaohui Zhong, Xiaoze Xu, Yuanqing Huang, Hao Li, Jie Feng, Wei Han, Libo Wu, and Yuan Qi. Fuxi weather: An end-to-end machine learning weather data assimilation and forecasting system, 2024. URL <https://arxiv.org/abs/2408.05472>.
- Matthew Tancik, Pratul Srinivasan, Ben Mildenhall, Sara Fridovich-Keil, Nithin Raghavan, Utkarsh Singhal, Ravi Ramamoorthi, Jonathan Barron, and Ren Ng. Fourier features let networks learn high frequency functions in low dimensional domains. *Advances in neural information processing systems*, 33:7537–7547, 2020.
- Vidar Thomée. From finite differences to finite elements a short history of numerical analysis of partial differential equations. In *Numerical analysis: Historical developments in the 20th century*, pages 361–414. Elsevier, 2001.
- Lai Tian and A Man-Cho So. Computing goldstein (ϵ, δ) -stationary points of lipschitz functions in $\mathcal{O}(\epsilon^{-3\delta-1})$ iterations via random conic perturbation. *arXiv preprint arXiv:2112.09002*, 2021.
- F Arend Torres, Marcello Massimo Negri, Marco Inversi, Jonathan Aellen, and Volker Roth. Lagrangian flow networks for conservation laws. *arXiv preprint arXiv:2305.16846*, 2023.
- Ulrich Trottenberg, Cornelius W Oosterlee, and Anton Schuller. *Multigrid*. Elsevier, 2000.
- Kiwon Um, Robert Brand, Yun Raymond Fei, Philipp Holl, and Nils Thuerey. Solver-in-the-loop: Learning from differentiable physics to interact with iterative pde-solvers. *Advances in Neural Information Processing Systems*, 33:6111–6122, 2020.
- A Vaswani. Attention is all you need. *Advances in Neural Information Processing Systems*, 2017.
- Anna Vaughan, Stratis Markou, Will Tebbutt, James Requeima, Wessel P. Bruinsma, Tom R. Andersson, Michael Herzog, Nicholas D. Lane, Matthew Chantry, J. Scott Hosking, and Richard E. Turner. Aardvark weather: end-to-end data-driven weather forecasting, 2024. URL <https://arxiv.org/abs/2404.00411>.
- Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. Graph attention networks. *arXiv preprint arXiv:1710.10903*, 2017.

- Simone Venturi and Tiernan Casey. Svd perspectives for augmenting deepnet flexibility and interpretability. *Computer Methods in Applied Mechanics and Engineering*, 403: 115718, 2023.
- Cyrill von Planta, Alena Kopanicáková, and Rolf Krause. Training of deep residual networks with stochastic mg/opt. *arXiv preprint arXiv:2108.04052*, 2021.
- Laura von Rueden, Sebastian Mayer, Rafet Sifa, Christian Bauckhage, and Jochen Garcke. Combining machine learning and simulation to a hybrid modelling approach: Current and future directions. In *Advances in Intelligent Data Analysis XVIII: 18th International Symposium on Intelligent Data Analysis, IDA 2020, Konstanz, Germany, April 27–29, 2020, Proceedings 18*, pages 548–560. Springer, 2020.
- Haixin Wang, Yadi Cao, Zijie Huang, Yuxuan Liu, Peiyan Hu, Xiao Luo, Zezheng Song, Wanxia Zhao, Jilin Liu, Jinan Sun, et al. Recent advances on machine learning for computational fluid dynamics: A survey. *arXiv preprint arXiv:2408.12171*, 2024.
- Minjie Wang, Da Zheng, Zihao Ye, Quan Gan, Mufei Li, Xiang Song, Jinjing Zhou, Chao Ma, Lingfan Yu, Yu Gai, et al. Deep graph library: A graph-centric, highly-performant package for graph neural networks. *arXiv preprint arXiv:1909.01315*, 2019.
- Sifan Wang, Yujun Teng, and Paris Perdikaris. Understanding and mitigating gradient pathologies in physics-informed neural networks. *arXiv preprint arXiv:2001.04536*, 2020.
- Sifan Wang, Hanwen Wang, and Paris Perdikaris. Learning the solution operator of parametric partial differential equations with physics-informed deepnets. *Science advances*, 7(40):eabi8605, 2021a.
- Sifan Wang, Hanwen Wang, and Paris Perdikaris. On the eigenvector bias of fourier feature networks: From regression to solving multi-scale pdes with physics-informed neural networks. *Computer Methods in Applied Mechanics and Engineering*, 384:113938, 2021b.
- Sifan Wang, Xinling Yu, and Paris Perdikaris. When and why pinns fail to train: A neural tangent kernel perspective. *Journal of Computational Physics*, 449:110768, 2022.
- Henning Wessels, Christian Weissenfels, and Peter Wriggers. The neural particle method—an updated lagrangian physics informed neural network for computational fluid dynamics. *Computer Methods in Applied Mechanics and Engineering*, 368:113127, 2020.
- Steffen Wiewel, Moritz Becher, and Nils Thuerey. Latent space physics: Towards learning the temporal evolution of fluid flow. In *Computer graphics forum*, volume 38, pages 71–82. Wiley Online Library, 2019.
- Stephen J Wright. Coordinate descent algorithms. *Mathematical programming*, 151(1): 3–34, 2015.

- Chao-Yuan Wu, Ross Girshick, Kaiming He, Christoph Feichtenhofer, and Philipp Krahenbuhl. A multigrid method for efficiently training video models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 153–162, 2020.
- Chenxi Wu, Min Zhu, Qinyang Tan, Yadhu Kartha, and Lu Lu. A comprehensive study of non-adaptive and residual-based adaptive sampling for physics-informed neural networks. *Computer Methods in Applied Mechanics and Engineering*, 403:115671, 2023.
- Qingsong Xu, Yilei Shi, Jonathan L Bamber, Chaojun Ouyang, and Xiao Xiang Zhu. Large-scale flood modeling and forecasting with floodcast. *Water Research*, 264:122162, 2024.
- Zhi-Qin John Xu, Yaoyu Zhang, Tao Luo, Yanyang Xiao, and Zheng Ma. Frequency principle: Fourier analysis sheds light on deep neural networks. *arXiv preprint arXiv:1901.06523*, 2019.
- Ling Yang, Zhilong Zhang, Yang Song, Shenda Hong, Runsheng Xu, Yue Zhao, Wentao Zhang, Bin Cui, and Ming-Hsuan Yang. Diffusion models: A comprehensive survey of methods and applications. *ACM Computing Surveys*, 56(4):1–39, 2023.
- Jiachen Yao, Chang Su, Zhongkai Hao, Songming Liu, Hang Su, and Jun Zhu. Multitadam: Parameter-wise scale-invariant optimizer for multiscale training of physics-informed neural networks. In *International Conference on Machine Learning*, pages 39702–39721. PMLR, 2023.
- Xi Ye and Guillaume-Alexandre Bilodeau. Video prediction by efficient transformers. *Image and Vision Computing*, 130:104612, 2023. ISSN 0262-8856. doi: <https://doi.org/10.1016/j.imavis.2022.104612>. URL <https://www.sciencedirect.com/science/article/pii/S0262885622002414>.
- Bing Yu et al. The deep ritz method: a deep learning-based numerical algorithm for solving variational problems. *Communications in Mathematics and Statistics*, 6(1): 1–12, 2018.
- Jeremy Yu, Lu Lu, Xuhui Meng, and George Em Karniadakis. Gradient-enhanced physics-informed neural networks for forward and inverse pde problems. *Computer Methods in Applied Mechanics and Engineering*, 393:114823, 2022.
- Jingzhao Zhang, Hongzhou Lin, Stefanie Jegelka, Suvrit Sra, and Ali Jadbabaie. Complexity of finding stationary points of nonconvex nonsmooth functions. In *International Conference on Machine Learning*, pages 11173–11182. PMLR, 2020.
- Yinhao Zhu, Nicholas Zabaras, Phaedon-Stelios Koutsourelakis, and Paris Perdikaris. Physics-constrained deep learning for high-dimensional surrogate modeling and uncertainty quantification without labeled data. *Journal of Computational Physics*, 394: 56–81, 2019.

Titre : Utilisation de techniques de Machine Learning pour la prédiction de crue

Mots clés : Prévion de crue, Apprentissage profond, Physique numérique, PINN, GNN

Résumé : Le changement climatique favorise les précipitations extrêmes, soulignant le besoin critique de prévisions précises et rapides des inondations afin de protéger les vies humaines, les infrastructures et l'environnement. En France seulement, les dommages annuels liés aux inondations s'élèvent en moyenne à 520 millions d'euros, ce qui souligne l'urgence de développer des outils de prévision plus efficaces. Les méthodes traditionnelles de prévision des inondations reposent sur des modèles physiques à forte intensité de calcul basés sur les équations de Barré de Saint-Venant qui sont souvent trop lents pour permettre une prise de décision en temps réel, en particulier dans les zones urbaines où des réponses rapides sont essentielles.

Cette thèse explore et présente deux paradigmes complémentaires d'apprentissage profond pour la prévision des inondations, reflétant une trajectoire de recherche adaptative et innovante. Au départ, nous nous sommes concentrés sur les réseaux de neurones informés par la physique (PINN) en raison de leur respect de la physique inhérent et de leur nature peu dépendante des données. Bien que les PINN soient très prometteurs et qu'ils aient permis des avancées significatives (comme les stratégies d'optimisation hiérarchique à plusieurs niveaux qui ont amélioré la convergence et la précision), leur transposition aux équations de Barré de Saint-Venant s'est avérée difficile malgré des améliorations méthodologiques substantielles. Les innovations comme les stratégies d'optimisation hiérarchique à plusieurs niveaux, les mécanismes informés par la fréquence et les algorithmes de décomposition de domaine ont permis de remédier à des limitations importantes et de faire progresser le domaine, en positionnant les PINN comme un outil précieux pour résoudre les EDP canoniques comme les équations de Poisson et de la chaleur. Parallèlement, nous avons poursuivi une autre approche en utilisant des réseaux de neurones en graphes (GNN) pour créer des modèles de substitution pour la simulation des inondations. Ce volet s'appuie sur des données d'inondation réelles, ce qui nous permet de produire des solutions pertinentes d'un point de vue opérationnel. Appliqués au bassin de la Têt dans le sud de la France, ces modèles GNN ont démontré une grande efficacité de calcul (plusieurs ordres de grandeur plus rapides que les méthodes numériques traditionnelles) tout en conservant une précision raisonnable. En capturant efficacement les variations de niveau d'eau et les événements d'inondation soudains, ils se sont révélés adaptés aux applications en temps réel dans les scénarios d'intervention d'urgence.

Ce travail établit une base pour les approches hybrides physique-ML dans la modélisation hydraulique tout en faisant progresser le domaine dans plusieurs aspects clés. Les modèles développés fournissent des solutions opérationnelles pour la prévision des inondations, proposant un juste milieu entre la précision physique des méthodes traditionnelles avec l'efficacité et la réactivité requises pour les interventions d'urgence. Les orientations futures de la recherche comprennent des capacités d'adaptation du modèle en temps réel, une quantification robuste de l'incertitude et une intégration transparente avec les systèmes d'alerte existants. L'ensemble de ces efforts marque une étape importante vers des systèmes de prévision des inondations plus fiables et plus réactifs, essentiels pour s'adapter aux défis posés par le changement climatique.

Title: Use of Machine Learning techniques for flood forecasting

Key words: Flood forecasting, Deep learning, Computational physics, PINN, GNN

Abstract: Climate change is intensifying extreme precipitation events, underscoring the critical need for accurate and rapid flood forecasting to protect lives, infrastructure, and the environment. In France alone, annual flood-related damages average €520 million, emphasizing the urgency of developing more efficient predictive tools. Traditional flood forecasting methods rely on computationally intensive physical models based on shallow water equations, which are often too slow for real-time decision-making, especially in urban settings where rapid responses are essential.

This research explores and advances two complementary deep learning paradigms for flood forecasting, reflecting an adaptive and innovative research trajectory. Initially, we focused on Physically-Informed Neural Networks (PINNs) due to their inherent physical constraints and data-efficient nature. Although PINNs showed significant promise and led to meaningful advancements (such as multi-level hierarchical optimization strategies that enhanced convergence and accuracy) scaling them to the shallow water equations proved challenging despite substantial methodological improvements. Innovations like multi-level hierarchical optimization strategies, frequency-informed mechanisms, and domain decomposition algorithms addressed key limitations and advanced the field, positioning PINNs as a valuable tool for solving canonical PDEs like the Poisson and heat equations.

In parallel, we pursued an alternative approach using Graph Neural Networks (GNNs) to create surrogate models for flood simulation. This pivot leveraged real flood data, enabling us to produce operationally relevant solutions. Applied to the Têt River basin in southern France, these GNN models demonstrated a great computational efficiency (orders of magnitude faster than traditional numerical methods) while maintaining a correct predictive accuracy. By effectively capturing both gradual water level changes and sudden flood events, they proved suitable for real-time applications in emergency response scenarios.

This work establishes a foundation for hybrid physics-ML approaches to hydrological modelling and advances the field in several key aspects. The developed models provide operationally relevant solutions for flood forecasting, balancing the physical accuracy of traditional methods with the efficiency and responsiveness required for emergency response. Future research directions include real-time model adaptation capabilities, robust uncertainty quantification, and seamless integration with existing warning systems. Together, these efforts represent a significant step towards more reliable and responsive flood forecasting systems, which are essential for adapting to the challenges of climate change.